关于本文档

文档名称：《高性能分布式对象存储 MINIO》

使用协议：《知识共享公共许可协议(CCPL)》

贡献者

贡献者名称	贡献度	文档变更记录	个人主页
马哥（马永亮）	主编		http://github.com/iKubernetes/
王晓春	作者	96页	http://www.wangxiaochun.com

文档协议

致谢

本文档中，部分素材参考了相关项目的文档，以及通过搜索引擎获得的内容，这里先一并向相关的贡献者表示感谢。

高性能分布式对象存储 MINIO

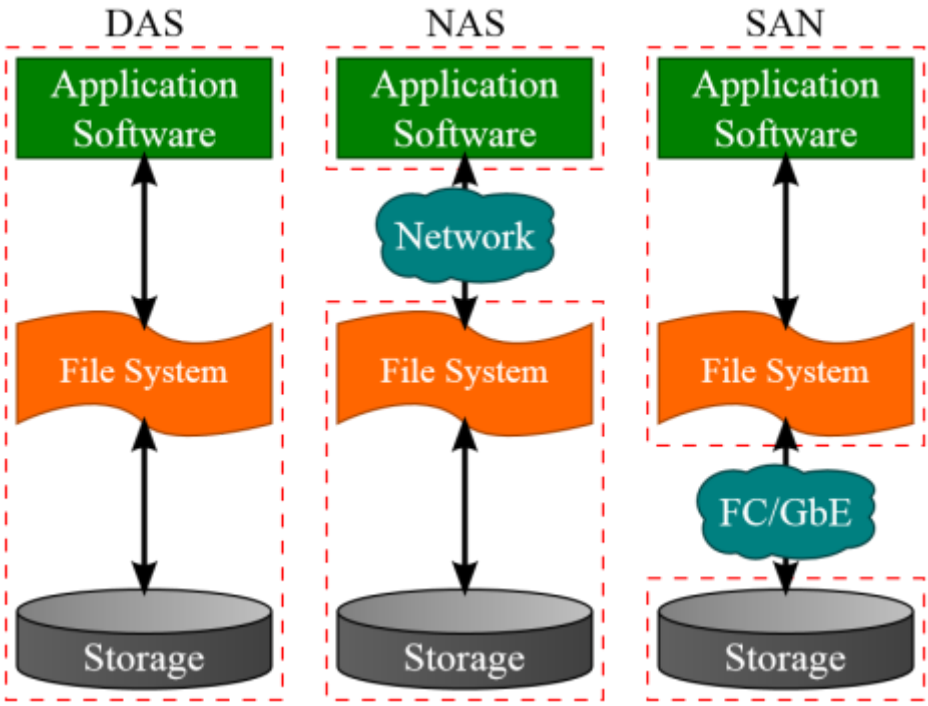
内容概述

- MINIO 介绍
- MINIO 部署
- MINIO 使用
- MINIO 故障恢复
- MINIO 扩容和缩容
- MINIO 备份和还原
- MINIO 监控

1 MINIO 介绍

1.1 分布式存储系统应用场景

1.1.1 常见的传统存储方案



- DAS

  Direct Attached Storage直连存储,储设备直接与主机服务器连接，其他主机不能使用这个存储设备

  普通的PC服务器的存储即为DAS，将外部的存储设备直接加在服务器内部来存储数据

  服务器本身容易成为系统瓶颈，服务器发生故障时，整个存储数据将不可访问。

  对于存在多个服务器的系统，设备分散，不便于管理。同时多台服务器使用DAS时，存储空间不能在服务器之间动态分配和共享，可能造成相当的资源浪费。

  这种存储方式，比较适用于小型网络结构，数据量小，对数据的传输与读取速度要求不高的场景下

  DAS 属于块设备

- NAS

  Network Attached Storage 网络附加存储

  存储系统直接接入网络，通过网络交换机，将服务器与存储连接在一起，用户可以通过TCPIP协议访问数据

  并通过标准的业界文件共享协议，如: FTP、CIFS、NFS来实现目录级的共享。

  此方式安装布署比较简单，可以即插即用，而且不依赖于操作系统，缺点就是存储的性能不太好

  NAS是“文件级”数据共享

- SAN

  Storage Area Network 存储区域网络

  交换机将磁盘阵列等存储设备与相关服务器连接起来的高速专用存储网络。最开始用的是FC-SAN，也就是基于FC设备及通信协议的存储区域网络。

  因为光纤网络的成本太高，后面又发展出一种基于以太网的SAN存储形式, 利用TCP/IP协议实现了对SCSI协议的封装

  相对于其它方式,更易于扩容、集中管理、更好的性能以及更灵活的备份策略。

  SAN提供的是裸设备，属于Block级共享

1.1.2 OS Object storage 对象存储

1.1.2.1 什么是 Object storage

对象存储（Object Storage）是一种用于存储和管理海量非结构化数据的存储架构，常用于存放图片、视频、备份文件、日志等大文件。与传统的块存储和文件存储不同，对象存储将数据作为**对象（Object）**来管理，而不是文件系统中的文件或块设备中的块。每个对象包含：

1. **数据**：需要存储的内容（例如一张图片或一个视频）。
2. **元数据**：关于这个数据的描述性信息，比如创建时间、文件类型等。
3. **唯一的标识符**：每个对象都有一个唯一的ID，通过它可以直接访问该对象。

1.1.2.2 OS 对象存储应用场景

当前互联网上有海量的非结构化数据的需要存储,如下数据

- 电商网站: 海量商品图片
- 视频网站: 海量视频文件
- 音乐网站: 海量音频文件
- 社交网站: 海量图片
- 云平台: 大量的虚拟机镜像文件
- 网盘: 海量文件

以上海量的数据如果是基于传统的SAN和NAS的方式进行存储，无论是性能,容器和可用性方方面面都已经力所不及。

当前常用的解决方案是基于对象存储服务(Object Storage Service, OSS)实现

1.1.2.3 对象存储服务 OSS (Object Storage Service) 特点

- 对象存储是一种分布式、基于Restful方式操作的存储服务
- 数据作为单独的对象进行存储
- 水平扩展性：对象存储的架构能够轻松扩展，支持存储海量数据，在上传大文件时，OSS会采用分片上传, 通常用于云存储服务（如Amazon S3、Google Cloud Storage等）
- 高可用性和持久性：对象存储的数据被自动复制并分布在多个物理位置，减少数据丢失的可能。
- 无层级结构：与文件存储的层次目录结构不同，对象存储是扁平结构，不直接使用目录树，所有对象存放在同一个命名空间中，可以通过其唯一ID直接访问。可以在互联网任何位置通过URL方式对每个对象进行存储和访问,但不支持挂载
- 适合大文件存储：适合存储非结构化数据：比如：视频、图片、日志、文本文件等，通常用于云计算环境中
- 适合读多写少的场景

1.1.2.4 OSS 分类

分为公有云的OSS服务和私有云的OSS服务

- 公有云: 阿里云的OSS，腾讯云的COS，天翼云的OSS，亚马逊的S3(2006年3月)，Microsoft Azure Blob存储等公有云提供OSS服务
- 私有云: MinIO, Ceph RGW

1.2 阿里云的OSS

阿里云等公有云提供了OSS服务

```
https://www.aliyun.com/product/oss
```

什么是 OSS

```
https://static-aliyun-doc.oss-cn-hangzhou.aliyuncs.com/file-manage-files/zh-CN/20220727/ttob/OSS%E6%96%B0%E7%89%88%E4%BA%A7%E5%93%81%E4%BB%8B%E7%BB%8D%EF%BC%88%E4%B8%AD%E6%96%87%EF%BC%89.mp4
```

阿里云对象存储OSS（Object Storage Service）是一款海量、安全、低成本、高可靠的云存储服务，可提供99.9999999999%（12个9）的数据持久性，99.995%的数据可用性。多种存储类型供选择，全面优化存储成本。

OSS具有与平台无关的RESTful API接口，您可以在任何应用、任何时间、任何地点存储和访问任意类型的数据。

阿里云 OSS重要特性

- 版本控制

  版本控制是针对存储空间（Bucket）级别的数据保护功能。开启版本控制后，针对数据的覆盖和删除操作将会以历史版本的形式保存下来。您在错误覆盖或者删除文件（Object）后，能够将Bucket中存储的Object恢复到任意时刻的历史版本。更多信息，请参见版本控制概述。

- Bucket Policy

  Bucket拥有者可通过Bucket Policy授权不同用户以何种权限访问指定的OSS资源。例如您需要进行跨账号或对匿名用户授权访问或管理整个Bucket或Bucket内的部分资源，或者需要对同账号下的不同RAM用户授予访问或管理Bucket资源的不同权限，例如只读、读写或完全控制的权限等。关于配置Bucket Policy的具体操作，请参见通过Bucket Policy授权用户访问指定资源。

- 跨区域复制

  跨区域复制（Cross-Region Replication）是跨不同OSS数据中心（地域）的Bucket自动、异步（近实时）复制Object，它会将Object的创建、更新和删除等操作从源存储空间复制到不同区域的目标存储空间。跨区域复制功能满足Bucket跨区域容灾或用户数据复制的需求。更多信息，请参见跨区域复制概述。

- 数据加密

  服务器端加密：上传文件时，OSS对收到的文件进行加密，再将得到的加密文件持久化保存；下载文件时，OSS自动将加密文件解密后返回给用户，并在返回的HTTP请求Header中，声明该文件进行了服务器端加密。更多信息，请参见服务器端加密。

  客户端加密：将文件上传到OSS之前在本地进行加密。更多信息，请参见客户端加密。

- 数据永久保存

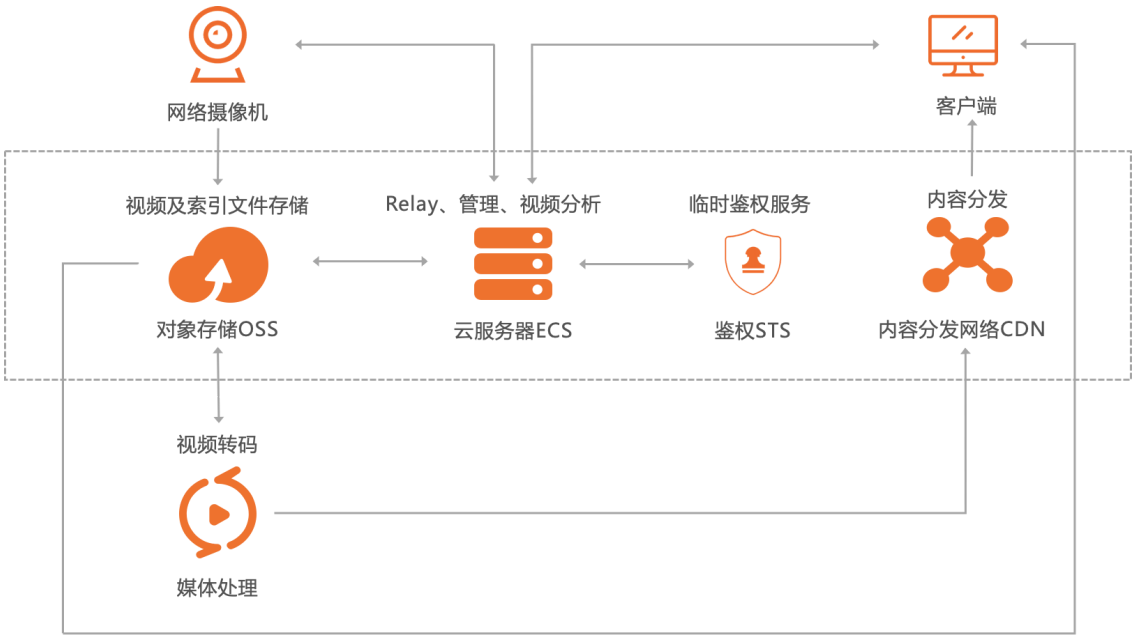
  OSS默认永久保存上传到Bucket的数据

阿里云 OSS应用场景

- 图片和音视频等应用的海量存储

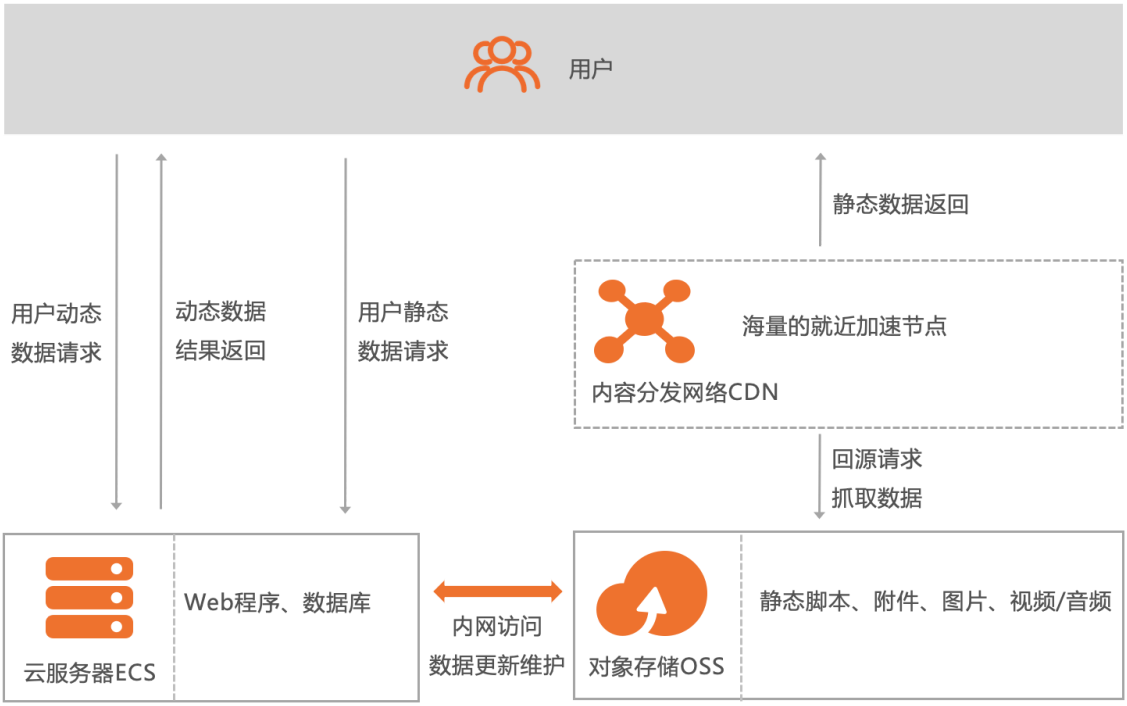
  OSS可用于图片、音视频、日志等海量文件的存储。各种终端设备、Web网站程序、移动应用可以直接向OSS写入或读取数据。

  OSS支持流式写入和文件写入两种方式。



- 网页或者移动应用的静态和动态资源分离

  利用海量互联网带宽，OSS可以实现海量数据的互联网并发下载。OSS提供原生的传输加速功能，支持上传加速、下载加速，提升跨国、跨洋数据上传、下载的体验。同时，OSS也可以结合CDN产品，提供静态内容存储、分发到边缘节点的解决方案，利用CDN边缘节点缓存的数据，提升同一个文件被同一地区客户大量重复并发下载的体验。



- 云端数据处理

  上传文件到OSS后，可以配合智能媒体管理服务和图片处理服务进行云端的数据处理。

1.3 MinIO介绍

MinIO 是GlusterFS创始人之一Anand Babu Periasamy发布的新的开源项目。

MinIO 是一个用 GoLang 语言开发的基于 GNU AGPL v3 开源协议的对象存储服务(Object Storage Service, OSS)

对象存储服务是一种海量、安全、低成本、高可靠的云存储服务，适合存放任意类型的文件。

对象存储服务支持容量和处理能力弹性扩展，多种存储类型供选择，全面优化存储成本。

MinIO 兼容亚马逊S3云存储服务接口，非常适合于存储大容量非结构化的数据，例如:图片、视频、日志文件、备份数据和容器/虚拟机镜像等

MinIO 支持的一个对象文件可以是任意大小，从几kb到最大5T不等

MinIO 是一个非常轻量的服务， 可以很简单的和其他应用的结合,也支持各种操作系统,比如:Linux,Windows,Mac等

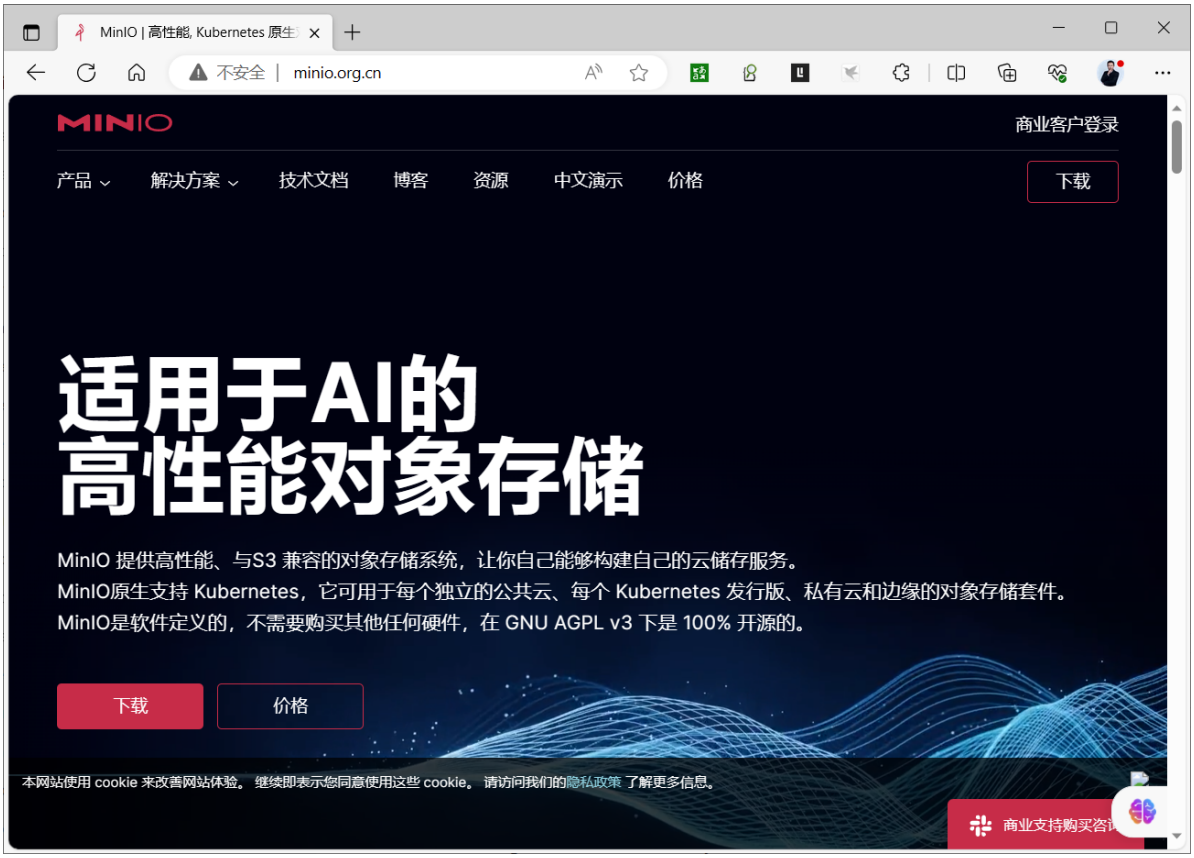
对于中小型企业的对象存储，如果不选择公有云存储，那么Minio是个不错的选择

MinIO 除了直接作为对象存储使用,还可以作为云上对象存储服务的网关层，无缝对接到Amazon S3、MicroSoft Azure。

国内的阿里巴巴、腾讯、百度、中国联通、华为、中国移动等9000多家企业也都在使用MinIO产品。

官网

```
https://min.io/
http://minio.org.cn/
https://github.com/minio/minio
```



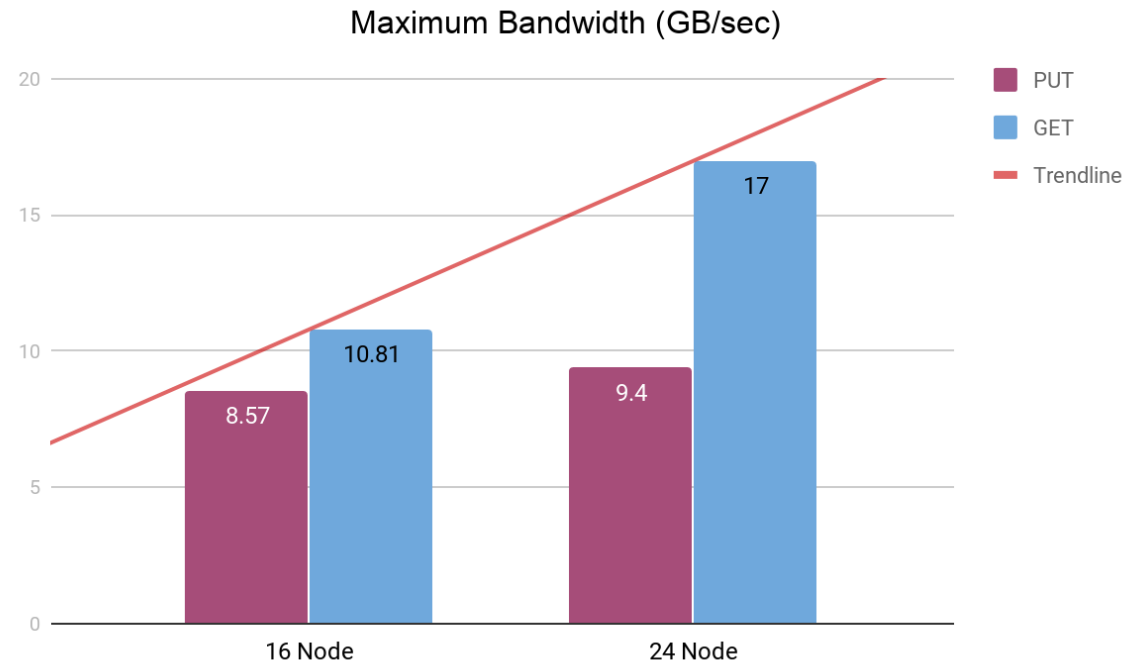
MinIO 特点

- MinIO 提供高性能、与S3 兼容的对象存储系统，让你自己能够构建自己的云储存服务。
- MinIO 使用和部署非常简单，可以让您在最快的时间内实现下载到生产环境的部署。
- MinIO 读写性能优异,高性能MinIO 是世界上最快的对象存储，没有之一。在 32 个 NVMe 驱动器节点和 100Gbe 网络上发布的 GET/PUT 结果超过 325 GiB/秒和 165 GiB/秒
- MinIO 支持的对象文件小到几kb到最大5T,并实现了数据的高可用
- MinIO 原生支持 Kubernetes，它可用于每个独立的公共云、每个 Kubernetes 发行版、私有云和边缘的对象存储套件。
- MinIO 是软件定义的，不需要购买其他任何硬件，在 GNU AGPL v3 下是 100% 开源的

MinIO 性能

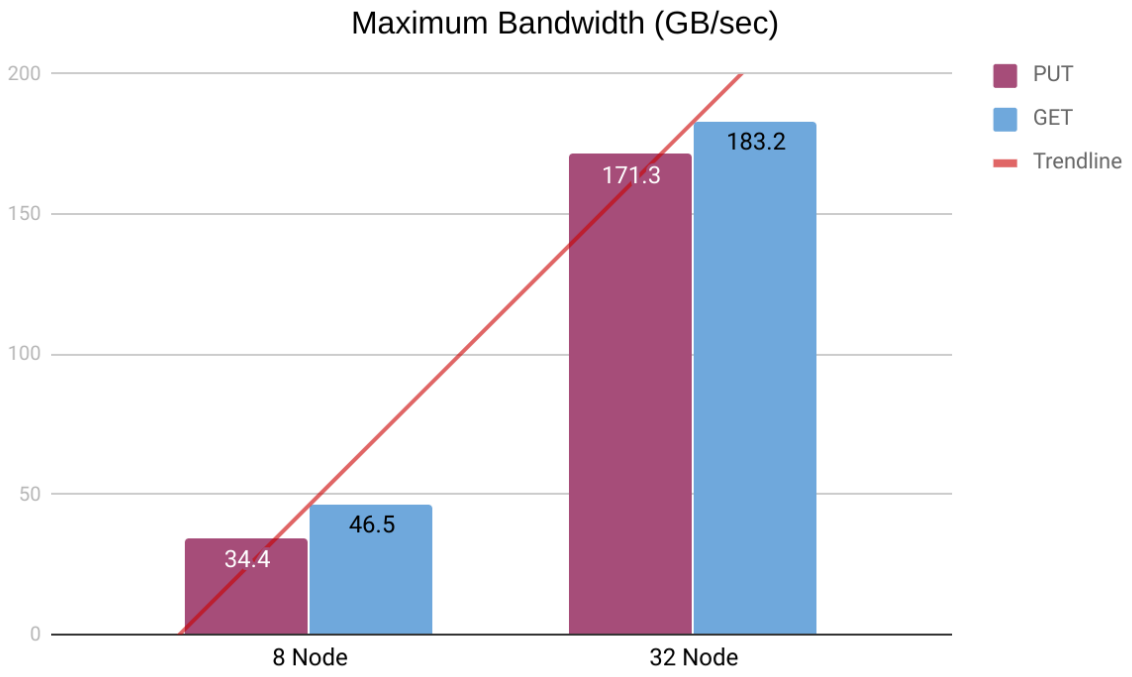
```
https://blog.min.io/scaling-minio-more-hardware-for-higher-scale/
```

机械硬盘



HDD Backend	PUT (GB/s)	GET (GB/s)
16 Node	8.57 GB/s	10.81 GB/s
24 Node	9.4 GB/s	17.0 GB/s
Scale Factor	1.1x	1.57x

NVME固态硬盘



NVMe Backend	PUT (GB/s)	GET (GB/s)
8 Node	34.4 GB/s	46.5 GB/s
32 Node	171.3 GB/s	183.2 GB/s
Scale Factor	4.9x	3.9x

1.4 MinIO 工作机制

1.4.1 MinIO 相关术语

- 0bject 对象

  存储到Minio的基本对象,如文件、字节流等任意数据

  一个文件就是一个对象

- Bucket 桶

  用来存储Object的逻辑空间。

  每个Bucket之间的数据是相互隔离的。

  对于客户端而言，就相当于一个存放文件的顶层文件夹，用于实现不同资源的分类存储

  通常一个项目或同一类资源可以对应于一个Bucket

- Drive 驱动器

  即存储数据的磁盘,一个Drive通常对应一块物理磁盘或者一个独立目录

  在MinIO启动时，以参数的方式传入。

  Minio 中所有的对象数据都会存储在Drive里

- Set 存储集

  set 是一组Drive的集合,即一组相关的磁盘的集合

  分布式部署时,MinIO会根据集群规模自动划分一个或多个Set，每个Set中的Drive分布在不同位置。

  一个对象存储在一个Set上

  一个集群划分为多个Set

  一个Set包含的Drive数量是固定的， 默认由系统根据集群规模自动计算得出

  一个Set中的Drive尽可能分布在不同的节点上

1.4.2 纠删码EC (Erasure Code)

```
https://blog.min.io/erasure-coding/
https://min.io/docs/minio/linux/operations/concepts/erasure-coding.html
https://www.minio.org.cn/docs/minio/kubernetes/upstream/operations/concepts.html
#id11
```

分布式存储，很关键的点在于数据的可靠性，即保证数据的完整，不丢失，不损坏。只有在可靠性实现的前提下，才有了追求一-致性、高可用、高性能的基础。而对于在存储领域，一 般对于保证数据可靠性的方法主要有两类，一类是冗余法，一 类是校验法。

分布式存储可靠性常用方法

- 冗余法

冗余法即对存储的数据进行副本备份，当数据出现丢失，损坏，即可使用备份内容进行恢复，而副本备份的多少，决定了数据可靠性的高低。这其中会有成本的考量，副本数据越多，数据越可靠，但需要的设备就越多，成本就越高。可靠性是允许丢失其中一份数据。

当前有很多分布式系统采用此种方式实现，如：ELasticsearch的索引副本，Kafka的副本，Redis的集群，MySQL的主从模式，Hadoop的多副本的文件系统

- 校验法

  校验法即通过校验码的数学计算的方式，对出现丢失、损坏的数据可以实现校验和还原两个功能

  通过对数据进行校验和(checksum)进行计算，可以检查数据是否完整，有无损坏或更改，在数据传输和保存时经常用到，如TCP协议

  恢复还原，通过对数据结合校验码进行数学计算，还原丢失或损坏的数据，可以在保证数据可靠的前提下，降低冗余，如单机硬盘存储中的RAID技术，纠删码(Erasure Code)技术等。

  MinIO采用的就是纠删码技术。

MinIO 纠删码EC (Erasure Code) 是一种提供数据高可用性功能，允许具有多个驱动器的 MinIO 部署即时自动重建对象，即使群集中丢失了多个驱动器或节点。 纠删码提供了对象级修复

Minio使用纠删码erasure code 与校验和checksum来保护数据免受硬件故障和无声数据损坏。即便您丢失一 半数量(N/2) 的硬盘,您仍然可以恢复数据。

纠删码是一种恢复铁和损坏数据的数学算法，Minio采用Reed-Solomon code将对象拆分成N/2数据和N/2奇偶校验块。

当损坏总磁盘数的一半磁盘时,只能读取而不能上传新的文件,只要保证正常磁盘数大于等n/2+1时,就可以支持写入新数据

这就意味着如果是12块盘，一个对象会被分成6个数据块、6个奇偶校验块，你可以失任意6块盘(不管其是存放的数据块还是奇偶校验块)，你仍可以从剩下的盘中的数据进行恢复。

实现纠删码 EC 至少需要4块Drive磁盘以上

纠删码是可以通过数学计算,实现数据冗余,功能上类似于RAID技术,当磁盘损坏时,可以通过计算把丢失的数据进行还原，它可以将n份原始数据,增加m份数据,并能通过n+m份中的任意n份数据,还原为原始数据。即如果有任意小于等于m份的数据失效,仍然能通过剩下的数据还原出来。

MinIO 根据纠删集的大小将对象拆分为数据和奇偶校验块，然后将数据和奇偶校验块随机均匀地分布在一组驱动器上，以便每个驱动器每个对象包含的块不超过一个。虽然驱动器可能包含多个对象的数据和奇偶校验块，但只要系统中有足够的驱动器，单个对象每个驱动器的块不超过一个。对于版本化对象，MinIO 选择相同的驱动器进行数据和奇偶校验存储，同时在任何一个驱动器上保持零重叠。

当一个大文件大于10 MB时，写入 MinIO，S3 API 将其分解为分段上传。分段大小由客户端在上传时确定。S3 要求每个部分至少为 5 MB（最后一部分除外）且不超过 5 GB。根据 S3 规范，一个对象最多可以分成10,000 个Part 部分。假设一个 320 MB 的对象。如果此对象分为 64 个部分，MinIO 会将每个部分: part.1、part.2 写入驱动器,...直到第 64 部分。这些部分的大小大致相等，例如，作为多部分上传的 320 MB 对象将拆分为 64 个 5 MB 部分。

上传的每个部分都通过条带进行纠删码。Part.1 是上载对象的第一部分，所有部分在驱动器上水平条带化。每个部分都由其数据块、奇偶校验块和 XL 元数据组成。MinIO 轮换写入，因此系统不会总是将数据写入相同的驱动器，并将奇偶校验写入相同的驱动器。每个对象都独立旋转，允许统一有效地使用群集中的所有驱动器，同时还增强了数据保护。

为了检索对象，MinIO 执行哈希计算以确定对象的保存位置，读取哈希并访问所需的纠删节和驱动器。

需要注意的是: 纠删码的内在工作原理和RAID实际是不同的，像RAID5可以在损失一块盘的情况下不丢数据，而Minio纠删码可以在丢失一半的盘的情况下，仍可以保证数据安全。 而且Minio纠删码是作用在对象级别，可以一次恢复一个对象，而RAID是作用在卷级别，数据恢复时间很长。 Minio对每个对象单独编码，存储服务一经部署，通常情况下是不需要更换硬盘或者修复。MinIO纠删码的设计目标是为了性

能和尽可能的使用硬件加速，与相类技术（如 RAID 或复制）相比，开销要少得多。

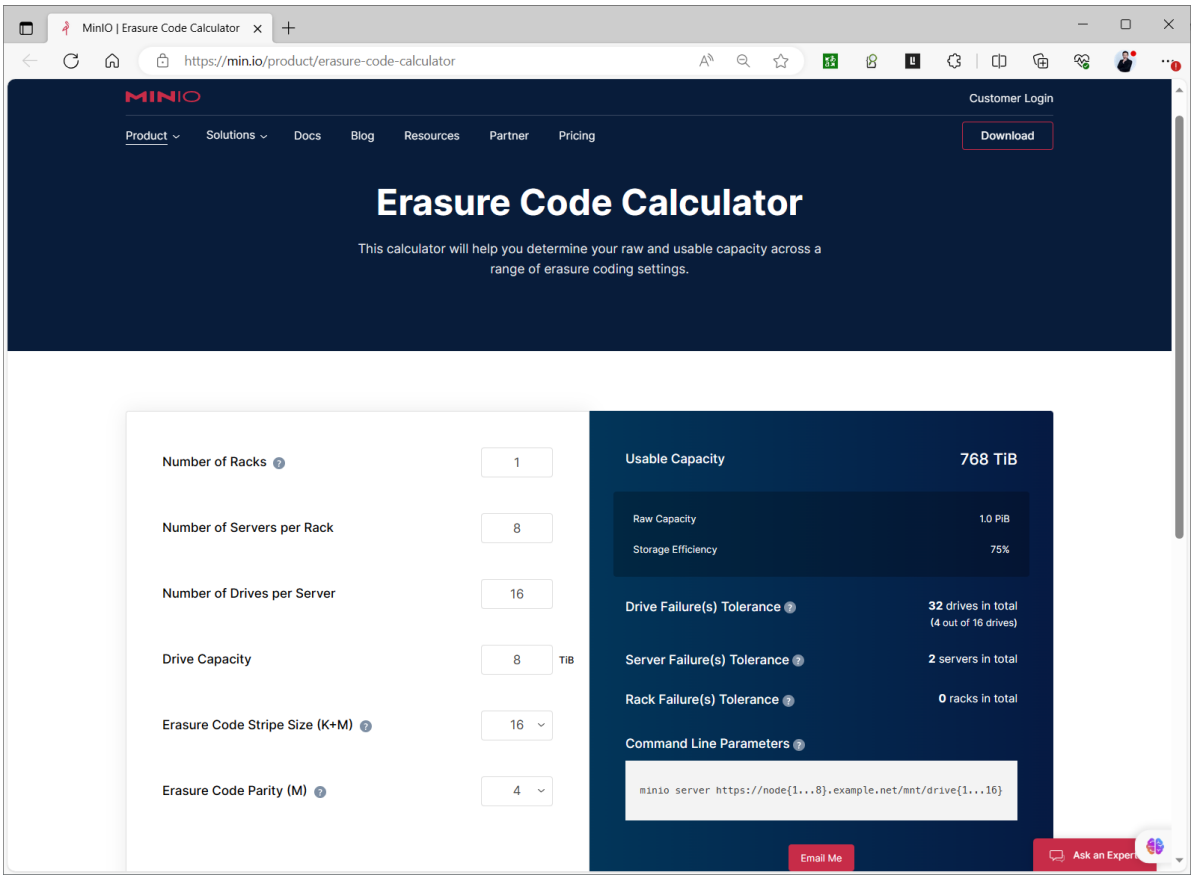
纠删码比 RAID 更适合对象存储有几个原因。MinIO 纠删码不仅可以在多个驱动器和节点发生故障时保护对象免受数据丢失，还可以在对象级别进行保护和修复。一次修复一个对象的能力是比在卷级别修复的 RAID 的显着优势。损坏的对象可以在几秒钟内在 MinIO 中恢复，而在 RAID 中则需要数小时。如果驱动器损坏并被更换，MinIO 会识别新驱动器，将其添加到纠删节，然后验证所有驱动器上的对象。更重要的是，读取和写入不会相互影响，从而实现大规模性能。有 MinIO 部署，在 PB 级存储中拥有数千亿个对象。

MinIO 中的纠删码实施可提高数据中心的运营效率。与复制不同，无需跨驱动器和节点进行冗长的重建或重新同步数据。这听起来可能微不足道，但移动/复制对象可能非常昂贵，并且 16TB 驱动器出现故障并通过数据中心网络复制到另一个驱动器会给存储系统和网络带来巨大的负担。

```
https://min.io/docs/minio/linux/operations/concepts/erasure-coding.html
```

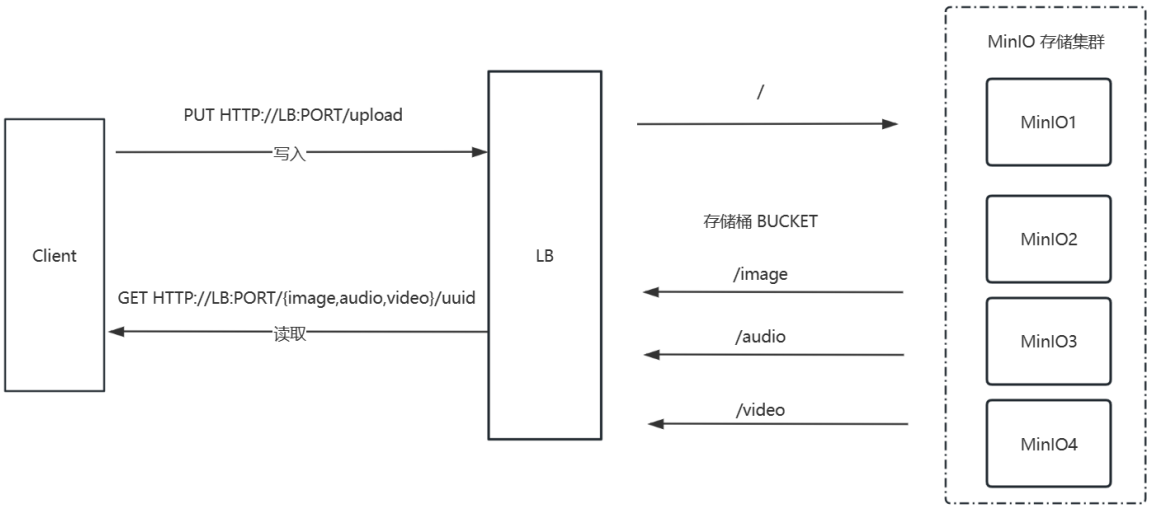
纠删码计算器

```
https://min.io/product/erasure-code-calculator
```



EC(Erasure-Code)算法的最底层的基本的数学原理：行列矩阵中-种特殊矩阵的性质：即任意N*M （N行M列{N<M}）的行列式，其任意NxN的子矩阵那是可逆，以实现数据恢复运算。
定义：erasure code是一种技术，它可以将n份原始数据，增加m份数据(用来存储erasure编码)， 并能通过n+m份中的任意n份数据，还原为原始数据。定义中包含了encode和decode两个过程，将原始的n份数据变为n+m份是encode，之后这n+m份数据可存放在不同的device上，如果有任意小于m份的数据失效，仍然能通过剩下的数据还原出来。也就是说，通常n+m的erasure编码，能容m块数据故障的场景，这时候的存储成本是1+m/n，通常m<n。
RS(Reed-Solomon) code是最基本的一种ES。RS codes是基于有限域的一种编码算法，有限域又称为GaloisField，是以法国著名数学家Galois 命名的，在RS codes中使用GF(2^w)，其中2^w>=n+ m. RS codes定义了一个(n + m) * n的分发矩阵(Distribution Matrix)

1.4.3 MinIO 工作流程



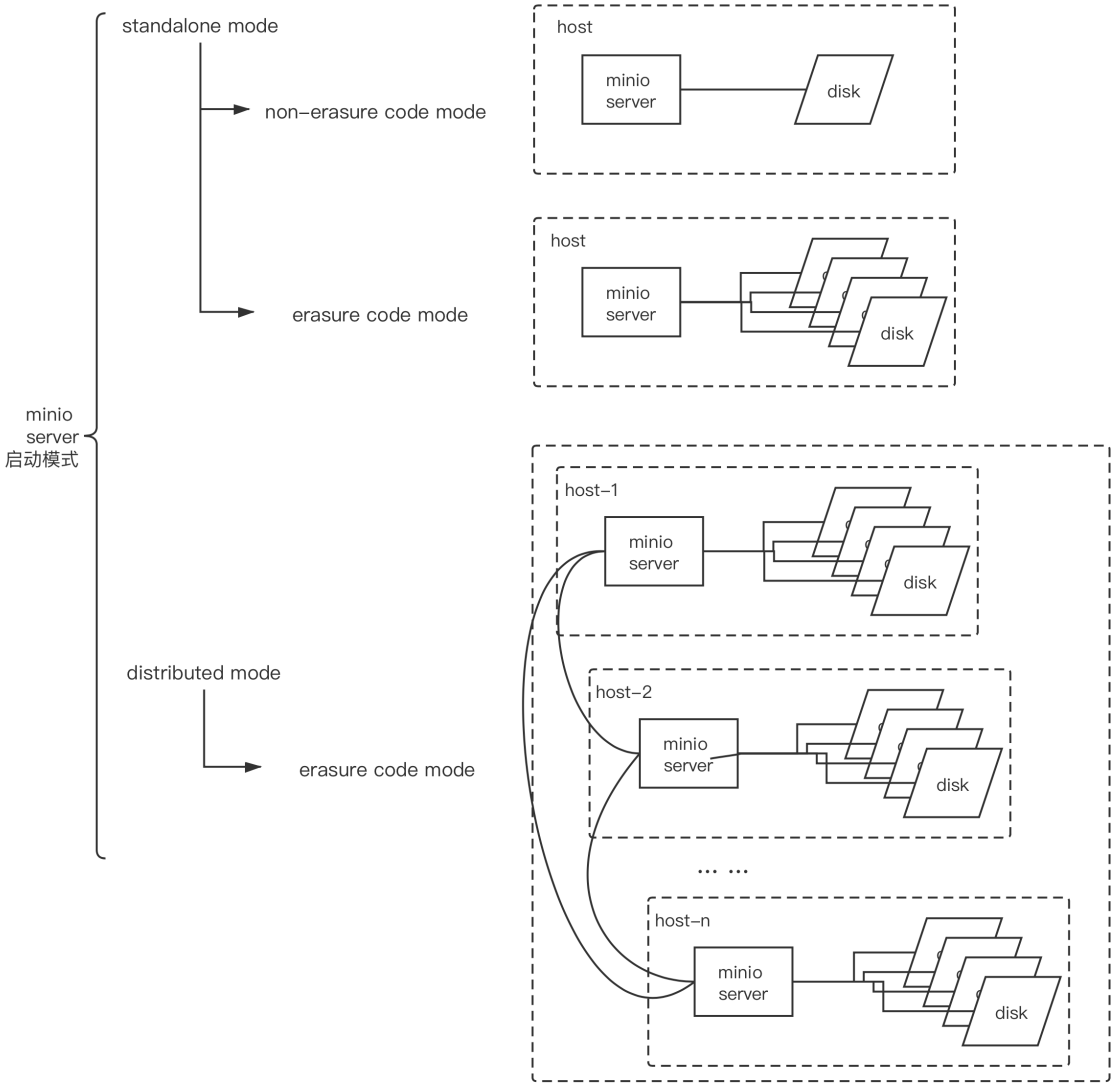
1.4.4 MinIO 存储机制

对象上传到MinIO后,以Bucket目录下,目录结构如下

```
./bucket_name/filename/xl.meta #元数据文件
./bucket_name/filename/hash/part.N

#编号为奇数的磁盘中的part.N为校验码文件
#编号为偶数的磁盘中的part.N为原始数据文件

#示例
[root@ubuntu2204 ~]#tree /minio/
/minio/
├── data1
│ └── mybucket
│     └── 19M图片.jpg
│         ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
│         │ └── part.1
│         └── xl.meta
├── data2
│ └── mybucket
│     └── 19M图片.jpg
│         ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
│         │ └── part.1
│         └── xl.meta
├── data3
│ └── mybucket
│     └── 19M图片.jpg
│         ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
│         │ └── part.1
│         └── xl.meta
├── data4
│ └── mybucket
│     └── 19M图片.jpg
│         ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
│         │ └── part.1
│         └── xl.meta
├── data5
│ └── mybucket
```

```
|      └── 19M图片.jpg
|          ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
|          |   └── part.1
|          └── xl.meta
├── data6
|   └── mybucket
|       └── 19M图片.jpg
|           ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
|           |   └── part.1
|           └── xl.meta
├── data7
|   └── mybucket
|       └── 19M图片.jpg
|           ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
|           |   └── part.1
|           └── xl.meta
└── data8
    └── mybucket
        └── 19M图片.jpg
            ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
            |   └── part.1
            └── xl.meta
```

1.4.5 MinIO 启动模式



2 MINIO 部署

2.1 部署模式和方法

部署模式

- 单机单硬盘
- 单机多硬盘
- 多机多硬盘

部署方法

- 包安装
- 二进制安装
- Docker 容器化安装
- 基于 Kubernetes 部署

2.2 单机部署

2.2.1 单机部署说明

```
https://min.io/docs/minio/linux/operations/install-deploy-manage/deploy-minio-single-node-single-drive.html
https://min.io/docs/minio/linux/operations/install-deploy-manage/deploy-minio-single-node-multi-drive.html
https://min.io/docs/minio/linux/index.html
https://www.minio.org.cn/docs/minio/linux/index.html
```

minio server的standalone模式，即单机模式，所有管理的磁盘都在一个主机上。

该启动模式一般仅用于实验环境、测试环境、开发环境的验证和学习使用。

在standalone模式下，还可以分为non-erasure code mode和erasure code mode(纠删码)。

- non-erasure code mode

  当minio server 运行时只传入一个本地磁盘参数。即为 non-erasure code mode

  在此启动模式下，对于每一份对象数据， minio直接存储这份数据，不会建立副本，也不会启用纠删码机制。

  因此,这种模式无论是服务实例还是磁盘都是”单点"，无任何高可用保障，磁盘损坏就意味着数据丢失。

- erasure code mode

  此模式需要为minio server实例传入多个本地磁盘参数。

  一旦遇到多于一 个磁盘的参数， minio server会自动启用erasure code mode.

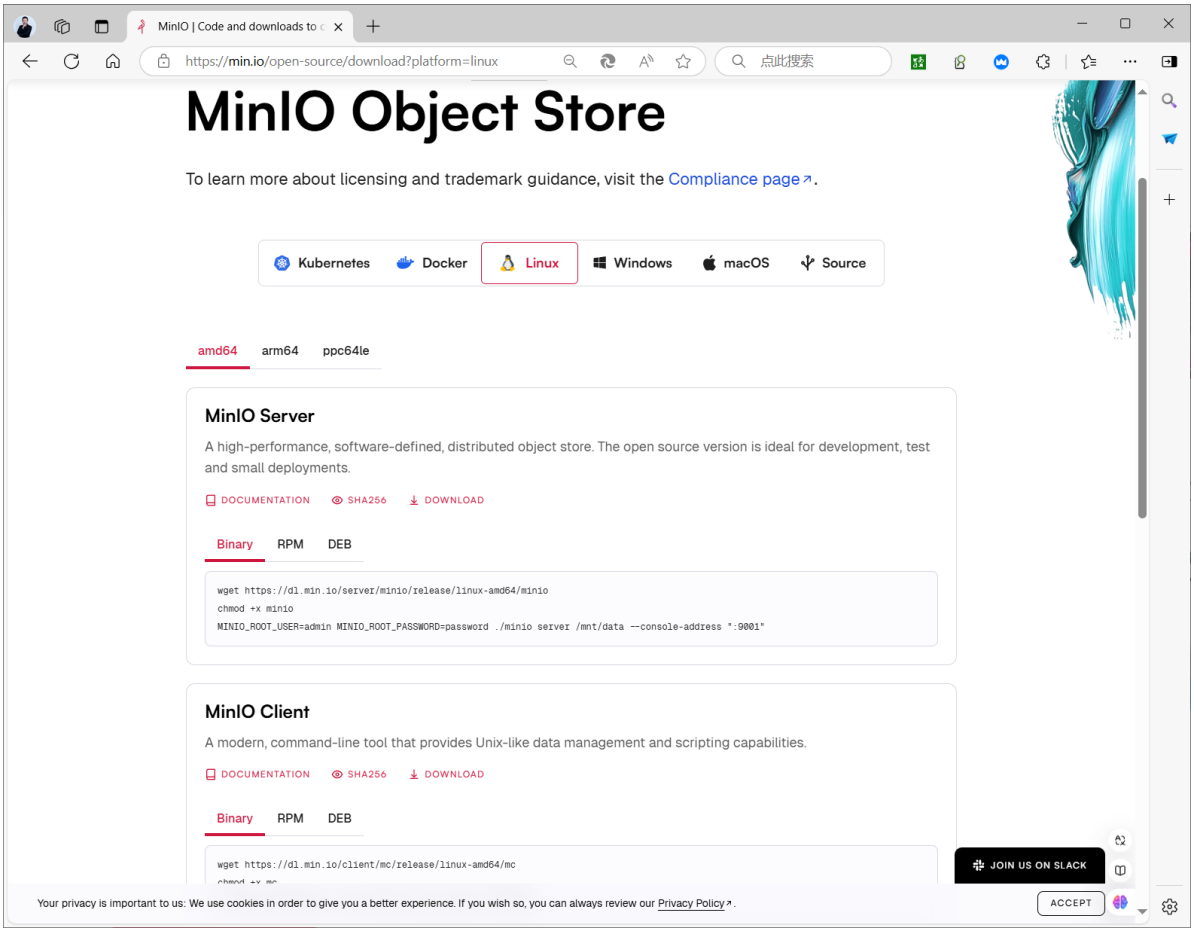
  erasure code对磁盘的个数是有要求的，至少4个磁盘, 如不满足要求，实例启动将失败。

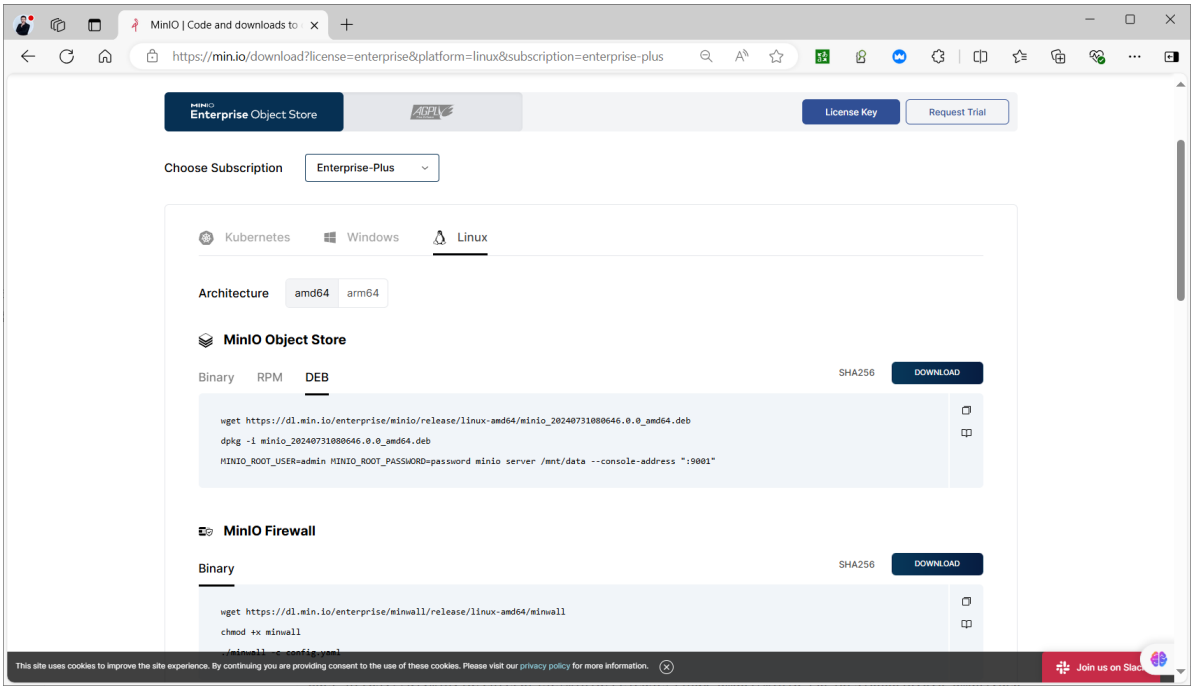
  erasure code启用后，要求传给minio server的endpoint(standalone模式下，即本地磁盘上的目录)至少为4个。

2.2.2 包安装

2.2.2.1 Debian/Ubuntu包安装

```
https://min.io/open-source/download?platform=linux
https://min.io/download?
license=enterprise&platform=linux&subscription=enterprise-plus
```





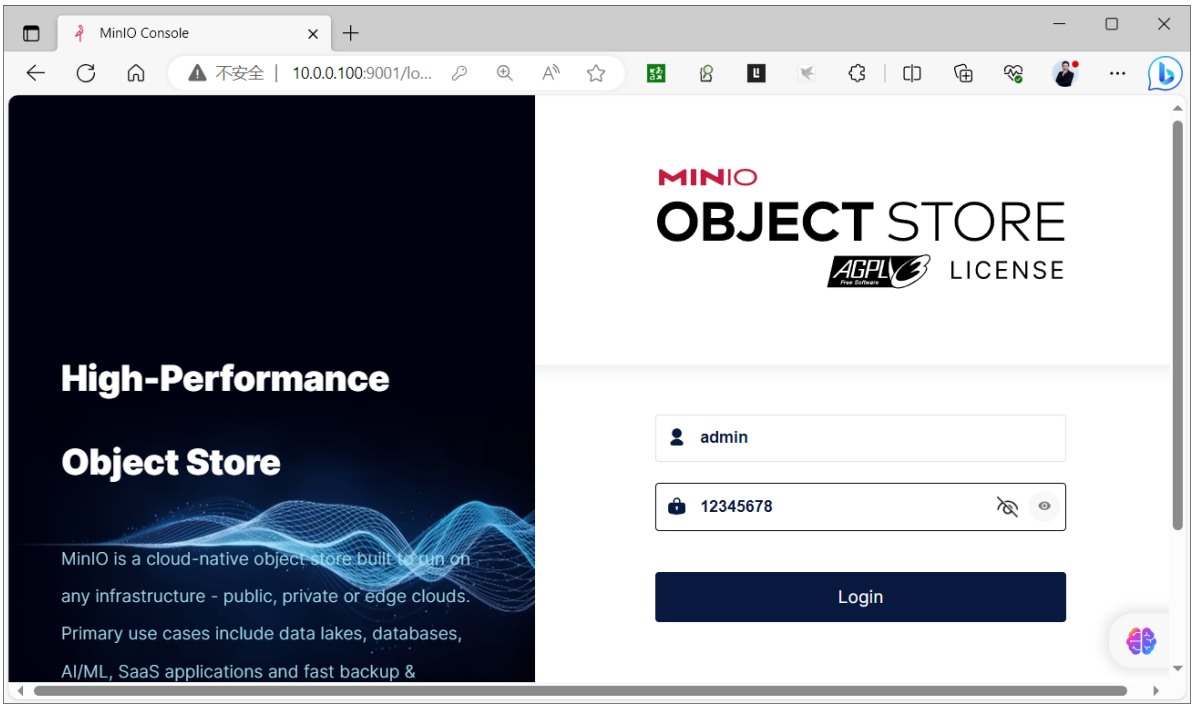
```
wget https://dl.min.io/enterprise/minio/release/linux-
amd64/minio_20240731080646.0.0_amd64.deb
dpkg -i minio_20240731080646.0.0_amd64.deb
MINIO_ROOT_USER=admin MINIO_ROOT_PASSWORD=password minio server /mnt/data --
console-address ":9001"
```

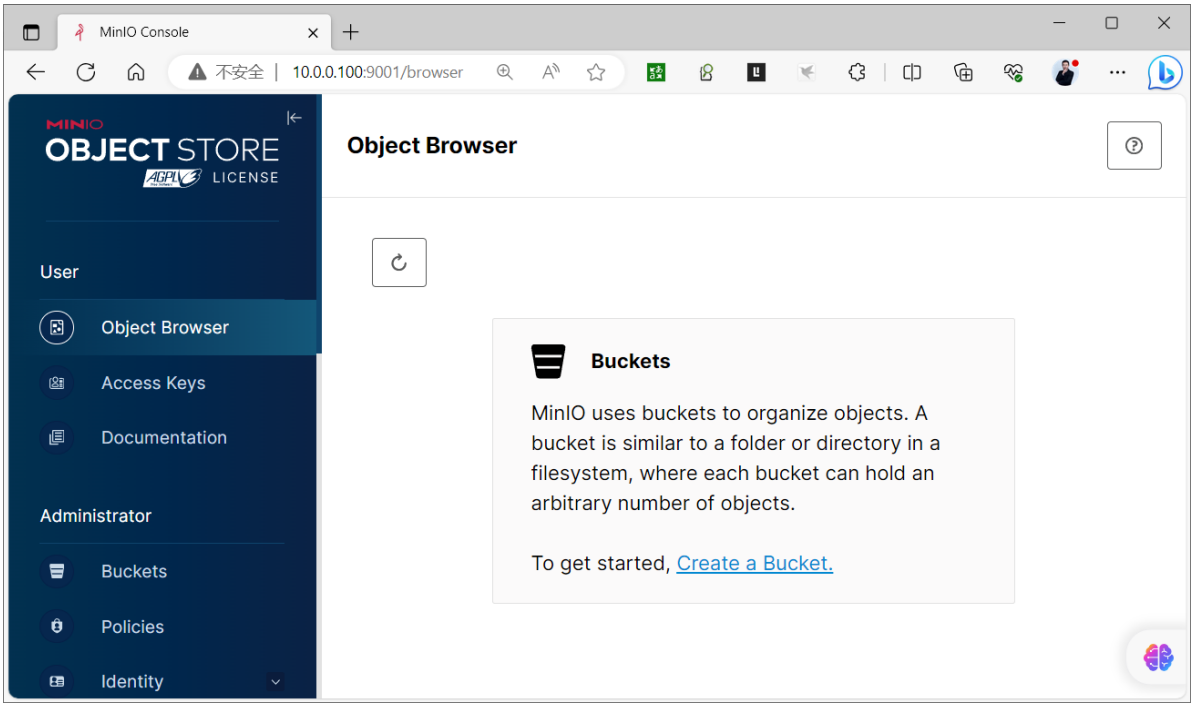
范例：Debian/Ubuntu包安装

```
[root@ubuntu2204 ~]#MINIO_VERION=20241013133411.0.0
[root@ubuntu2204 ~]#MINIO_VERION=20230829230735.0.0
[root@ubuntu2204 ~]#wget https://dl.min.io/server/minio/release/linux-
amd64/archive/minio_${MINIO_VERION}_amd64.deb -O minio.deb
[root@ubuntu2204 ~]#dpkg -i minio.deb
[root@ubuntu2204 ~]#dpkg -L minio
/lib
/lib/systemd
/lib/systemd/system
/lib/systemd/system/minio.service
/usr
/usr/local
/usr/local/bin
/usr/local/bin/minio

[root@ubuntu2204 ~]#which minio
/usr/local/bin/minio

[root@ubuntu2204 ~]#minio -v
minio version RELEASE.2023-08-29T23-07-35Z (commit-
id=07b1281046c8934c47184d1b56c78995ef960f7d)
Runtime: go1.19.12 linux/amd64
License: GNU AGPLv3 <https://www.gnu.org/licenses/agpl-3.0.html>
Copyright: 2015-2023 MinIO, Inc.

#查看service文件
[root@ubuntu2204 ~]#grep -Ev "#|^$" /lib/systemd/system/minio.service
[Unit]
Description=MinIO
Documentation=https://docs.min.io
Wants=network-online.target
After=network-online.target
AssertFileIsExecutable=/usr/local/bin/minio

[Service]
Type=notify
WorkingDirectory=/usr/local
User=minio-user
Group=minio-user
ProtectProc=invisible
EnvironmentFile=-/etc/default/minio
ExecStartPre=/bin/bash -c "if [ -z \"${MINIO_VOLUMES}\" ]; then echo \"Variable
MINIO_VOLUMES not set in /etc/default/minio\"; exit 1; fi"
ExecStart=/usr/local/bin/minio server $MINIO_OPTS $MINIO_VOLUMES
Restart=always
LimitNOFILE=1048576
TasksMax=infinity
TimeoutStopSec=infinity
```

```
SendSIGKILL=no
[Install]
WantedBy=multi-user.target

#需要手动创建用户
[root@ubuntu2204 ~]#groupadd -r minio-user
[root@ubuntu2204 ~]#useradd -M -r -s /sbin/nologin -g minio-user minio-user

#准备多个数据目录,minio数据目录不允许和根文件系统在一起，注意：新版允许
[root@ubuntu2204 ~]#mkdir -p /data
[root@ubuntu2204 ~]#lvcreate -n minio -L 10G ubuntu-vg
[root@ubuntu2204 ~]#mkfs.ext4 /dev/ubuntu-vg/minio
[root@ubuntu2204 ~]#echo /dev/ubuntu-vg/minio /data  ext4 defaults 0 0 >>
/etc/fstab
[root@ubuntu2204 ~]#mount -a
[root@ubuntu2204 ~]#mkdir -p /data/minio{1..4}
[root@ubuntu2204 ~]#chown -R minio-user.minio-user /data/
[root@ubuntu2204 ~]#ll  -d /data/minio{1..4}
drwxr-xr-x 3 minio-user minio-user  4096 Oct 17 15:33 /data/minio1/
drwxr-xr-x 3 minio-user minio-user  4096 Oct 17 15:33 /data/minio2/
drwxr-xr-x 3 minio-user minio-user  4096 Oct 17 15:33 /data/minio3/
drwxr-xr-x 3 minio-user minio-user  4096 Oct 17 15:33 /data/minio4/

[root@ubuntu2204 ~]#cat > /etc/default/minio <<EOF
MINIO_ROOT_USER=admin                #默认minioadmin
MINIO_ROOT_PASSWORD=12345678         #默认minioadmin，密码不能低于8位
MINIO_VOLUMES="/data/minio{1...4}"   #必选项
MINIO_OPTS='--console-address :9001' #默认使用随机端口，可以访问9000进行跳转到此随机端口
#MINIO_CONSOLE_ADDRESS=":9001" #功能同上
EOF

[root@ubuntu2204 ~]#systemctl enable --now minio.service
Created symlink /etc/systemd/system/multi-user.target.wants/minio.service →
/lib/systemd/system/minio.service.
[root@ubuntu2204 ~]#systemctl status  minio.service
● minio.service - MinIO
     Loaded: loaded (/lib/systemd/system/minio.service; enabled; vendor preset:
enabled)
     Active: active (running) since Tue 2023-10-17 15:42:50 CST; 5s ago
       Docs: https://docs.min.io
    Process: 1871 ExecStartPre=/bin/bash -c if [ -z "${MINIO_VOLUMES}" ]; then
echo "Variable MINIO_VOLUMES not set i>
   Main PID: 1873 (minio)
      Tasks: 7
     Memory: 87.7M
        CPU: 362ms
     CGroup: /system.slice/minio.service
             └─1873 /usr/local/bin/minio server --console-address :9001
/data/minio{1...4}

Oct 17 15:42:49 ubuntu2204 systemd[1]: Starting MinIO...
Oct 17 15:42:50 ubuntu2204 systemd[1]: Started MinIO.
Oct 17 15:42:50 ubuntu2204 minio[1873]: MinIO Object Storage Server
Oct 17 15:42:50 ubuntu2204 minio[1873]: Copyright: 2015-2023 MinIO, Inc.
Oct 17 15:42:50 ubuntu2204 minio[1873]: License: GNU AGPLv3
<https://www.gnu.org/licenses/agpl-3.0.html>
Oct 17 15:42:50 ubuntu2204 minio[1873]: Version: RELEASE.2023-08-29T23-07-35Z
(go1.19.12 linux/amd64)
```

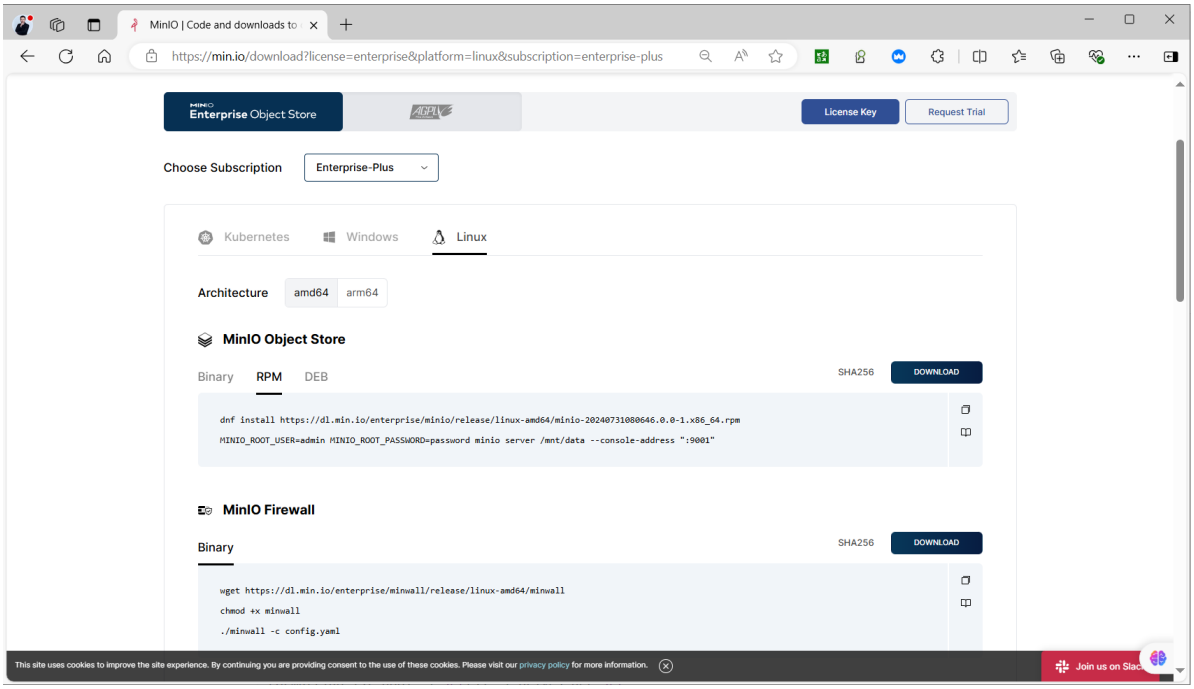
```
Oct 17 15:42:50 ubuntu2204 minio[1873]: Status:         4 Online, 0 Offline.
Oct 17 15:42:50 ubuntu2204 minio[1873]: S3-API: http://10.0.0.100:9000
http://127.0.0.1:9000
Oct 17 15:42:50 ubuntu2204 minio[1873]: Console: http://10.0.0.100:9001
http://127.0.0.1:9001
Oct 17 15:42:50 ubuntu2204 minio[1873]: Documentation:
https://min.io/docs/minio/linux/index.html

[root@ubuntu2204 ~]#ss -ntlp|grep minio
LISTEN 0      4096       127.0.0.1:9000      0.0.0.0:*    users:
(("minio",pid=1873,fd=10))
LISTEN 0      4096               *:9000            *:*    users:
(("minio",pid=1873,fd=11))
LISTEN 0      4096           [::1]:9000         [::]:*    users:
(("minio",pid=1873,fd=12))
LISTEN 0      4096               *:9001            *:*    users:
(("minio",pid=1873,fd=7))


#登录 http://10.0.0.100:9001/ 用户名密码:admin/12345678
```





2.2.2.2 红帽系统包安装

```
https://min.io/download?
license=enterprise&platform=linux&subscription=enterprise-plus
```




```
dnf install https://dl.min.io/enterprise/minio/release/linux-amd64/minio-
20240731080646.0.0-1.x86_64.rpm
MINIO_ROOT_USER=admin MINIO_ROOT_PASSWORD=password minio server /mnt/data --
console-address ":9001"
```

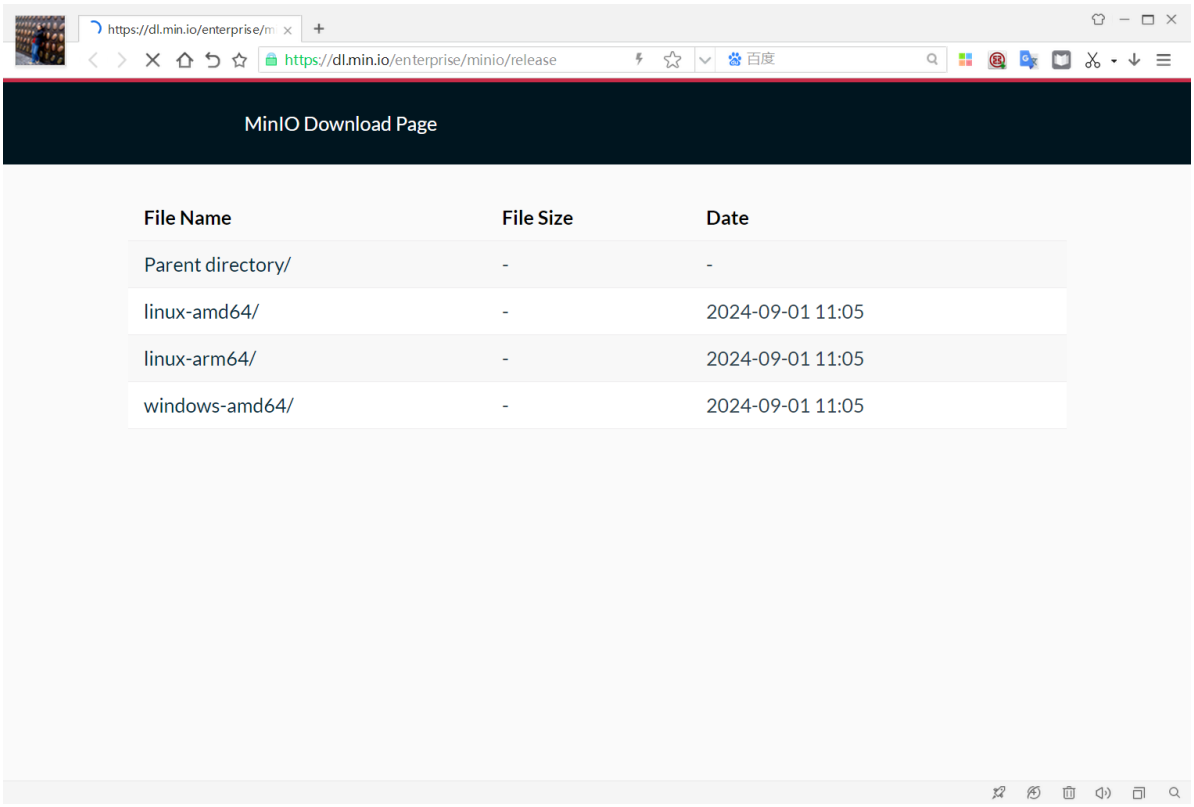
范例: 红帽系统包安装

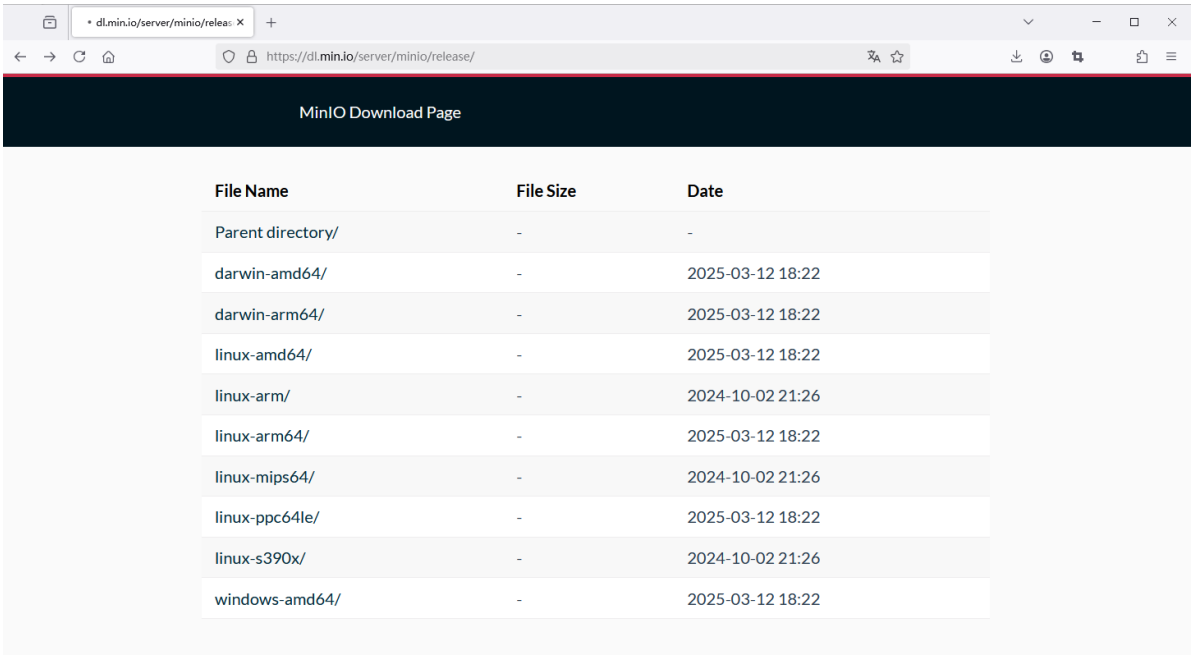
```
wget https://dl.min.io/server/minio/release/linux-amd64/archive/minio-
20230829230735.0.0.x86_64.rpm -O minio.rpm
sudo dnf install minio.rpm
mkdir ~/minio
minio server ~/minio --console-address :9090

#默认用户和密码
#RootUser: minioadmin
#RootPass: minioadmin
```

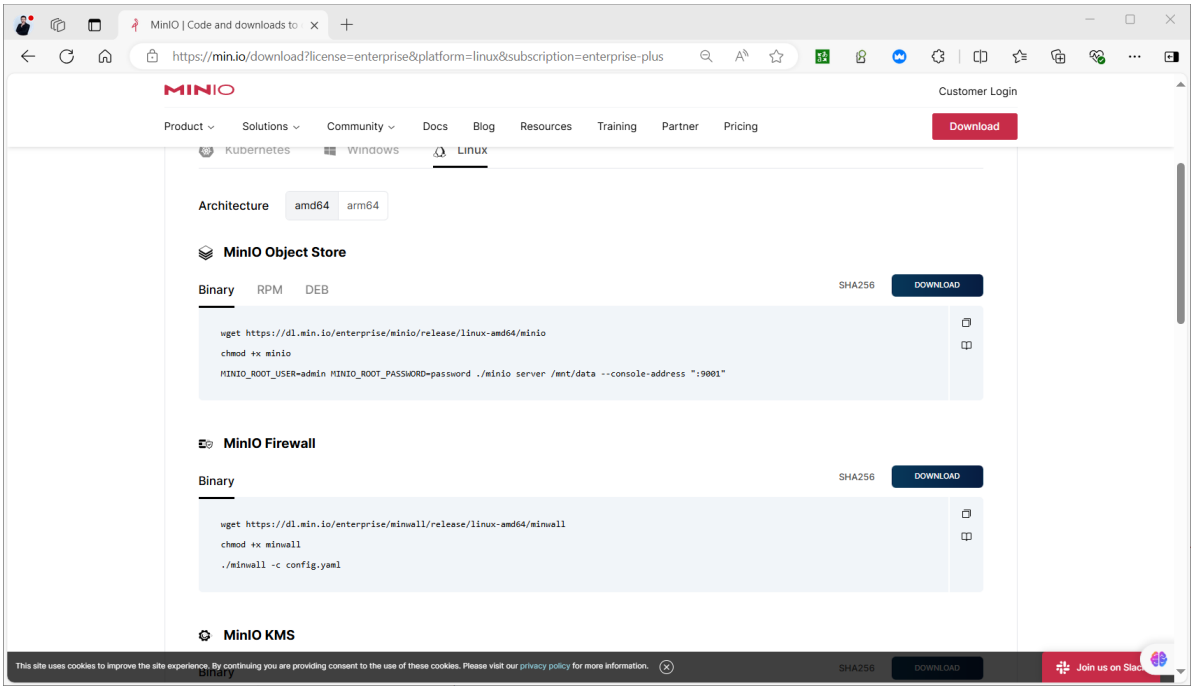
2.2.3 二进制部署

2.2.3.1 二进制部署单机单磁盘模式

```
https://dl.min.io/enterprise/minio/release/
https://min.io/download?
license=enterprise&platform=linux&subscription=enterprise-plus
```








```
wget https://dl.min.io/enterprise/minio/release/linux-amd64/minio
chmod +x minio
MINIO_ROOT_USER=admin MINIO_ROOT_PASSWORD=password ./minio server /mnt/data --
console-address ":9001"
```

范例: 二进制部署单机单磁盘

```
#下载
[root@ubuntu2204 ~]#wget https://dl.min.io/server/minio/release/linux-
amd64/minio
#国内镜像下载
[root@ubuntu2204 ~]#wget https://dl.minio.org.cn/server/minio/release/linux-
amd64/minio
[root@ubuntu2204 ~]#install minio  /usr/local/bin/

#准备数据目录,建议此目录为LVM
[root@ubuntu2204 ~]#mkdir -p /data/minio
```

```
#启动服务命令选项说明
#示例: minio server /data/minio --console-address :9090
server #表示start object storage server
--console-address #指定管理后面端口,管理后台登录用户和密码,默认为minioadmin/minioadmin.
也可以通过还境变量MINIO_ROOT_USER（至少3个字符）和MINIO_ROOT_PASSWORD（至少8个字符）进行指
定

#执行启动
[root@ubuntu2204 ~]#minio server /data/minio --console-address :9090
#输出信息如下
Formatting 1st pool, 1 set(s), 1 drives per set.
WARNING: Host local has more than 0 drives of set. A host failure will result in
data becoming unavailable.
WARNING: Detected default credentials 'minioadmin:minioadmin', we recommend that
you change these values with 'MINIO_ROOT_USER' and 'MINIO_ROOT_PASSWORD'
environment variables
MinIO Object Storage Server
Copyright: 2015-2023 MinIO, Inc.
License: GNU AGPLv3 <https://www.gnu.org/licenses/agpl-3.0.html>
Version: RELEASE.2023-08-29T23-07-35Z (go1.19.12 linux/amd64)

Status:         1 Online, 0 Offline.
S3-API: http://10.0.0.101:9000  http://10.10.53.81:9000  http://127.0.0.1:9000

RootUser: minioadmin
RootPass: minioadmin

Console: http://10.0.0.101:9090 http://10.10.53.81:9090 http://127.0.0.1:9090

RootUser: minioadmin
RootPass: minioadmin

Command-line: https://min.io/docs/minio/linux/reference/minio-mc.html#quickstart
   $ mc alias set myminio http://10.0.0.101:9000 minioadmin minioadmin

Documentation: https://min.io/docs/minio/linux/index.html
Warning: The standard parity is set to 0. This can lead to data loss.
```

2.2.3.2 Shell 脚本实现单机单磁盘模式

范例: 一键安装Shell 脚本实现单机单磁盘模式

```
[root@ubuntu2204 ~]#cat install_minio_single_node.sh
#!/bin/bash
#
#**********************************************************************
#Author:            wangxiaochun
#QQ:                29308620
#Date:              2022-06-21
#FileName:          install_minio_single_node.sh
#URL:               http://www.wangxiaochun.com
#Description:       The test script
#Copyright (C):     2022 All rights reserved
#**********************************************************************

#支持在线和离线安装
```

```
#管理后台用户和密码,注意:用户名至少3个字符,密码至少8个字符,如果不指定下面环境变量，默认用户名/密码为minioadmin/minioadmin
MINIO_ROOT_USER=admin
MINIO_ROOT_PASSWORD=12345678

#管理后台监听端口
MINIO_PORT=9999

#数据目录
MINIO_DATA=/data/minio

#文件名和下载URL
MINIO_FILE=minio
MINIO_URL="https://dl.minio.org.cn/server/minio/release/linux-amd64/${MINIO_FILE}"
#MINIO_URL="https://dl.min.io/server/minio/release/linux-amd64/${MINIO_FILE}"


HOST_IP=`hostname -I|awk '{print $1}'`



COLOR_SUCCESS="echo -e \\033[1;32m"
COLOR_FAILURE="echo -e \\033[1;31m"
END="\033[m"


color () {
    RES_COL=60
    MOVE_TO_COL="echo -en \\033[${RES_COL}G"
    SETCOLOR_SUCCESS="echo -en \\033[1;32m"
    SETCOLOR_FAILURE="echo -en \\033[1;31m"
    SETCOLOR_WARNING="echo -en \\033[1;33m"
    SETCOLOR_NORMAL="echo -en \E[0m"
    echo -n "$1" && $MOVE_TO_COL
    echo -n "["
    if [ $2 = "success" -o $2 = "0" ] ;then
        ${SETCOLOR_SUCCESS}
        echo -n $"  OK  "
    elif [ $2 = "failure" -o $2 = "1"  ] ;then
        ${SETCOLOR_FAILURE}
        echo -n $"FAILED"
    else
        ${SETCOLOR_WARNING}
        echo -n $"WARNING"
    fi
    ${SETCOLOR_NORMAL}
    echo -n "]"
    echo
}


install_minio(){
    if  [ ! -e ${MINIO_FILE}  ] ;then
        wget ${MINIO_URL} || ${COLOR_FAILURE} "下载失败!" ${END}
    fi
    install ${MINIO_FILE} /usr/local/bin/minio
    useradd -s /sbin/nologin  -r minio
```

```
    mkdir -p ${MINIO_DATA}
    chown -R minio:minio  ${MINIO_DATA}
    cat > /lib/systemd/system/minio.service <<EOF
[Unit]
Description=Minio
After=systemd-networkd.service systemd-resolved.service
Documentation=https://min.io

[Service]
Type=simple
Environment=MINIO_ROOT_USER=${MINIO_ROOT_USER}
MINIO_ROOT_PASSWORD=${MINIO_ROOT_PASSWORD}
ExecStart=/usr/local/bin/minio server ${MINIO_DATA} --console-address
":${MINIO_PORT}"
Restart=on-failure
User=minio
Group=minio
LimitNOFILE=infinity
LimitNPROC=infinity
LimitCORE=infinity
OOMScoreAdjust=-1000

[Install]
WantedBy=multi-user.target
EOF

    systemctl daemon-reload
    systemctl enable --now  minio.service &>/dev/null
    systemctl is-active minio.service &>/dev/null
    if [ $?  -eq 0 ];then
        echo
        color "MinIO 安装完成!" 0
        echo "------------------------------------------------------------------
-"
        echo -e "请访问链接: \E[32;1mhttp://${HOST_IP}:${MINIO_PORT}/\E[0m"
        echo -e "用户和密码:
\E[32;1m${MINIO_ROOT_USER}/${MINIO_ROOT_PASSWORD}\E[0m"
    else
        color "MinIO 安装失败!" 1
        exit
    fi
}


install_minio
```

2.2.4 容器化部署

```
https://min.io/docs/minio/container/index.html
```

范例: 容器化部署单机单磁盘

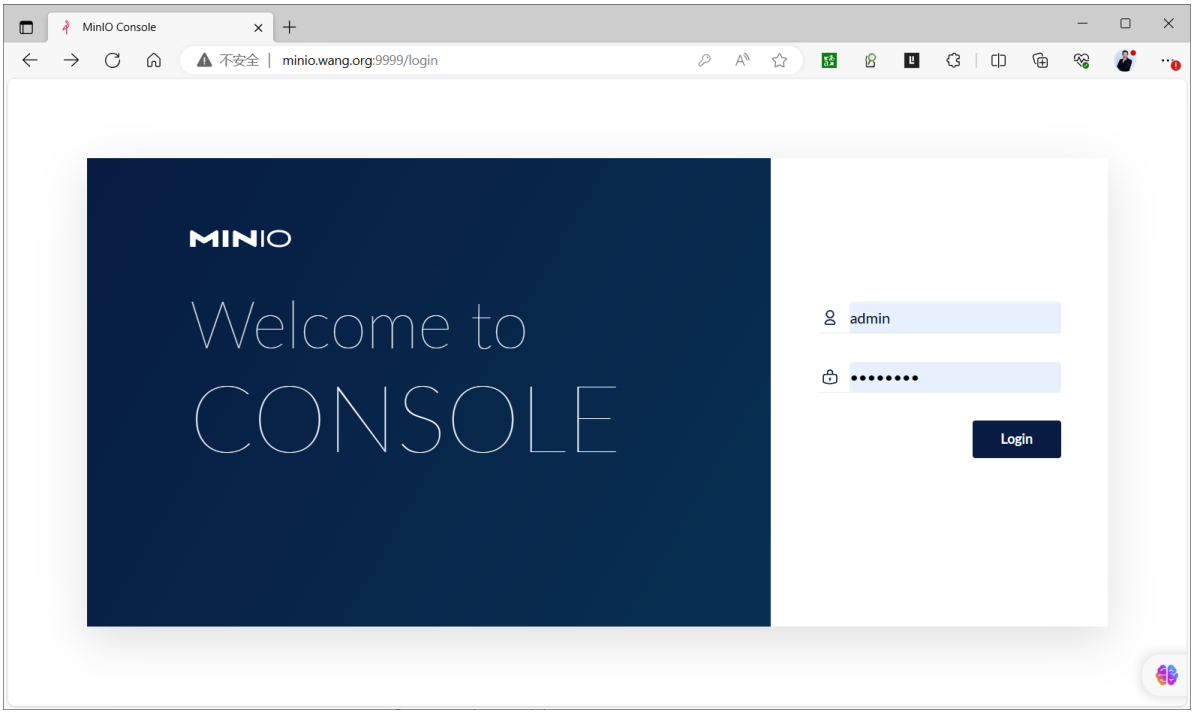
```
docker pull registry.cn-beijing.aliyuncs.com/wangxiaochun/minio:RELEASE.2024-11-
07T00-52-20Z
docker pull minio/minio
```

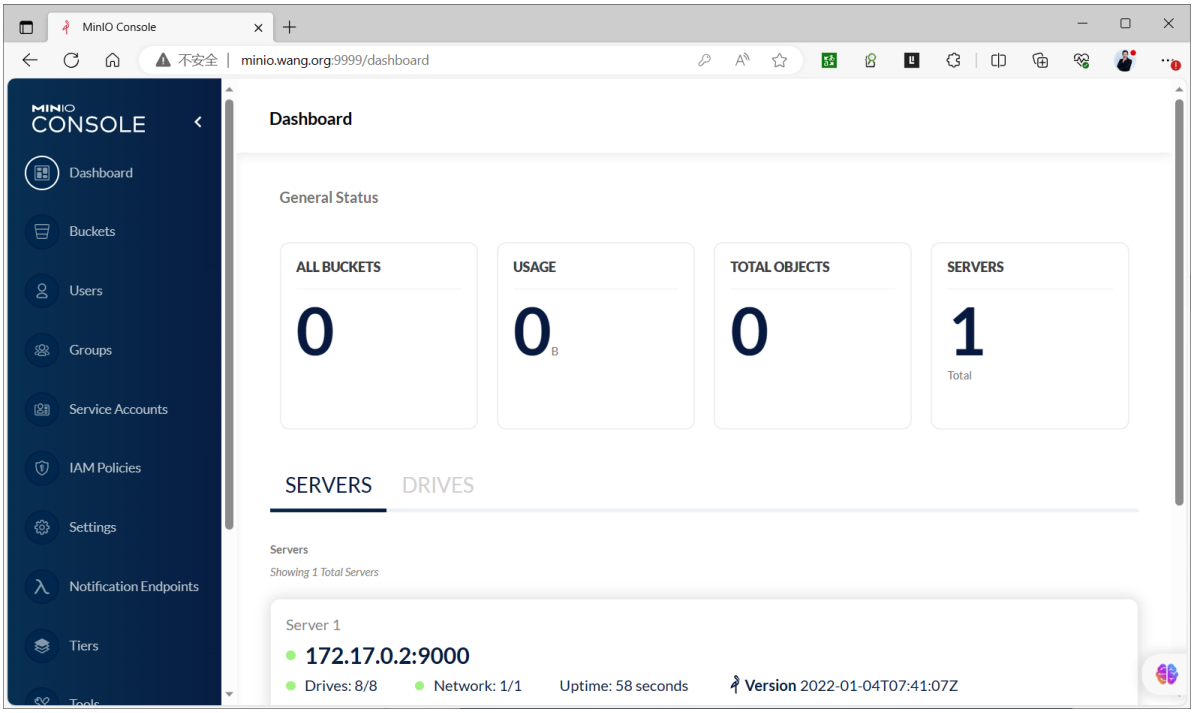
```
#可选
mkdir -p /data/minio

#docker部署
docker run -d \
   -p 9000:9000 \
   -p 9090:9090 \
   --name minio \
   -v /data/minio:/data \
   -e "MINIO_ROOT_USER=admin" \
   -e "MINIO_ROOT_PASSWORD=12345678" \
   registry.cn-beijing.aliyuncs.com/wangxiaochun/minio:RELEASE.2024-11-07T00-52-20Z server /data --console-address ":9090"
   #minio/minio server /data --console-address ":9090"

#podman 部署
podman run -p 9000:9000 -p 9001:9001 minio/minio server /data/minio --console-address ":9001"
```

范例: Docker 部署单机多磁盘启用纠删除码功能

```
[root@ubuntu2204 ~]#docker run -d \
  -p 9000:9000 \
  -p 9999:9999 \
  --name minio \
  -e "MINIO_ROOT_USER=admin" \
  -e "MINIO_ROOT_PASSWORD=12345678" \
  -v /minio/data1:/data1 \
  -v /minio/data2:/data2 \
  -v /minio/data3:/data3 \
  -v /minio/data4:/data4 \
  -v /minio/data5:/data5 \
  -v /minio/data6:/data6 \
  -v /minio/data7:/data7 \
  -v /minio/data8:/data8 \
  registry.cn-beijing.aliyuncs.com/wangxiaochun/minio:RELEASE.2024-11-07T00-52-20Z /data{1...8} --console-address ":9999"
  #minio/minio server /data{1...8} --console-address ":9999"


[root@ubuntu2204 ~]#docker logs minio
Automatically configured API requests per node based on available memory on the system: 17
Status:         8 Online, 0 Offline.
API: http://172.17.0.2:9000  http://127.0.0.1:9000

Console: http://172.17.0.2:9999 http://127.0.0.1:9999

Documentation: https://docs.min.io

 You are running an older version of MinIO released 2 years ago
 Update:
Run `mc admin update`

[root@ubuntu2204 ~]#tree /minio/
/minio/
```

```
├── data1
├── data2
├── data3
├── data4
├── data5
├── data6
├── data7
└── data8

8 directories, 0 files
```





2.2.5 基于 Docker Compose 部署

```
https://github.com/minio/minio/tree/master/docs/orchestration/docker-compose
```

范例：基于 Docker Compose 部署单节点

```
[root@ubuntu2404 ~]#cat docker-compose.yaml
version: '3.8'

services:
  minio:
    #image: minio/minio:RELEASE.2024-11-07T00-52-20Z
    image: registry.cn-beijing.aliyuncs.com/wangxiaochun/minio:RELEASE.2024-11-
07T00-52-20Z
    container_name: minio
    restart: always
    environment:
      MINIO_ROOT_USER: 'admin'
      MINIO_ROOT_PASSWORD: '12345678'
      MINIO_ADDRESS: ':9000'
      MINIO_CONSOLE_ADDRESS: ':9001'
    ports:
      - "9000:9000"
      - "9001:9001"
    networks:
      - minio_net
    volumes:
      - ./data:/data
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
      interval: 30s
      timeout: 20s
      retries: 3
    command: server /data
    #command: server --console-address ":9001" /data

networks:
  minio_net:
    driver: bridge

[root@ubuntu2404 ~]#apt update && apt -y install docker-compose python3-pip
[root@ubuntu2404 ~]#docker-compose version
docker-compose version 1.29.2, build unknown
docker-py version: 5.0.3
CPython version: 3.12.3
OpenSSL version: OpenSSL 3.0.13 30 Jan 2024

[root@ubuntu2404 ~]#docker-compose up -d

[root@ubuntu2404 ~]#docker-compose up -d

[root@ubuntu2404 ~]#docker-compose ps
Name               Command                  State
         Ports
--------------------------------------------------------------------------------
-------------------------------------------------
```

```
minio   /usr/bin/docker-entrypoint ...   Up (healthy)   0.0.0.0:9000-
>9000/tcp,:::9000->9000/tcp,
                                                       0.0.0.0:9001-
>9001/tcp,:::9001->9001/tcp
[root@ubuntu2404 ~]#docker network ls
NETWORK ID     NAME              DRIVER    SCOPE
dc0d93c3c604   bridge            bridge    local
92408dddfe8e   host              host      local
f26890b252b5   none              null      local
6354afd6686a   root_minio_net    bridge    local
```

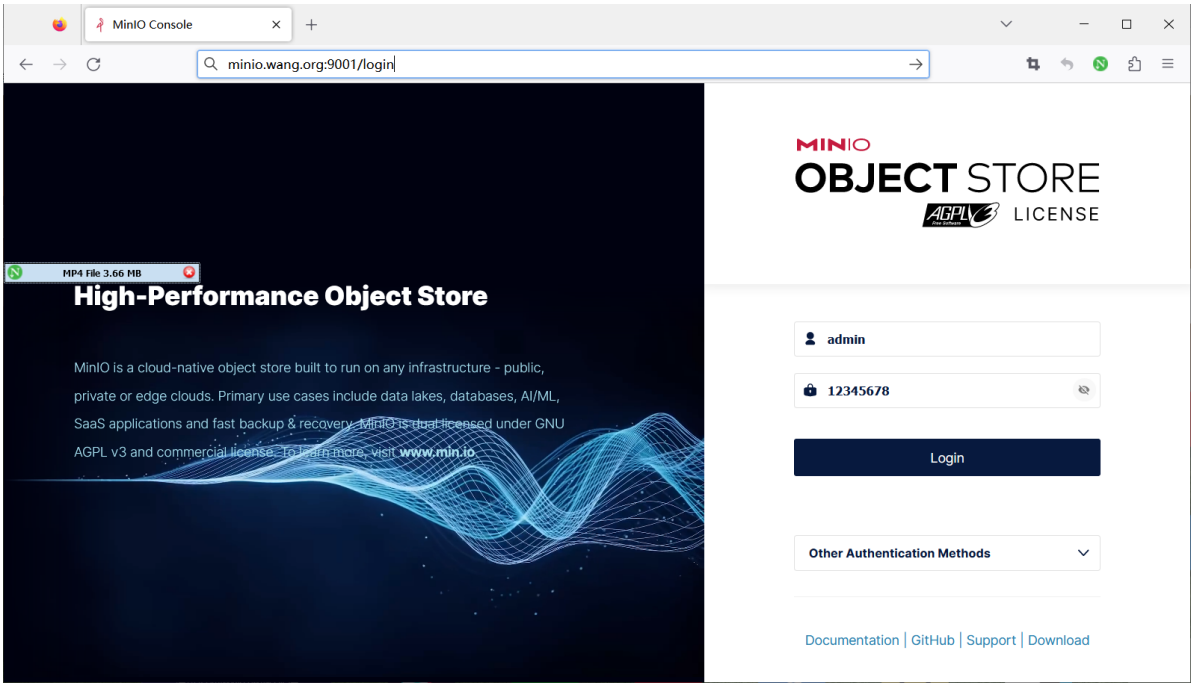
范例: 基于 Docker Compose 部署多服务节点多设备集群并利用 Nginx实现反向代理负载均衡

```
#https://github.com/minio/minio/blob/master/docs/orchestration/docker-
compose/docker-compose.yaml

[root@ubuntu2204 minio]#ls
docker-compose.yaml  nginx.conf

[root@ubuntu2204 minio]#cat docker-compose.yaml
version: '3.7'

# Settings and configurations that are common for all containers
x-minio-common: &minio-common
  image: quay.io/minio/minio:RELEASE.2023-08-31T15-31-16Z
  command: server --console-address ":9001" http://minio{1...4}/data{1...2}
  expose:
    - "9000"
    - "9001"
  environment:
    MINIO_ROOT_USER: admin
    MINIO_ROOT_PASSWORD: 12345678
  # environment:
    # MINIO_ROOT_USER: minioadmin
    # MINIO_ROOT_PASSWORD: minioadmin
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
    interval: 30s
    timeout: 20s
    retries: 3

# starts 4 docker containers running minio server instances.
# using nginx reverse proxy, load balancing, you can access
# it through port 9000.
services:
  minio1:
    <<: *minio-common
    hostname: minio1
    volumes:
      - data1-1:/data1
      - data1-2:/data2

  minio2:
    <<: *minio-common
    hostname: minio2
    volumes:
      - data2-1:/data1
```

```
      - data2-2:/data2

  minio3:
    <<: *minio-common
    hostname: minio3
    volumes:
      - data3-1:/data1
      - data3-2:/data2

  minio4:
    <<: *minio-common
    hostname: minio4
    volumes:
      - data4-1:/data1
      - data4-2:/data2

  nginx:
    image: nginx:1.19.2-alpine
    hostname: nginx
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    ports:
      - "9000:9000"
      - "9001:9001"
    depends_on:
      - minio1
      - minio2
      - minio3
      - minio4

## By default this config uses default local driver,
## For custom volumes replace with volume driver configuration.
volumes:
  data1-1:
  data1-2:
  data2-1:
  data2-2:
  data3-1:
  data3-2:
  data4-1:
  data4-2:

#nginx.conf

[root@ubuntu2204 minio]#cat nginx.conf
user  nginx;
worker_processes  auto;

error_log  /var/log/nginx/error.log warn;
pid        /var/run/nginx.pid;

events {
    worker_connections  4096;
}

http {
    include       /etc/nginx/mime.types;
    default_type  application/octet-stream;
```

```
    log_format  main  '$remote_addr - $remote_user [$time_local] "$request" '
                      '$status $body_bytes_sent "$http_referer" '
                      '"$http_user_agent" "$http_x_forwarded_for"';

    access_log  /var/log/nginx/access.log  main;
    sendfile        on;
    keepalive_timeout  65;

    # include /etc/nginx/conf.d/*.conf;

    upstream minio {
        server minio1:9000;
        server minio2:9000;
        server minio3:9000;
        server minio4:9000;
    }

    upstream console {
        ip_hash;
        server minio1:9001;
        server minio2:9001;
        server minio3:9001;
        server minio4:9001;
    }

    server {
        listen       9000;
        listen  [::]:9000;
        server_name  localhost;

        # To allow special characters in headers
        ignore_invalid_headers off;
        # Allow any size file to be uploaded.
        # Set to a value such as 1000m; to restrict file size to a specific
value
        client_max_body_size 0;
        # To disable buffering
        proxy_buffering off;
        proxy_request_buffering off;

        location / {
            proxy_set_header Host $http_host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;

            proxy_connect_timeout 300;
            # Default is HTTP/1, keepalive is only enabled in HTTP/1.1
            proxy_http_version 1.1;
            proxy_set_header Connection "";
            chunked_transfer_encoding off;

            proxy_pass http://minio;
        }
    }

    server {
```

```
        listen       9001;
        listen  [::]:9001;
        server_name  localhost;

        # To allow special characters in headers
        ignore_invalid_headers off;
        # Allow any size file to be uploaded.
        # Set to a value such as 1000m; to restrict file size to a specific
value
        client_max_body_size 0;
        # To disable buffering
        proxy_buffering off;
        proxy_request_buffering off;

        location / {
            proxy_set_header Host $http_host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
            proxy_set_header X-NginX-Proxy true;

            # This is necessary to pass the correct IP to be hashed
            real_ip_header X-Real-IP;

            proxy_connect_timeout 300;

            # To support websocket
            proxy_http_version 1.1;
            proxy_set_header Upgrade $http_upgrade;
            proxy_set_header Connection "upgrade";

            chunked_transfer_encoding off;

            proxy_pass http://console;
        }
    }
}

#浏览器访问代理:http://minio.wang.org:9001,用户名密码:minioadmin/minioadmin
```

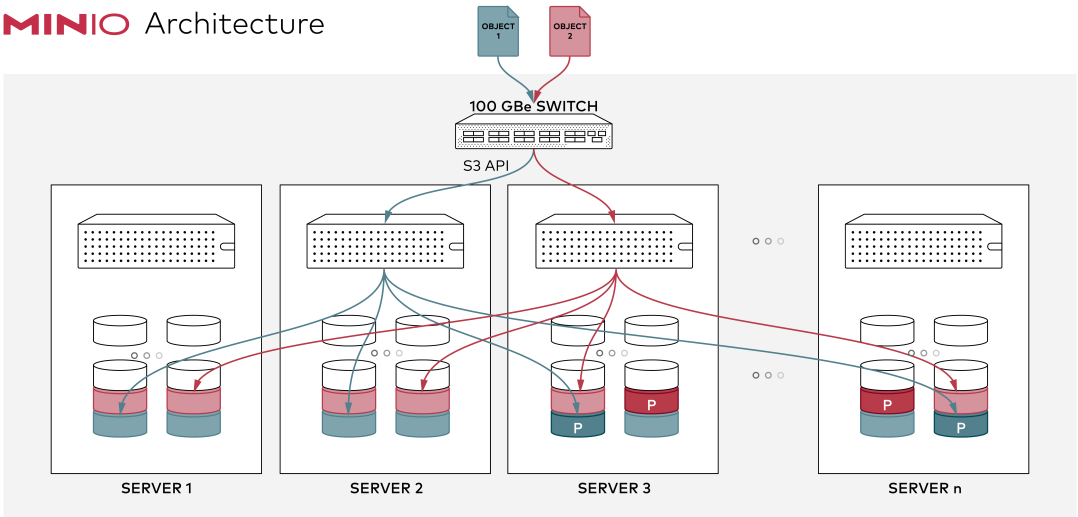


2.3 分布式集群部署

2.3.1 分布式集群机制

分布式Minio 将在不同的机器上的多块硬盘组成一个对象存储服务。

由于硬盘分布在不同的节点上,分布式MInio有更好的性能，同时避免了单点故障。




```
minio server http://host{1...n}/export{1...m}
```

分布式Minio优势

- 数据保护

  分布式Minio采用纠删码来防范多个节点宕机和位衰减bit rot。

  分布式Minio至少需要4个硬盘，使用分布式Minio 会自动引入纠删码功能。

- 高可用

  单机Minio服务存在单点故障，相反，如果是一个有N块硬盘的分布式Minio,只要有N/2硬盘在线，数据就是安全的。不过你需要至少有N/2+1个硬盘来创建新的对象。

例如，一个16节点Minio集群，每个节点16块硬盘，就算8台服务器宕机，这个集群仍然可读的，不过需要9台服务器才能写数据。

- 性能

  多个节点负载均衡，提升性能

- 一致性

  Minio在分布式和单机模式下，所有读写操作都严格遵守read-after-write一致性模型。

分布式Minio实现条件

- 在所有节点运行同样的命令启动一个分布式Minio实例，只需要把硬盘位置做为参数传给minio server命令即可
- 分布式Minio中的所有节点需要有同样的环境变量MINIO_ACCESS_KEY和MINIO_SECRET_KEY，才能建立分布式集群

  注意：新版本使用环境变量 MINIO_ROOT_USER和MINIO_ROOT_PASSWORD

- 分布式Minio使用的磁盘里必须是干净的，里面没有数据
- 分布式Minio里的节点时间差不能超过3秒，可以使用NTP来保证时间一致。

分布式Minio注意事项

- minio服务器多块数据磁盘需要使用独立的磁盘，在物理底层要相互独立避免遇到磁盘io竞争，导致minio性能直线下降,如果性能下降严重，数据量大时甚至会导致集群不可用
- minio数据磁盘最大不超过2T,如果使用lvm逻辑卷，逻辑卷大小也不要超过2T，过大的磁盘或文件系统会导致后期IO延迟较高导致minio性能降低
- minio集群共M节点每个节点N块数据磁盘，磁盘只要存活M*N/2，minio集群数据就是安全的，在节点数剩余M/2+1时节点可以正常读写
- 如果使用lvm方式扩展集群容量，请在部署阶段minio数据目录就使用lvm。
- 如果需要备份minio集群数据，请准备存放minio集群所有对象数据的空间容量

  比如: 一个2T/盘*4块/节点*8节点=64T的集群所有容量的一半存储空间即32T服务器，配置内存CPU配置不需要太高

- 如果网络环境允许请把minio集群节点配置双网卡，节点通信网络与客户端访问网络分开避免网络瓶颈
- 配置反向代理实现MinIO的负载均衡, 可以使用云服务SLB或者2台haproxy/nginx结合keepalived实现高可用
- minio系统中不要安装消耗IO较高的应用,比如:updatedb程序，如安装请排除扫描minio数据目录,否则可以会导致磁盘io延迟过高，会导致cpu负载过高，从而降低minio性能

2.3.2 分布式集群部署案例

```
https://min.io/docs/minio/linux/operations/install-deploy-manage/deploy-minio-multi-node-multi-drive.html
```

注意:每个数据目录默认不能使用系统的根文件系统 rootfs,必须为独立的文件系统

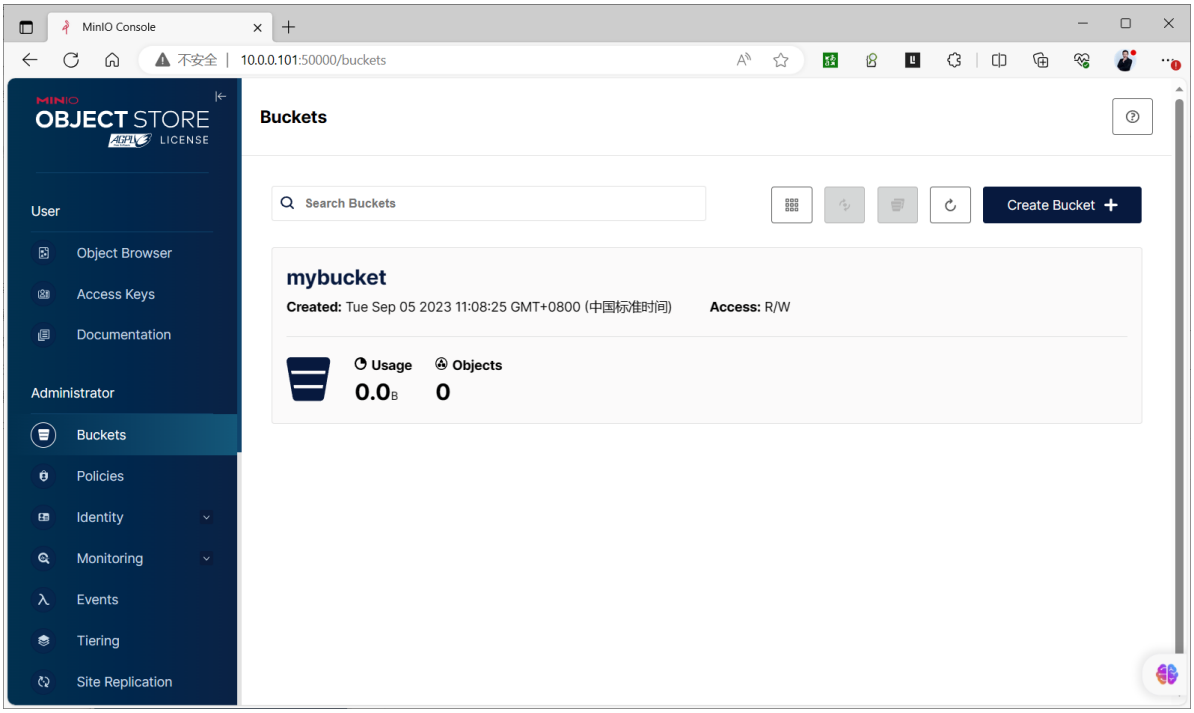
2.3.2.1 二进制部署分布式集群

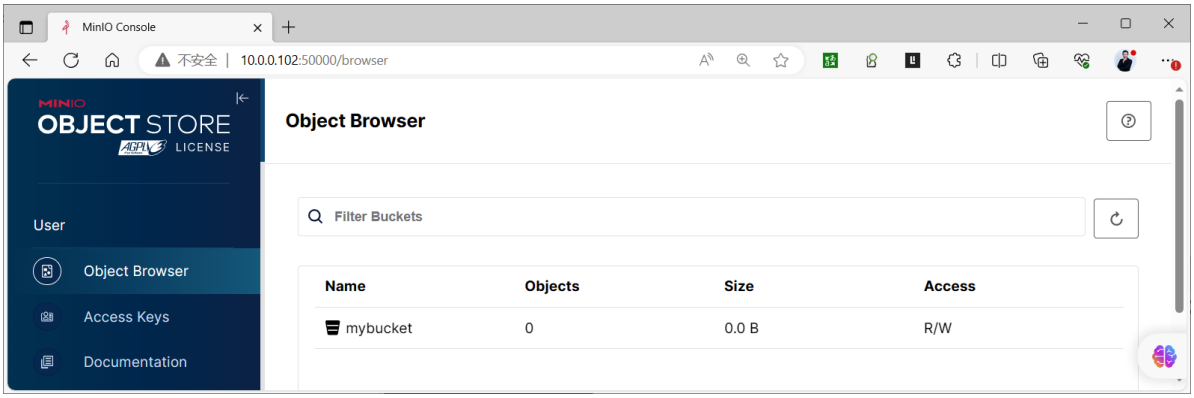
范例：3个节点，每节点1块盘

```
#启动分布式Minio实例，3个节点，每节点1块盘，需要在每个节点上都运行下面的命令
#先在每个主机上将磁盘格式化并挂载至/data/minio目录下，
export MINIO_ROOT_USER=admin
export MINIO_ROOT_PASSWORD=12345678
minio server http://10.0.0.{101..103}:9000/data/minio  --console-address :50000
```

范例：3个节点,每节点4块盘

```
#先每个主机的四块磁盘格式化,并分别挂载到/minio/dirve{1..4}上
export MINIO_ROOT_USER=admin
export MINIO_ROOT_PASSWORD=12345678
minio server https://10.0.0.{101..103}/minio/drive{1..4}  --console-address
:50000

#输出信息
Waiting for all MinIO sub-systems to be initialize...
Automatically configured API requests per node based on available memory on the
system: 16
All MinIO sub-systems initialized successfully in 3.683865209s
MinIO Object Storage Server
Copyright: 2015-2023 MinIO, Inc.
License: GNU AGPLv3 <https://www.gnu.org/licenses/agpl-3.0.html>
Version: RELEASE.2023-08-29T23-07-35Z (go1.19.12 linux/amd64)

Use `mc admin info` to look for latest server/drive info
 Status:         8 Online, 4 Offline.
S3-API: http://10.0.0.101:9000  http://127.0.0.1:9000
RootUser: admin
RootPass: 12345678

Console: http://10.0.0.101:50000 http://127.0.0.1:50000
RootUser: admin
RootPass: 12345678

Command-line: https://min.io/docs/minio/linux/reference/minio-mc.html#quickstart
   $ mc alias set myminio http://10.0.0.102:9000 admin 12345678


#在每个节点上创建新的bucket和上传文件,在集群中的其它节点上也可以查看到此bucket和上传的文件
```





范例：4个节点每节点4块盘集群

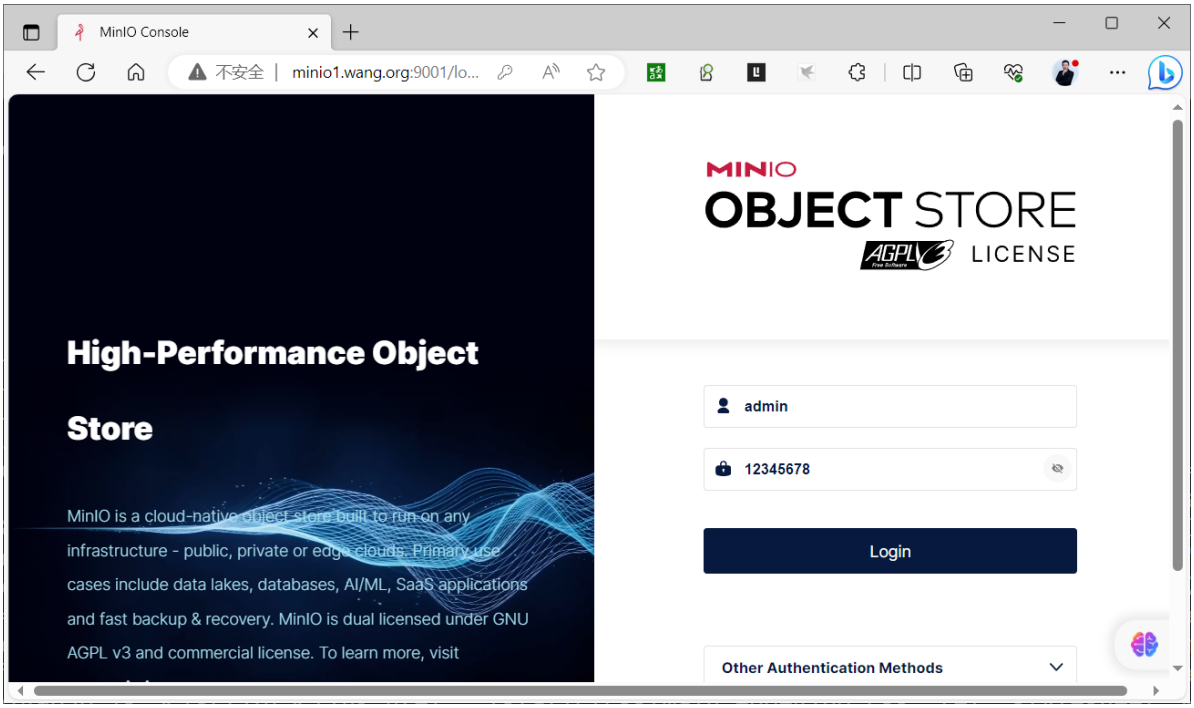
```
#在4个节点都执行下面指令
export MINIO_ROOT_USER=admin
export MINIO_ROOT_PASSWORD=12345678
minio server http://10.0.0.101/export1 \
             http://10.0.0.101/export2 \
             http://10.0.0.101/export3 \
             http://10.0.0.101/export4 \
             http://10.0.0.102/export1 \
             http://10.0.0.102/export2 \
             http://10.0.0.102/export3 \
             http://10.0.0.102/export4 \
             http://10.0.0.103/export1 \
             http://10.0.0.103/export2 \
             http://10.0.0.103/export3 \
             http://10.0.0.103/export4 \
             http://10.0.0.104/export1 \
             http://10.0.0.104/export2 \
             http://10.0.0.104/export3 \
             http://10.0.0.104/export4 \
             --console-address :50000

#或者下面用法
minio server http://10.0.0.10{1..4}/export{1..4} --console-address :50000
```

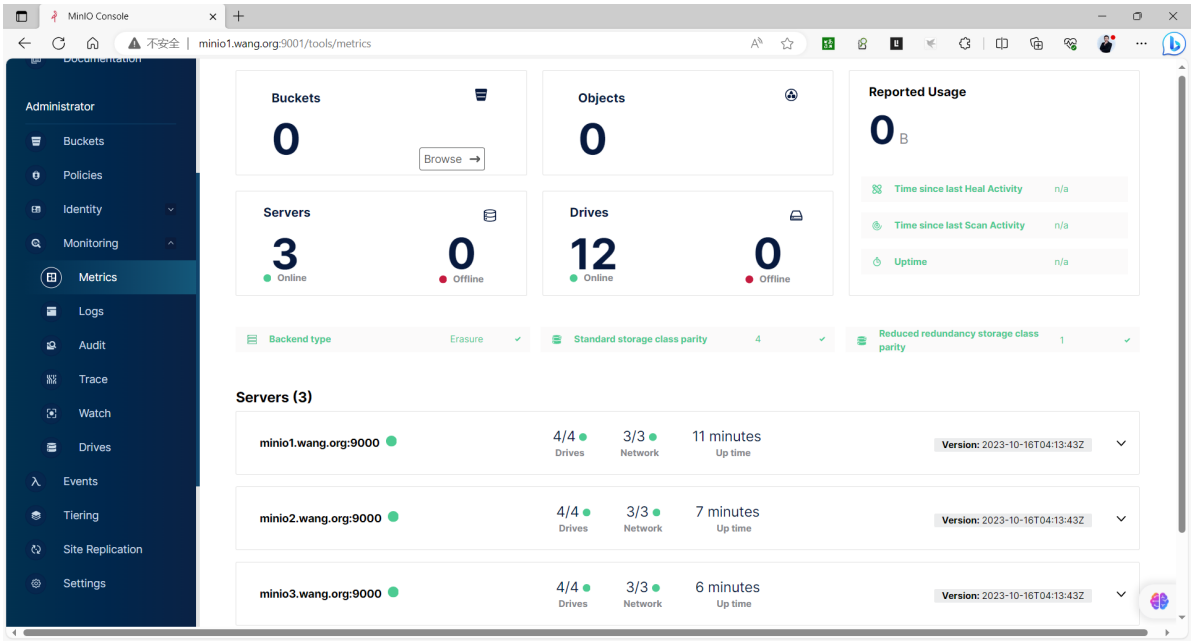
范例：基于LVM实现二进制部署MinIO 实现3节点的分布式高可用集群

```
#准备三台主机,在所有节点上做好名称解析
[root@minio1 ~]#cat /etc/hosts
10.0.0.101 minio1.wang.org
10.0.0.102 minio2.wang.org
10.0.0.103 minio3.wang.org

#在所有节点上准备数据目录和用户
#当前环境中有LV，并且VG中有空闲空间，所以基于当前环境创建新LV实现MinIO存储
[root@ubuntu2204 ~]#lsblk
NAME                      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                         8:0    0  200G  0 disk
├─sda1                      8:1    0    1M  0 part
├─sda2                      8:2    0    2G  0 part /boot
└─sda3                      8:3    0  198G  0 part
  └─ubuntu--vg-ubuntu--lv 253:0    0   99G  0 lvm  /
sr0                        11:0    1 1024M  0 rom
[root@minio1 ~]#pvs
  PV         VG        Fmt  Attr PSize    PFree
  /dev/sda3  ubuntu-vg lvm2 a--  <198.00g 99.00g
[root@minio1 ~]#vgs
  VG        #PV #LV #SN Attr   VSize    VFree
  ubuntu-vg   1   1   0 wz--n- <198.00g 99.00g

[root@minio1 ~]#lvs
  LV        VG        Attr       LSize   Pool Origin Data%  Meta%  Move Log
Cpy%Sync Convert
  ubuntu-lv ubuntu-vg -wi-ao---- <99.00g

#基于当前VG创建LV
[root@minio1 ~]#lvcreate -n minio -L 10G ubuntu-vg

[root@minio1 ~]#lvs
  LV        VG        Attr       LSize   Pool Origin Data%  Meta%  Move Log
Cpy%Sync Convert
  minio     ubuntu-vg -wi-a-----  10.00g

  ubuntu-lv ubuntu-vg -wi-ao---- <99.00g

#XFS
[root@minio1 ~]#mkfs.xfs  /dev/ubuntu-vg/minio
[root@minio1 ~]#echo /dev/ubuntu-vg/minio /data/ xfs defaults 0 0 >> /etc/fstab
#EXT4
[root@minio1 ~]#mkfs.ext4 /dev/ubuntu-vg/minio
[root@minio1 ~]#echo /dev/ubuntu-vg/minio /data/ ext4 defaults 0 0 >> /etc/fstab

[root@minio1 ~]#mkdir /data/
[root@minio1 ~]#mount -a
[root@minio1 ~]#mkdir -p /data/minio{1..4}
[root@minio1 ~]#useradd -r -s /sbin/nologin minio
[root@minio1 ~]#chown -R minio: /data/minio*


#在所有节点上下载二进制程序
[root@minio1 ~]#wget https://dl.minio.org.cn/server/minio/release/linux-
amd64/minio
```

```
[root@minio1 ~]#install minio /usr/local/bin/minio

#在所有节点上准备环境变量文件
[root@minio1 ~]#cat > /etc/default/minio
MINIO_ROOT_USER=admin
MINIO_ROOT_PASSWORD=12345678
MINIO_VOLUMES='http://minio{1...3}.wang.org:9000/data/minio{1...4}'
MINIO_OPTS='--console-address :9001'
MINIO_PROMETHEUS_AUTH_TYPE="public"

#注意事项
#MINIO_ROOT_USER管理员用户名，MINIO_ROOT_PASSWORD管理员密码
#如果minio服务器IP地址连续可以直接写IP地址写法，如果IP地址不连续,则使用前面在本地hosts解析文件配置的连续的主机名
#方法例如：MINIO_VOLUMES="http://10.0.0.10{1...3}:9000/data/minio{1..4}"

#在所有节点上创建service文件
[root@minio1 ~]#cat > /lib/systemd/system/minio.service
[Unit]
Description=MinIO
Documentation=https://docs.min.io
Wants=network-noline.target
After=network-noline.target

[Service]
WorkingDirectory=/usr/local
User=minio
Group=minio
EnvironmentFile=-/etc/default/minio
ExecStartPre=/bin/bash -c "if [ -z \"${MINIO_VOLUMES}\" ]; then echo \"Variable MINIO_VOLUMES not set in /etc/default/minio\"; exit 1; fi"
ExecStart=/usr/local/bin/minio server $MINIO_OPTS $MINIO_VOLUMES
Restart=always
LimitNOFILE=1048576
TasksMax=infinity
OOMScoreAdjust=-1000

[Install]
WantedBy=multi-user.target


#在所有节点启动服务
[root@minio1 ~]#systemctl daemon-reload
[root@minio1 ~]#systemctl enable --now  minio.service
[root@minio1 ~]#systemctl status minio.service
● minio.service - MinIO
     Loaded: loaded (/lib/systemd/system/minio.service; enabled; vendor preset: enabled)
     Active: active (running) since Tue 2023-10-17 16:28:02 CST; 6s ago
       Docs: https://docs.min.io
    Process: 1614 ExecStartPre=/bin/bash -c if [ -z "${MINIO_VOLUMES}" ]; then echo "Variable MINIO_VOLUMES not set i>
   Main PID: 1615 (minio)
      Tasks: 8 (limit: 2193)
     Memory: 145.8M
        CPU: 1.008s
     CGroup: /system.slice/minio.service
```

```
             └─1615 /usr/local/bin/minio server --console-address :9001
http://minio{1...3}.wang.org:9000/data/minio{>

Oct 17 16:28:03 minio1.wang.org minio[1615]: All MinIO sub-systems initialized
successfully in 10.007151ms
Oct 17 16:28:03 minio1.wang.org minio[1615]: MinIO Object Storage Server
Oct 17 16:28:03 minio1.wang.org minio[1615]: Copyright: 2015-2023 MinIO, Inc.
Oct 17 16:28:03 minio1.wang.org minio[1615]: License: GNU AGPLv3
<https://www.gnu.org/licenses/agpl-3.0.html>
Oct 17 16:28:03 minio1.wang.org minio[1615]: Version: RELEASE.2023-10-16T04-13-
43Z (go1.21.3 linux/amd64)
Oct 17 16:28:03 minio1.wang.org minio[1615]: Use `mc admin info` to look for
latest server/drive info
Oct 17 16:28:03 minio1.wang.org minio[1615]:  Status:         8 Online, 4
Offline.
Oct 17 16:28:03 minio1.wang.org minio[1615]: S3-API: http://10.0.0.103:9000
http://127.0.0.1:9000
Oct 17 16:28:03 minio1.wang.org minio[1615]: Console: http://10.0.0.103:9001
http://127.0.0.1:9001
Oct 17 16:28:03 minio1.wang.org minio[1615]: Documentation:
https://min.io/docs/minio/linux/index.html

[root@minio1 ~]#ss -ntlp |grep minio
LISTEN 0      4096       127.0.0.1:9000      0.0.0.0:*    users:
(("minio",pid=2685,fd=10))
LISTEN 0      4096               *:9000            *:*    users:
(("minio",pid=2685,fd=11))
LISTEN 0      4096           [::1]:9000         [::]:*    users:
(("minio",pid=2685,fd=12))
LISTEN 0      4096               *:9001            *:*    users:
(("minio",pid=2685,fd=24))

#浏览器访问 http://minio{1..3}.wang.org:9001/ 用户名密码:admin/12345678
```





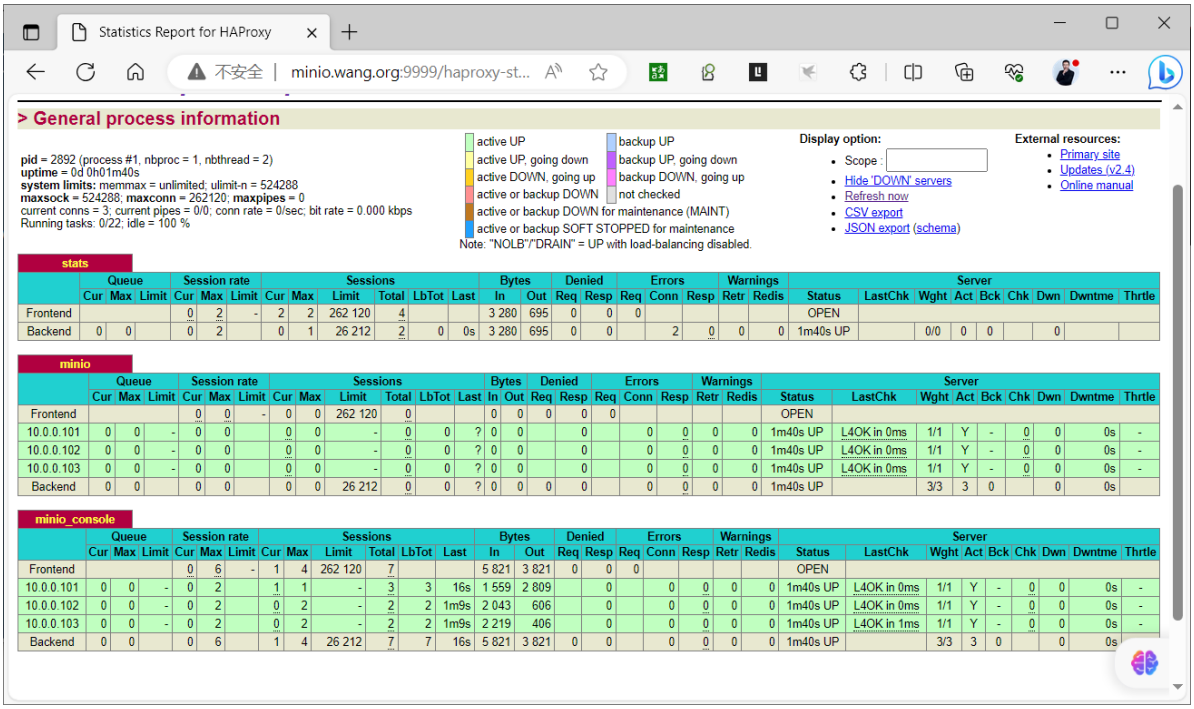

```
#配置反向代理,通过Haproxy或者Nginx实现minio的反向代理
#方式1:Haproxy配置
[root@ubuntu2204 ~]#apt update && apt -y install haproxy
[root@ubuntu2204 ~]#cat   /etc/haproxy/haproxy.cfg
......
listen stats
    mode http
    bind 0.0.0.0:9999
    stats enable
    log global
    stats uri /haproxy-status
    stats auth admin:123456

listen minio
    bind 10.0.0.100:9000
    mode http
    log global
    server 10.0.0.101 10.0.0.101:9000 check inter 3000 fall 2 rise 5
    server 10.0.0.102 10.0.0.102:9000 check inter 3000 fall 2 rise 5
    server 10.0.0.103 10.0.0.103:9000 check inter 3000 fall 2 rise 5

listen minio_console
    bind 10.0.0.100:9001
    mode http
    log global
    server 10.0.0.101 10.0.0.101:9001 check inter 3000 fall 2 rise 5
    server 10.0.0.102 10.0.0.102:9001 check inter 3000 fall 2 rise 5
    server 10.0.0.103 10.0.0.103:9001 check inter 3000 fall 2 rise 5

#方式2:nginx配置文件实现反向代理
[root@ubuntu2204 ~]#apt update && apt -y install nginx
[root@ubuntu2204 ~]#vim /etc/nginx/nginx.conf
....
http {
    upstream minio {
        server 10.0.0.101:9000;
        server 10.0.0.102:9000;
        server 10.0.0.103.9000;
    }
```

```
    upstream console {
        #ip_hash;
        hash $remote_addr;
        server 10.0.0.101:9001;
        server 10.0.0.102:9001;
        server 10.0.0.103:9001;
    }

    server {
        listen 9000;
        listen [::]:9000;
        server_name localhost;
        # To allow special characters in headers
        ignore_invalid_headers off;
        # Allow any size file to be uploaded.
        # Set to a value such as 1000m; to restrict file size to a specific
value
        client_max_body_size 0;
        # To disable buffering
        proxy_buffering off;
        location / {
            proxy_set_header Host $http_host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
            proxy_connect_timeout 300;
            # Default is HTTP/1, keepalive is only enabled in HTTP/1.1
            proxy_http_version 1.1;
            proxy_set_header Connection "";
            chunked_transfer_encoding off;
            proxy_pass http://minio;
            }
        }
        server {
            listen 9001;
            listen [::]:9001;
            server_name localhost;
            # To allow special characters in headers
            ignore_invalid_headers off;
            # Allow any size file to be uploaded.
            # Set to a value such as 1000m; to restrict file size to a specific
value
            client_max_body_size 0;
            # To disable buffering
            proxy_buffering off;
            location / {
                proxy_set_header Host $http_host;
                proxy_set_header X-Real-IP $remote_addr;
                proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
                proxy_set_header X-Forwarded-Proto $scheme;
                proxy_set_header X-NginX-Proxy true;
                # This is necessary to pass the correct IP to be hashed
                real_ip_header X-Real-IP;
                proxy_connect_timeout 300;
                # Default is HTTP/1, keepalive is only enabled in HTTP/1.1
                proxy_http_version 1.1;
                proxy_set_header Connection "";
                chunked_transfer_encoding off;
```

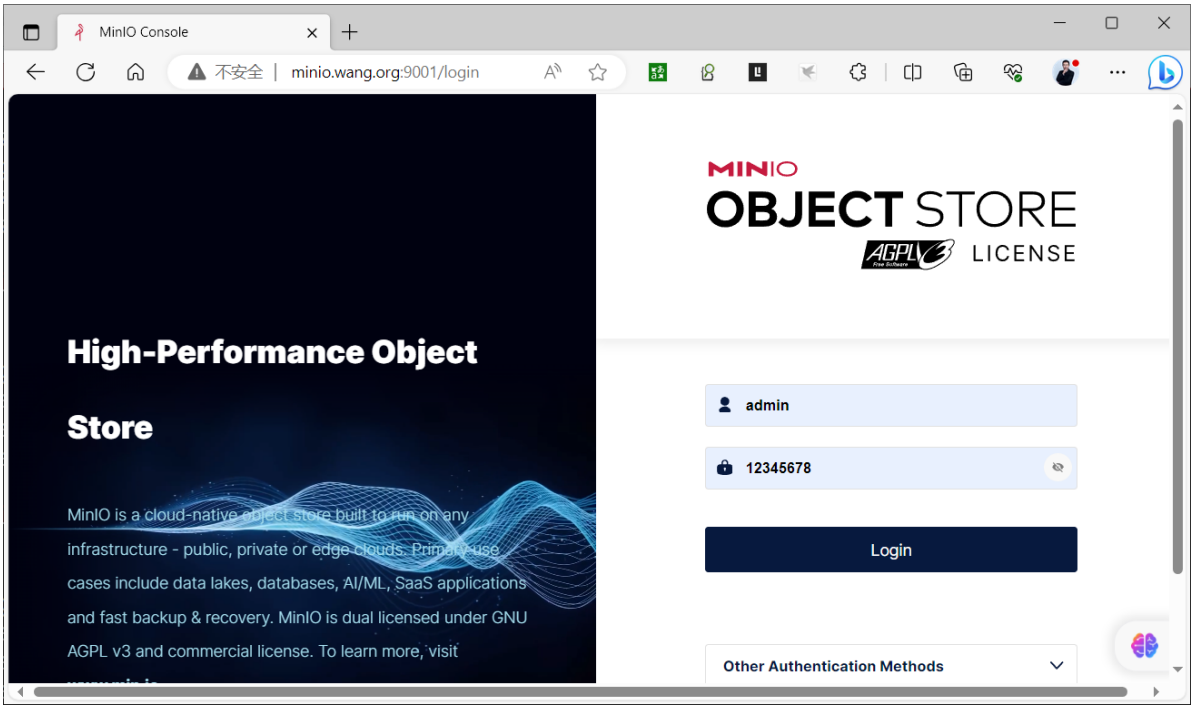
```
                proxy_pass http://console;
            }
        }
}
```

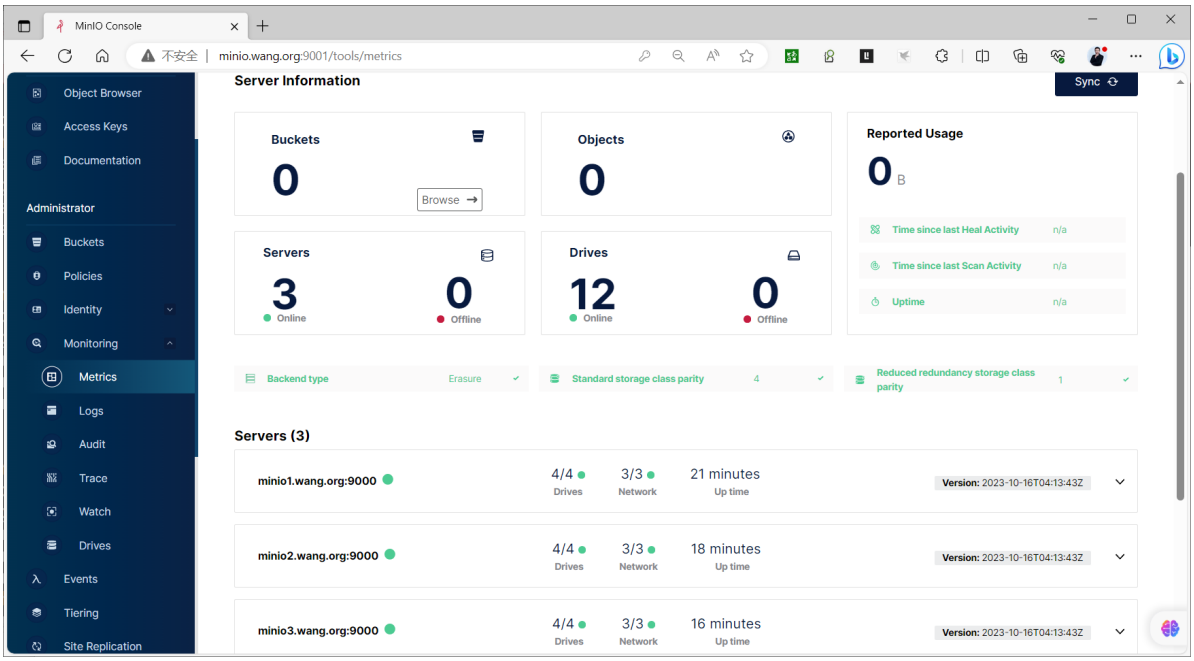
#通过Keepalived服务实现Nginx的高可用
略

#访问代理查看集群状态，用户名密码:admin/123456
http://minio.wang.org:9999/haproxy-status



#通过代理访问minio的控制台,用户名密码:admin/12345678
http://minio.wang.org:9001/





2.3.2.2 容器化部署分布式集群

范例：容器化部署

```
[root@localhost ~]# docker network create -d macvlan --subnet=192.168.100.0/24 --ip-range=192.168.100.0/24 --gateway=192.168.100.1 -o parent=eth0 minio-net

#启动三个节点的容器
[root@localhost ~]#docker run --name minio1 -d --restart always --network minio-net --ip=192.168.100.101 -v /data/node1/export1:/export1 -v /data/node1/export2:/export2 -e "MINIO_ROOT_USER=admin" -e "MINO_ROOT_PASSWORD-123456".-p 9000:9000 -p 9001:9001 minio/minio server http://192.168.100.10{1..3}/export{1..2}

[root@localhost ~]#docker run --name minio2 -d --restart always --network minio-net --ip=192.168.100.102 -v /data/node2/export1:/export1 -v /data/node2/export2:/export2 -e "MINIO_ROOT_USER=admin" -e "MINO_ROOT_PASSWORD-123456".-p 9000:9000 -p 9001:9001 minio/minio server http://192.168.100.10{1..3}/export{1..2}

[root@localhost ~]#docker run --name minio3 -d --restart always --network minio-net --ip=192.168.100.103 -v /data/node3/export1:/export1 -v /data/node3/export2:/export2 -e "MINIO_ROOT_USER=admin" -e "MINO_ROOT_PASSWORD-123456".-p 9000:9000 -p 9001:9001 minio/minio server http://192.168.100.10{1..3}/export{1..2}


[root@localhost ~]#docker logs minio1 minio2 minio3
```

2.4 基于Kubernetes 部署

```
https://github.com/minio/minio/tree/master/docs/orchestration/kubernetes
```

基于Kubernetes 部署 MinIO 有多种方法

- Manifest YAML 清单文件
- MinIO Operator
- Helm

2.4.1 YAML清单文件

环境要求: 依赖于一个支持PV动态置备的名称为sc-nfs的StorageClass

service 资源

```
#minio-service.yaml
---
kind: Service
apiVersion: v1
metadata:
  name: minio-headless
  labels:
    app: minio
spec:
  clusterIP: None
  #publishNotReadyAddresses: true
  selector:
    app: minio
  ports:
    - name: http
```

```
      port: 9000
      targetPort: 9000
---
apiVersion: v1
kind: Service
metadata:
  name: minio
spec:
  type: LoadBalancer
  selector:
    app: minio
  ports:
    - port: 9000
      targetPort: 9000
      name: http
    - port: 9001
      targetPort: 9001
      name: console
---
```

secret 资源

```
#minio-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: minio-secret
data:
  MINIO_ROOT_USER: YWRtaW4=
  # username: admin
  MINIO_ROOT_PASSWORD: MTIzNDU2Nzg=
  # root password: 12345678
```

statefulset 资源

```
#minio-statefulset.yaml
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: minio
  labels:
    app: minio
spec:
  serviceName: "minio-headless"
  replicas: 4
  podManagementPolicy: "Parallel" #并行启动pod，不配置的话模式是按顺序启动pod，minio、
nacos都需要配置并行启动
  selector:
    matchLabels:
      app: minio
  template:
    metadata:
      labels:
        app: minio
      annotations:
```

```
        prometheus.io/scrape: "true"
        prometheus.io/port: "9000"
        prometheus.io/path: "/minio/v2/metrics/node"
        #prometheus.io/path: "/minio/v2/metrics/cluster"
        #prometheus.io/path: "/minio/v2/metrics/bucket"
    spec:
      containers:
      - name: minio
        #image: minio/minio:RELEASE.2023-12-02T10-51-33Z
        image: minio/minio:RELEASE.2024-08-03T04-33-23Z
        volumeMounts:
        - name: data
          mountPath: /data
        args:
        - server
        - http://minio-{0...3}.minio-
headless.$(MINIO_POD_NAMESPACE).svc.cluster.local/data
        - "--address=:9000"
        - "--console-address=:9001"
        env:
        # MinIO access key and secret key
        - name: MINIO_ROOT_USER
          valueFrom:
            secretKeyRef:
              name: minio-secret
              key: MINIO_ROOT_USER
        - name: MINIO_ROOT_PASSWORD
          valueFrom:
            secretKeyRef:
              name: minio-secret
              key: MINIO_ROOT_PASSWORD
        - name: MINIO_POD_NAMESPACE
          valueFrom:
            fieldRef:
              fieldPath: metadata.namespace
        - name: MINIO_PROMETHEUS_AUTH_TYPE
          value: "public"
        livenessProbe:
          failureThreshold: 3
          httpGet:
            path: /minio/health/live
            port: http
            scheme: HTTP
          initialDelaySeconds: 120
          periodSeconds: 15
          successThreshold: 1
          timeoutSeconds: 10
        ports:
        - containerPort: 9000
          name: http
          protocol: TCP
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes:
        - ReadWriteOnce
      resources:
```

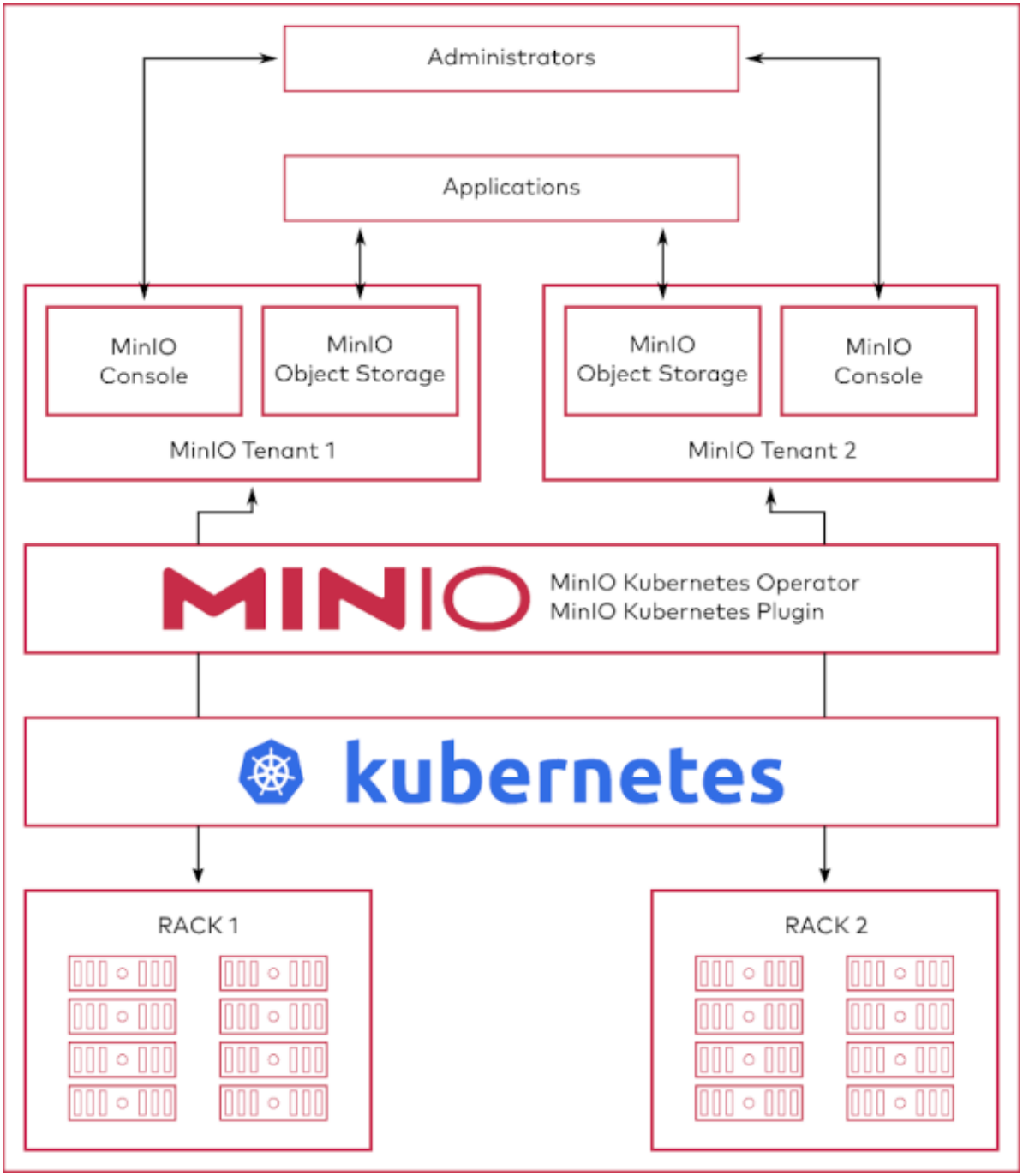
```
        requests:
          storage: 10Gi
      storageClassName: "sc-nfs"
```

ingress 资源

```
#minio-ingress.yaml
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: minio
spec:
  ingressClassName: nginx
  rules:
  - host: minio.wang.org
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: minio
            port:
              number: 9001
```

2.4.2 MinIO Operator

```
https://github.com/minio/operator/blob/master/README.md
```



2.4.3 Helm 安装

```
https://github.com/minio/minio/tree/master/helm/minio
```

Configure MinIO Helm repo

```
helm repo add minio https://charts.min.io/
```

Installing the Chart

Install this chart using:

```
helm install --namespace minio --set rootUser=rootuser,rootPassword=rootpass123 --generate-name minio/minio
```

The command deploys MinIO on the Kubernetes cluster in the default configuration. The configuration section lists the parameters that can be configured during installation.

Installing the Chart (toy-setup)

Minimal toy setup for testing purposes can be deployed using:

```
helm install --set resources.requests.memory=512Mi --set replicas=1 --set
persistence.enabled=false --set mode=standalone --set
rootUser=rootuser,rootPassword=rootpass123 --generate-name minio/minio
```

Upgrading the Chart

You can use Helm to update MinIO version in a live release. Assuming your release is named as `my-release`, get the values using the command:

```
helm get values my-release > old_values.yaml
```

Then change the field `image.tag` in `old_values.yaml` file with MinIO image tag you want to use. Now update the chart using

```
helm upgrade -f old_values.yaml my-release minio/minio
```

Default upgrade strategies are specified in the `values.yaml` file. Update these fields if you'd like to use a different strategy.

Configuration

Refer the Values file for all the possible config fields.

You can specify each parameter using the `--set key=value[,key=value]` argument to `helm install`. For example,

```
helm install --name my-release --set persistence.size=1Ti minio/minio
```

The above command deploys MinIO server with a 1Ti backing persistent volume.

Alternately, you can provide a YAML file that specifies parameter values while installing the chart. For example,

```
helm install --name my-release -f values.yaml minio/minio
```

Persistence

This chart provisions a PersistentVolumeClaim and mounts corresponding persistent volume to default location `/export`. You'll need physical storage available in the Kubernetes cluster for this to work. If you'd rather use `emptyDir`, disable PersistentVolumeClaim by:

```
helm install --set persistence.enabled=false minio/minio
```

> *"An emptyDir volume is first created when a Pod is assigned to a Node, and exists as long as that Pod is running on that node. When a Pod is removed from a node for any reason, the data in the emptyDir is deleted forever."*

Existing PersistentVolumeClaim

If a Persistent Volume Claim already exists, specify it during installation.

1. Create the PersistentVolume
2. Create the PersistentVolumeClaim
3. Install the chart

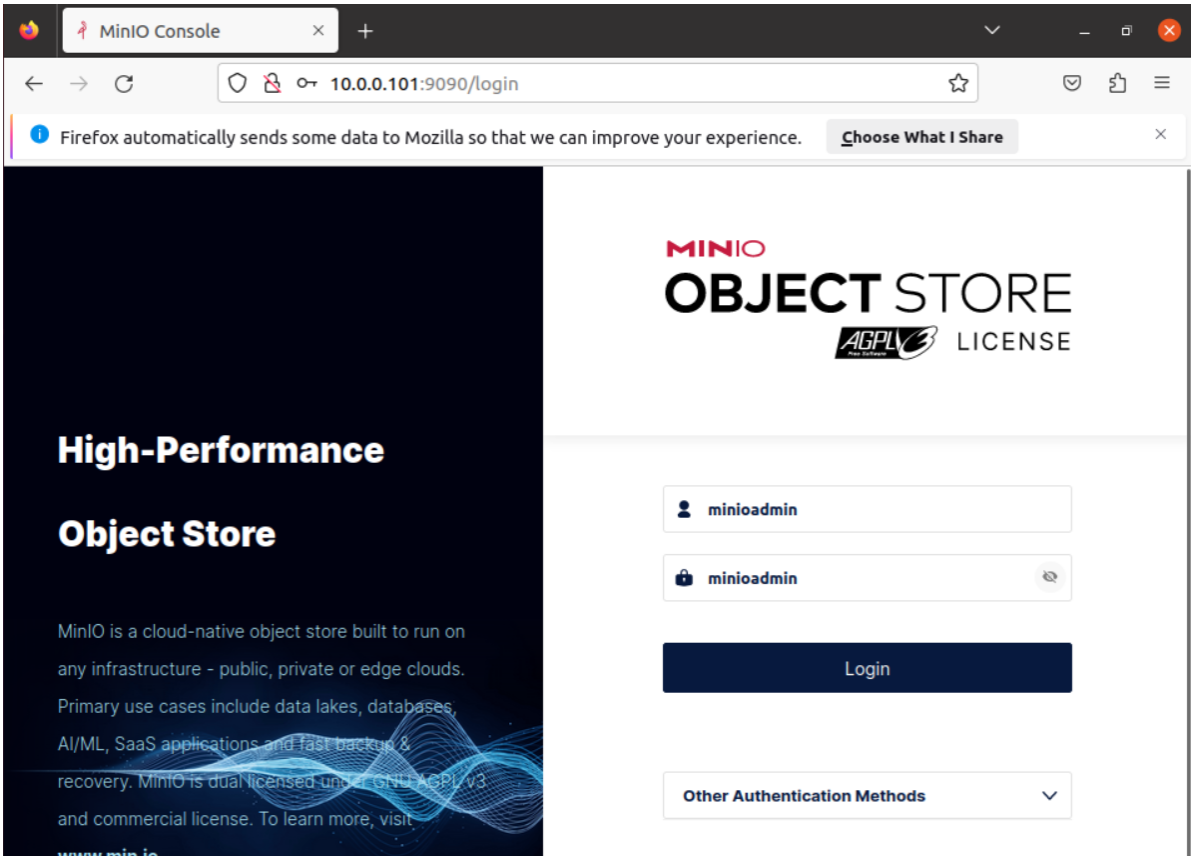
```
helm install --set persistence.existingClaim=PVC_NAME minio/minio
```

3 MINIO 使用

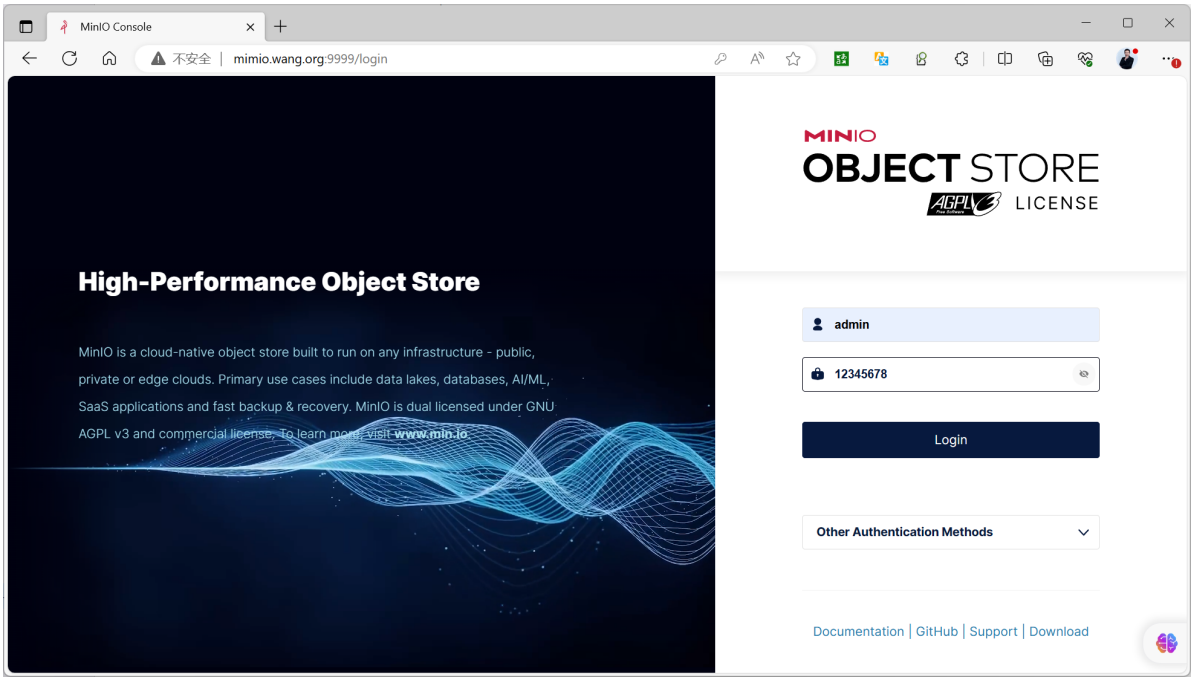
3.1 MINIO 控制台管理

3.1.1 登录控制台

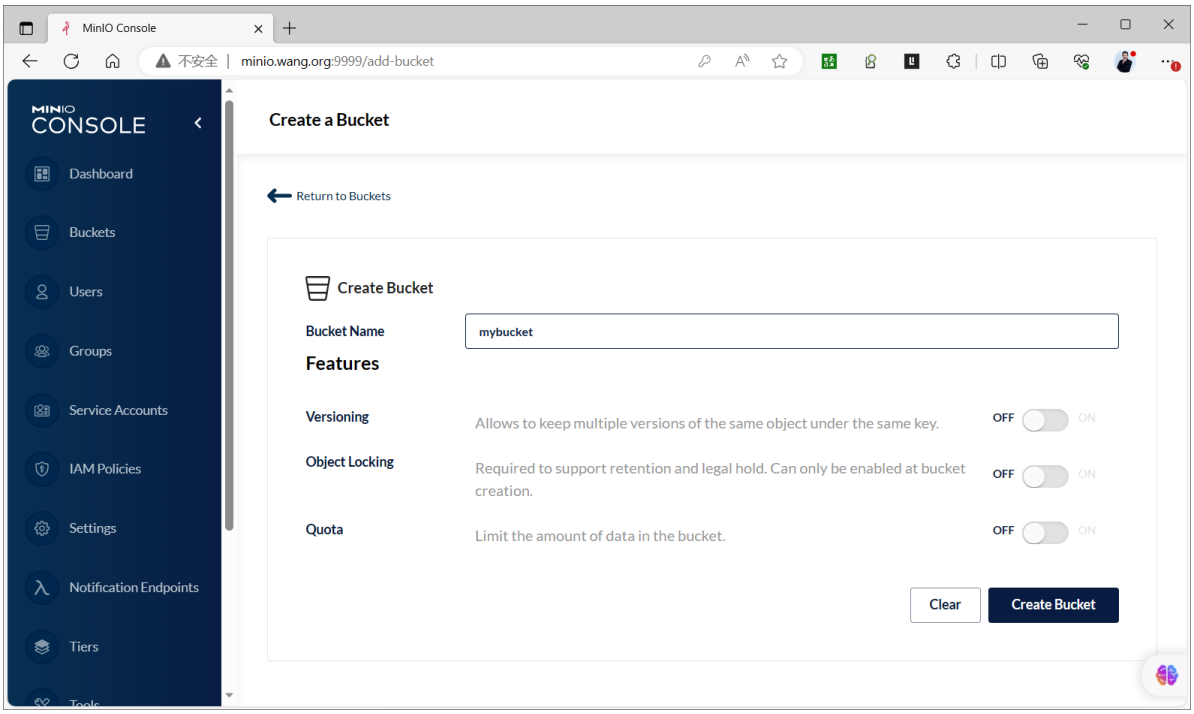
如果安装时没有指定用户密码信息，则登录默认用户和密码 minioadmin/minioadmin



使用指定用户和密码登录

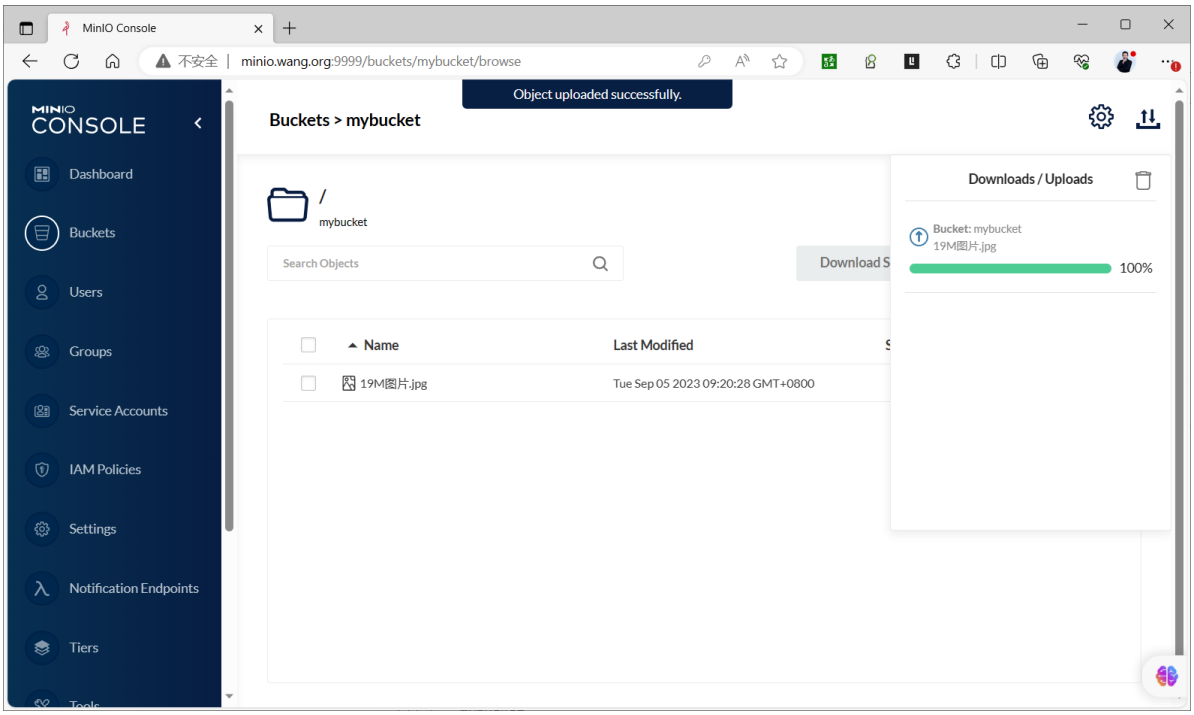


3.1.2 创建Bucket和上传文件




```
#查看生成目录
[root@ubuntu2204 ~]#tree /minio/
/minio/
├── data1
│   └── mybucket
├── data2
│   └── mybucket
├── data3
│   └── mybucket
├── data4
│   └── mybucket
├── data5
│   └── mybucket
├── data6
│   └── mybucket
```

```
├── data7
│   └── mybucket
└── data8
    └── mybucket

16 directories, 0 files

#上传文件19M图片的文件
```

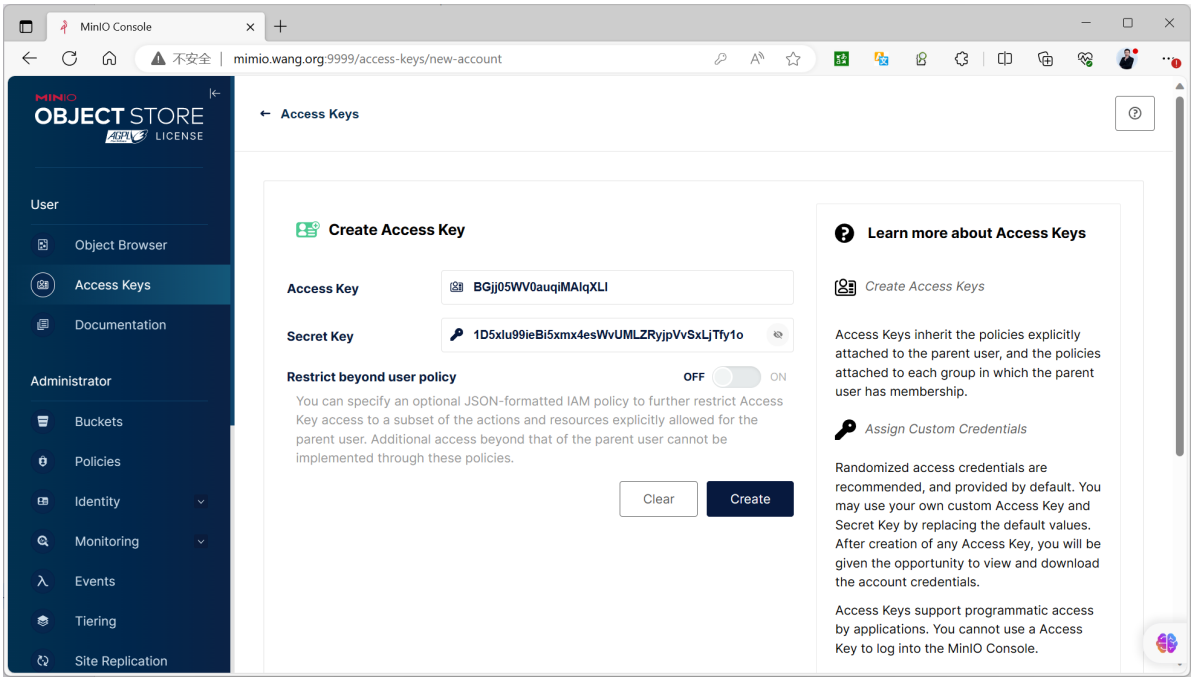



```
[root@ubuntu2204 ~]#tree /minio/
/minio/
├── data1
│   └── mybucket
│       └── 19M图片.jpg
│           ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
│           │   └── part.1
│           └── xl.meta
├── data2
│   └── mybucket
│       └── 19M图片.jpg
│           ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
│           │   └── part.1
│           └── xl.meta
├── data3
│   └── mybucket
│       └── 19M图片.jpg
│           ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
│           │   └── part.1
│           └── xl.meta
├── data4
│   └── mybucket
│       └── 19M图片.jpg
│           ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
│           │   └── part.1
│           └── xl.meta
├── data5
│   └── mybucket
```

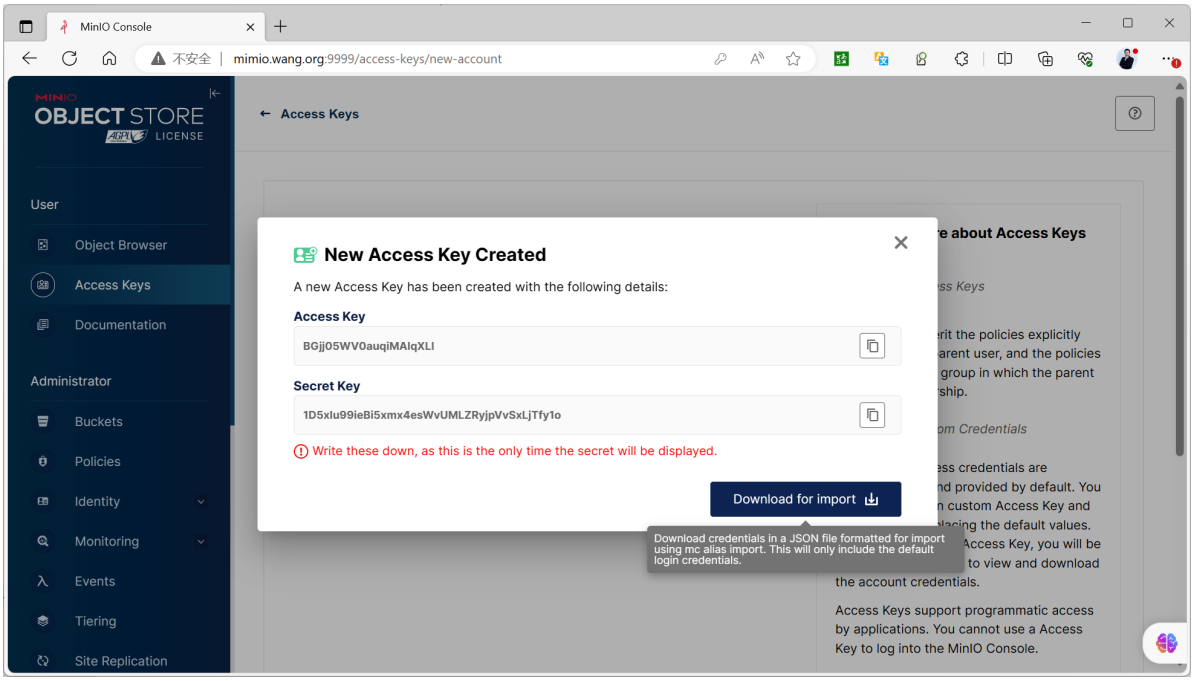
```
|       └── 19M图片.jpg
|           ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
|           |   └── part.1
|           └── xl.meta
├── data6
|   └── mybucket
|       └── 19M图片.jpg
|           ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
|           |   └── part.1
|           └── xl.meta
├── data7
|   └── mybucket
|       └── 19M图片.jpg
|           ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
|           |   └── part.1
|           └── xl.meta
└── data8
    └── mybucket
        └── 19M图片.jpg
            ├── ff0324e9-45d5-4cb3-af1e-b04cb9b948b2
            |   └── part.1
            └── xl.meta

32 directories, 16 files
[root@ubuntu2204 ~]#du -sh /minio/
45M /minio/
```

3.1.3 创建访问 Access key 和 Secret key

创建 Access Key 和 Secret key 用于访问数据的凭据



注意： 此处的 Secret key 是有一次性显示，需要立即复制保存，否则只能重新生成新的 Access key



3.2 MINIO 客户端 MC

3.2.1 MC 介绍

```
https://min.io/docs/minio/linux/reference/minio-mc.html
https://min.io/docs/minio/linux/reference/minio-mc-admin.html
```

MC是MinIO 客户端命令行工具 MinIO Client (mc)

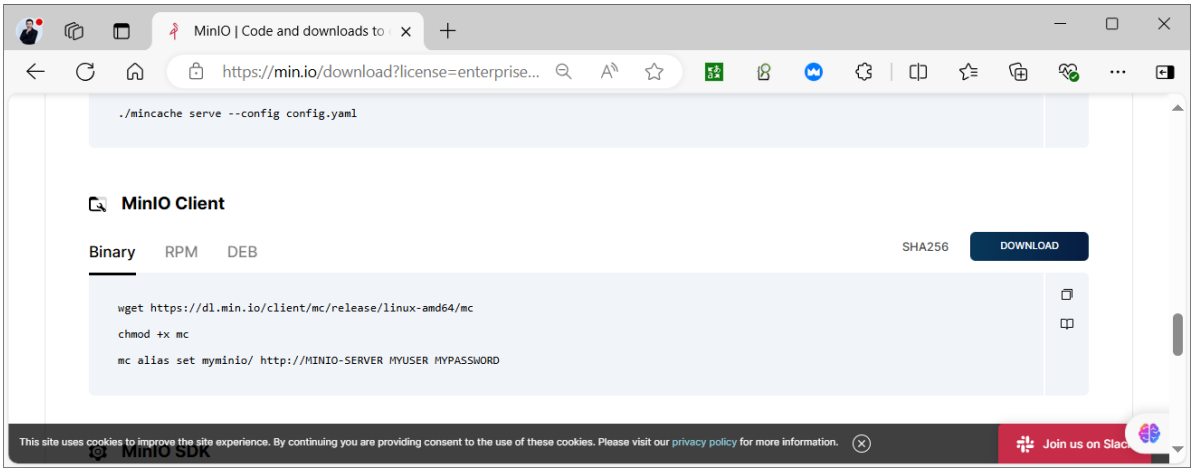
MC 可以用于访问MinIO的文件, 使用方法类似于常用的UNIX命令: 如:ls, cat, cp,rm,diff, find等

它支持文件系统和兼容Amazon S3的云存储服务(AWS Signature v2和v4)。

3.2.2 MC 部署

3.2.2.1 二进制部署

```
#二进制部署下载
https://dl.min.io/client/mc/release/linux-amd64/mc
http://dl.minio.org.cn/client/mc/release/linux-amd64/mc

https://min.io/download?
license=enterprise&platform=linux&subscription=enterprise-plus
```



范例: 二进制部署

```
#官网下载
[root@ubuntu2204 ~]#wget https://dl.min.io/client/mc/release/linux-amd64/mc
#国内镜像下载
[root@ubuntu2204 ~]#wget http://dl.minio.org.cn/client/mc/release/linux-amd64/mc

[root@ubuntu2204 ~]#install  mc /usr/local/bin/mc
```

3.2.2.2 包安装

```
https://min.io/download?
license=enterprise&platform=linux&subscription=enterprise-plus
```

DEB 包

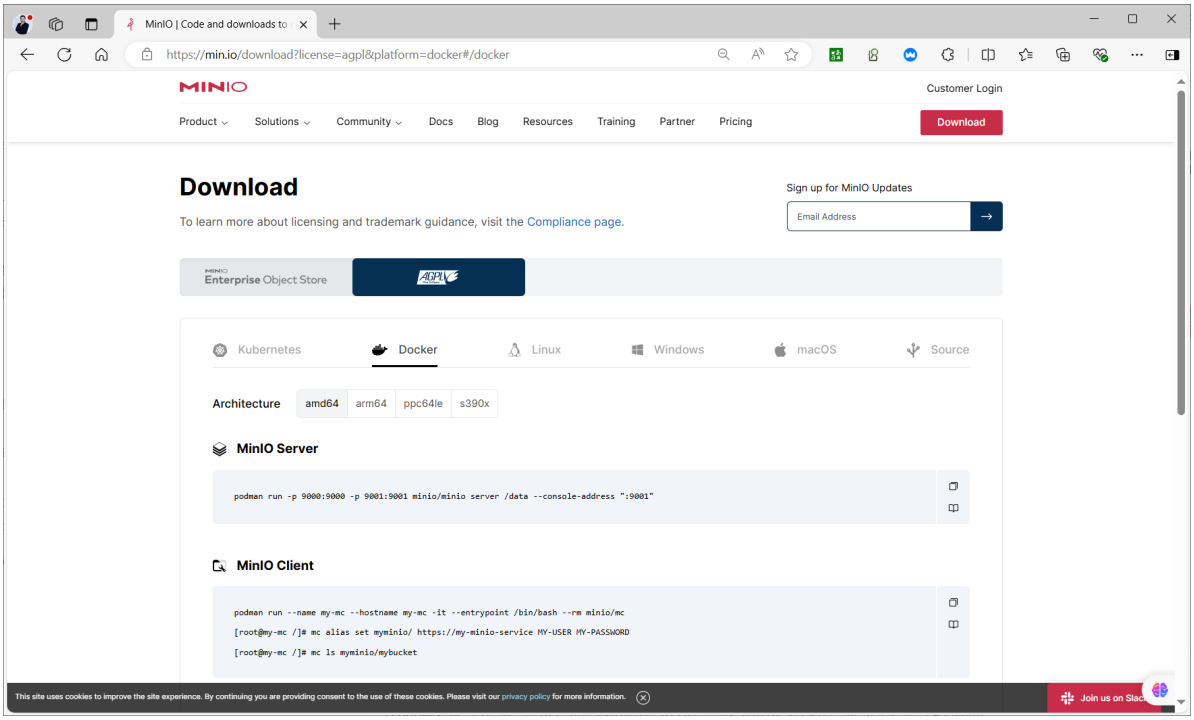
```
wget https://dl.min.io/client/mc/release/linux-
amd64/mcli_20240731155833.0.0_amd64.deb
dpkg -i mcli_20240731155833.0.0_amd64.deb
mcli alias set myminio/ http://MINIO-SERVER MYUSER MYPASSWORD
```

RPM 包

```
dnf install https://dl.min.io/client/mc/release/linux-amd64/mcli-
20240731155833.0.0-1.x86_64.rpm
mcli alias set myminio/ http://MINIO-SERVER MYUSER MYPASSWORD
```

3.2.2.3 容器化部署

```
#容器化部署
https://min.io/download#/docker
https://min.io/download?license=agpl&platform=docker#/docker
```



范例:容器化部署

```
[root@ubuntu2204 ~]#docker run --name my-mc --hostname my-mc -it --entrypoint
/bin/bash --rm minio/mc
[root@my-mc /]# mc alias set myminio/ https://my-minio-service MY-USER MY-
PASSWORD
[root@my-mc /]# mc ls myminio/mybucket
```

3.2.3 MC 命令使用

3.2.3.1 MC 命令说明

```
#帮助查看
mc --help
mc [GLOBALFLAGS] COMMAND --help
```

常见命令

```
ls #列出文件和文件夹
mb  #创建一个存储桶或一一个文件夹
cat  #显示文件和对象内容
pipe #一个STDIN重定向到一一个对象或者文件或者STDOUT
share #生成用于共享的URL
cp  # 拷贝文件和对象
mirror #给存储桶和文件夹做镜像
find  #基于参数查找文件
diff   #对两个文件夹或者存储桶比较差异
rm   #删除文件和对象
events #管理对象通知
watch #监视文件和对象的事件
policy #管理访问策略
config #管理mc配置文件
update #检查软件更新
version #输出版本信息
```

3.2.3.2 MC 使用

范例: 查看帮助

```
[root@ubuntu2204 ~]#mc
[root@ubuntu2204 ~]#mc --help
NAME:
  mc - MinIO Client for object storage and filesystems.

USAGE:
  mc [FLAGS] COMMAND [COMMAND FLAGS | -h] [ARGUMENTS...]

COMMANDS:
  alias      manage server credentials in configuration file
  ls         list buckets and objects
  mb         make a bucket
  rb         remove a bucket
  cp         copy objects
  mv         move objects
  rm         remove object(s)
  mirror     synchronize object(s) to a remote site
  cat        display object contents
```

```
  head        display first 'n' lines of an object
  pipe        stream STDIN to an object
  find        search for objects
  sql         run sql queries on objects
  stat        show object metadata
  tree        list buckets and objects in a tree format
  du          summarize disk usage recursively
  retention   set retention for object(s)
  legalhold   manage legal hold for object(s)
  support     support related commands
  license     license related commands
  share       generate URL for temporary access to an object
  version     manage bucket versioning
  ilm         manage bucket lifecycle
  quota       manage bucket quota
  encrypt     manage bucket encryption config
  event       manage object notifications
  watch       listen for object notification events
  undo        undo PUT/DELETE operations
  anonymous   manage anonymous access to buckets and objects
  tag         manage tags for bucket and object(s)
  diff        list differences in object name, size, and date between two buckets
  replicate   configure server side bucket replication
  admin       manage MinIO servers
  idp         manage MinIO IDentity Provider server configuration
  update      update mc to latest release
  ready       checks if the cluster is ready or not
  ping        perform liveness check
  od          measure single stream upload and download
  batch       manage batch jobs

GLOBAL FLAGS:
  --autocompletion              install auto-completion for your shell
  --config-dir value, -C value  path to configuration folder (default:
"/root/.mc")
  --quiet, -q                   disable progress bar display
  --no-color                    disable color theme
  --json                        enable JSON lines formatted output
  --debug                       enable debug output
  --insecure                    disable SSL certificate verification
  --limit-upload value          limits uploads to a maximum rate in KiB/s,
MiB/s, GiB/s. (default: unlimited)
  --limit-download value        limits downloads to a maximum rate in KiB/s,
MiB/s, GiB/s. (default: unlimited)
  --help, -h                    show help
  --version, -v                 print the version

TIP:
  Use 'mc --autocompletion' to enable shell autocompletion

COPYRIGHT:
  Copyright (c) 2015-2023 MinIO, Inc.

LICENSE:
  GNU AGPLv3 <https://www.gnu.org/licenses/agpl-3.0.html>
```

范例: 开启mc命令自动补全功能

```
[root@ubuntu2204 ~]#mc --autocompletion
mc: Your shell is set to 'bash', by env var 'SHELL'.
mc: enabled autocompletion in your 'bash' rc file. Please restart your shell.

#上面命令本质上就是修改.bashrc文件
[root@ubuntu2204 ~]#tail -n1 .bashrc
complete -C /usr/local/bin/mc mc

#需要重新登录才能生效
[root@ubuntu2204 ~]#mc
.cache/ data/   .mc/    snap/   .ssh/
[root@ubuntu2204 ~]#exit

#重新登录后,支持自动补会mc命令
[root@ubuntu2204 ~]#mc  <tab> <tab>
admin       cat         encrypt     idp         ls          od          rb          rm
        support     update
alias       cp          event       ilm         mb          ping        ready
share       tag         version
anonymous   diff        find        legalhold   mirror      pipe        replicate   sql
       tree        watch
batch       du          head        license     mv          quota       retention
stat        undo
```

范例: 查看客户端版本和连接信息

```
[root@ubuntu2204 ~]#mc --version
mc version RELEASE.2023-08-01T23-30-57Z (commit-
id=0a529d5e642f1a50a74b256c683be453e26bf7e9)
Runtime: go1.19.12 linux/amd64
Copyright (c) 2015-2023 MinIO, Inc.
License GNU AGPLv3 <https://www.gnu.org/licenses/agpl-3.0.html>

#查看默认连接信息
[root@ubuntu2204 ~]#mc alias ls
gcs
  URL       : https://storage.googleapis.com
  AccessKey : YOUR-ACCESS-KEY-HERE
  SecretKey : YOUR-SECRET-KEY-HERE
  API       : S3v2
  Path      : dns

local
  URL       : http://localhost:9000
  AccessKey :
  SecretKey :
  API       :
  Path      : auto

play
  URL       : https://play.min.io
  AccessKey : Q3AM3UQ867SPQQA43P2F
  SecretKey : zuf+tfteSlswRu7BJ86wekitnifILbZam1KYY3TG
  API       : S3v4
  Path      : auto
```

```
s3
  URL       : https://s3.amazonaws.com
  AccessKey : YOUR-ACCESS-KEY-HERE
  SecretKey : YOUR-SECRET-KEY-HERE
  API       : S3v4
  Path      : dns

[root@ubuntu2204 ~]#cat /root/.mc/config.json
{
    "version": "10",
    "aliases": {
        "gcs": {
            "url": "https://storage.googleapis.com",
            "accessKey": "YOUR-ACCESS-KEY-HERE",
            "secretKey": "YOUR-SECRET-KEY-HERE",
            "api": "S3v2",
            "path": "dns"
        },
        "local": {
            "url": "http://localhost:9000",
            "accessKey": "",
            "secretKey": "",
            "api": "S3v4",
            "path": "auto"
        },
        "play": {
            "url": "https://play.min.io",
            "accessKey": "Q3AM3UQ867SPQQA43P2F",
            "secretKey": "zuf+tfteSlswRu7BJ86wekitnifILbZam1KYY3TG",
            "api": "S3v4",
            "path": "auto"
        },
        "s3": {
            "url": "https://s3.amazonaws.com",
            "accessKey": "YOUR-ACCESS-KEY-HERE",
            "secretKey": "YOUR-SECRET-KEY-HERE",
            "api": "S3v4",
            "path": "dns"
        }
    }
}

#默认无法访问
[root@ubuntu2404 ~]#mc admin info local
mc: <ERROR> Unable to get service info. Access Denied.
```

范例: 初始连接配置单机存储

```
#建议使用key进行连接
[root@ubuntu2204 ~]#mc alias set minio-server http://minio.wang.org:9000
Ib4kdhJ3y5EOsbtcp8nH eTbjZq2RO6xBgpDNMIqOWQOo7z7rItaSDZEitAAY
Added `minio-server` successfully.

[root@ubuntu2204 ~]#mc admin info minio-server2
●  minio1.wang.org:9000
   Uptime: 2 hours
```

```
   Version: 2024-10-02T17:50:41Z
   Network: 3/3 OK
   Drives: 4/4 OK
   Pool: 1

•  minio2.wang.org:9000
   Uptime: 2 hours
   Version: 2024-10-02T17:50:41Z
   Network: 3/3 OK
   Drives: 4/4 OK
   Pool: 1

•  minio3.wang.org:9000
   Uptime: 2 hours
   Version: 2024-10-02T17:50:41Z
   Network: 3/3 OK
   Drives: 4/4 OK
   Pool: 1

┌──────┬──────────────────────┬──────────────────────┬──────────────┐
| Pool | Drives Usage         | Erasure stripe size | Erasure sets |
| 1st  | 1.4% (total: 80 GiB) | 12                   | 1            |
└──────┴──────────────────────┴──────────────────────┴──────────────┘

81 MiB Used, 2 Buckets, 17 Objects
12 drives online, 0 drives offline, EC:4


#使用用户名和密码连接
[root@ubuntu2204 ~]#mc alias set local http://127.0.0.1:9000 minioadmin
minioadmin
[root@ubuntu2204 ~]#mc config host ls
[root@ubuntu2204 ~]#mc admin info local

#验证后创建主机连接
[root@ubuntu2204 ~]#mc config host add local http://127.0.0.1:9000 admin
12345678
mc: Configuration written to `/root/.mc/config.json`. Please update your access
credentials.
mc: Successfully created `/root/.mc/share`.
mc: Initialized share uploads `/root/.mc/share/uploads.json` file.
mc: Initialized share downloads `/root/.mc/share/downloads.json` file.
Added `local` successfully.

#查看连接主机
[root@ubuntu2204 ~]#mc config host ls
gcs
  URL       : https://storage.googleapis.com
  AccessKey : YOUR-ACCESS-KEY-HERE
  SecretKey : YOUR-SECRET-KEY-HERE
  API       : S3v2
  Path      : dns

local
  URL       : http://127.0.0.1:9000
  AccessKey : minioadmin
  SecretKey : minioadmin
  API       : s3v4
```

```
  Path      : auto

play
  URL       : https://play.min.io
  AccessKey : Q3AM3UQ867SPQQA43P2F
  SecretKey : zuf+tfteSlswRu7BJ86wekitnifILbZam1KYY3TG
  API       : S3v4
  Path      : auto

s3
  URL       : https://s3.amazonaws.com
  AccessKey : YOUR-ACCESS-KEY-HERE
  SecretKey : YOUR-SECRET-KEY-HERE
  API       : S3v4
  Path      : dns

#配置存放在下面文件中
[root@ubuntu2204 ~]#cat .mc/config.json
{
    "version": "10",
    "aliases": {
        "gcs": {
            "url": "https://storage.googleapis.com",
            "accessKey": "YOUR-ACCESS-KEY-HERE",
            "secretKey": "YOUR-SECRET-KEY-HERE",
            "api": "S3v2",
            "path": "dns"
        },
        "local": {
            "url": "http://127.0.0.1:9000",
            "accessKey": "minioadmin",
            "secretKey": "minioadmin",
            "api": "S3v4",
            "path": "auto"
        },
        "play": {
            "url": "https://play.min.io",
            "accessKey": "Q3AM3UQ867SPQQA43P2F",
            "secretKey": "zuf+tfteSlswRu7BJ86wekitnifILbZam1KYY3TG",
            "api": "S3v4",
            "path": "auto"
        },
        "s3": {
            "url": "https://s3.amazonaws.com",
            "accessKey": "YOUR-ACCESS-KEY-HERE",
            "secretKey": "YOUR-SECRET-KEY-HERE",
            "api": "S3v4",
            "path": "dns"
        }
    }
}


#单机显示
[root@ubuntu2204 ~]#mc admin info local
●  127.0.0.1:9000
   Uptime: 21 minutes
   Version: 2023-08-29T23:07:35Z
```

```
   Network: 1/1 OK
   Drives: 1/1 OK
   Pool: 1

Pools:
   1st, Erasure sets: 1, Drives per erasure set: 1

0 B Used, 1 Bucket, 0 Objects
1 drive online, 0 drives offline

#查看存储中的数据文件
[root@ubuntu2204 ~]#mc ls local
[2023-09-05 09:19:38 CST]     0B mybucket/

[root@ubuntu2204 ~]#mc ls local/mybucket
[2023-09-05 09:20:28 CST]  18MiB STANDARD 19M图片.jpg
```

范例: 管理分布式存储

```
#添加分布式存储的任意一个节点连接
[root@ubuntu2204 ~]#mc config host add minio-cluster http://10.0.0.101:9000
admin 12345678

#查看连接信息
[root@ubuntu2204 ~]#mc config host ls
gcs
  URL       : https://storage.googleapis.com
  AccessKey : YOUR-ACCESS-KEY-HERE
  SecretKey : YOUR-SECRET-KEY-HERE
  API       : S3v2
  Path      : dns

local
  URL       : http://localhost:9000
  AccessKey :
  SecretKey :
  API       :
  Path      : auto

minio-cluster
  URL       : http://10.0.0.101:9000
  AccessKey : admin
  SecretKey : 12345678
  API       : s3v4
  Path      : auto

play
  URL       : https://play.min.io
  AccessKey : Q3AM3UQ867SPQQA43P2F
  SecretKey : zuf+tfteSlswRu7BJ86wekitnifILbZam1KYY3TG
  API       : S3v4
  Path      : auto

s3
  URL       : https://s3.amazonaws.com
  AccessKey : YOUR-ACCESS-KEY-HERE
  SecretKey : YOUR-SECRET-KEY-HERE
```

```
  API       : S3v4
  Path      : dns

#集群显示
[root@ubuntu2204 ~]#mc admin info minio-cluster
●  10.0.0.101:9000
   Uptime: 2 hours
   Version: 2023-08-29T23:07:35Z
   Network: 3/3 OK
   Drives: 1/1 OK
   Pool: 1

●  10.0.0.102:9000
   Uptime: 2 hours
   Version: 2023-08-29T23:07:35Z
   Network: 3/3 OK
   Drives: 1/1 OK
   Pool: 1

●  10.0.0.103:9000
   Uptime: 2 hours
   Version: 2023-08-29T23:07:35Z
   Network: 3/3 OK
   Drives: 1/1 OK
   Pool: 1

Pools:
   1st, Erasure sets: 1, Drives per erasure set: 3

18 MiB Used, 1 Bucket, 1 Object
3 drives online, 0 drives offline

#测试连通
[root@ubuntu2204 ~]#mc ping -c 1 minio-cluster
  1: http://10.0.0.101:9000:9000   min=1.92ms     max=1.92ms     average=1.92ms
    errors=0   roundtrip=1.92ms

#查看文件
[root@ubuntu2204 ~]#mc ls minio-cluster
[2023-09-05 11:26:05 CST]     0B mybucket/
[root@ubuntu2204 ~]#mc ls minio-cluster/mybucket
[2023-09-05 14:05:33 CST]  12MiB STANDARD 11M.jpg
[2023-09-05 11:27:21 CST]  18MiB STANDARD image-19M.jpg

#查看存储目录大小
[root@ubuntu2204 ~]#mc du minio-cluster
30MiB   2 objects

#删除连接信息
[root@ubuntu2204 ~]#mc config host rm  minio-cluster
Removed `minio-cluster` successfully.

#确认删除
[root@ubuntu2204 ~]#mc config host ls
```

范例: 文件下载上传

```
[root@ubuntu2204 ~]#mc ls minio-cluster/mybucket
[2023-09-05 14:05:33 CST]  12MiB STANDARD 11M.jpg
[2023-09-05 11:27:21 CST]  18MiB STANDARD image-19M.jpg

#下载
[root@ubuntu2204 ~]#mc cp minio-cluster/mybucket/11M.jpg .
...000/mybucket/11M.jpg: 11.51 MiB / 11.51 MiB
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 152.93 MiB/s 0s

[root@ubuntu2204 ~]#ls
11M.jpg  data  mc  snap

#上传
[root@ubuntu2204 ~]#mc cp /var/log/syslog minio-cluster/mybucket
/var/log/syslog:         717.02 KiB / 717.02 KiB
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 22.96 MiB/s
0s[root@ubuntu2204 ~]#mc ls minio-cluster/mybucket
[2023-09-05 14:05:33 CST]  12MiB STANDARD 11M.jpg
[2023-09-05 11:27:21 CST]  18MiB STANDARD image-19M.jpg
[2023-09-05 14:44:16 CST] 717KiB STANDARD syslog


#上传目录,需要加-r选项,注意:目录以/后缀表示只上传目录的内容,并不上传目录本身
[root@ubuntu2204 ~]#mc cp -r /etc/netplan/ minio-cluster/mybucket
...nstaller-config.yaml: 754 B / 754 B
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 32.28
KiB/s 0s[root@ubuntu2204 ~]#mc ls minio-cluster/mybucket
[2023-09-05 14:47:54 CST]   305B STANDARD 01-netcfg.yaml
[2023-09-05 14:47:54 CST]   333B STANDARD 01-netcfg.yaml.bak
[2023-09-05 14:05:33 CST]  12MiB STANDARD 11M.jpg
[2023-09-05 11:27:21 CST]  18MiB STANDARD image-19M.jpg
[2023-09-05 14:44:16 CST] 717KiB STANDARD syslog
[2023-09-05 14:48:00 CST]     0B bak/

#上传目录,需要加-r选项,注意:目录不以/后缀表示会上传目录本身及内容
[root@ubuntu2204 ~]#mc cp -r /etc/netplan minio-cluster/mybucket
...nstaller-config.yaml: 754 B / 754 B
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 28.04
KiB/s 0s[root@ubuntu2204 ~]#mc ls minio-cluster/mybucket
[2023-09-05 14:47:54 CST]   305B STANDARD 01-netcfg.yaml
[2023-09-05 14:47:54 CST]   333B STANDARD 01-netcfg.yaml.bak
[2023-09-05 14:05:33 CST]  12MiB STANDARD 11M.jpg
[2023-09-05 11:27:21 CST]  18MiB STANDARD image-19M.jpg
[2023-09-05 14:44:16 CST] 717KiB STANDARD syslog
[2023-09-05 14:48:13 CST]     0B bak/
[2023-09-05 14:48:13 CST]     0B netplan/

#删除多个文件
[root@ubuntu2204 ~]#mc rm  minio-cluster/mybucket/01-netcfg.yaml.bak minio-
cluster/mybucket/01-netcfg.yaml


#查看目录结构
[root@ubuntu2204 ~]#mc tree minio-cluster
minio-cluster
└─ mybucket
   └─ netplan
      └─ bak
```

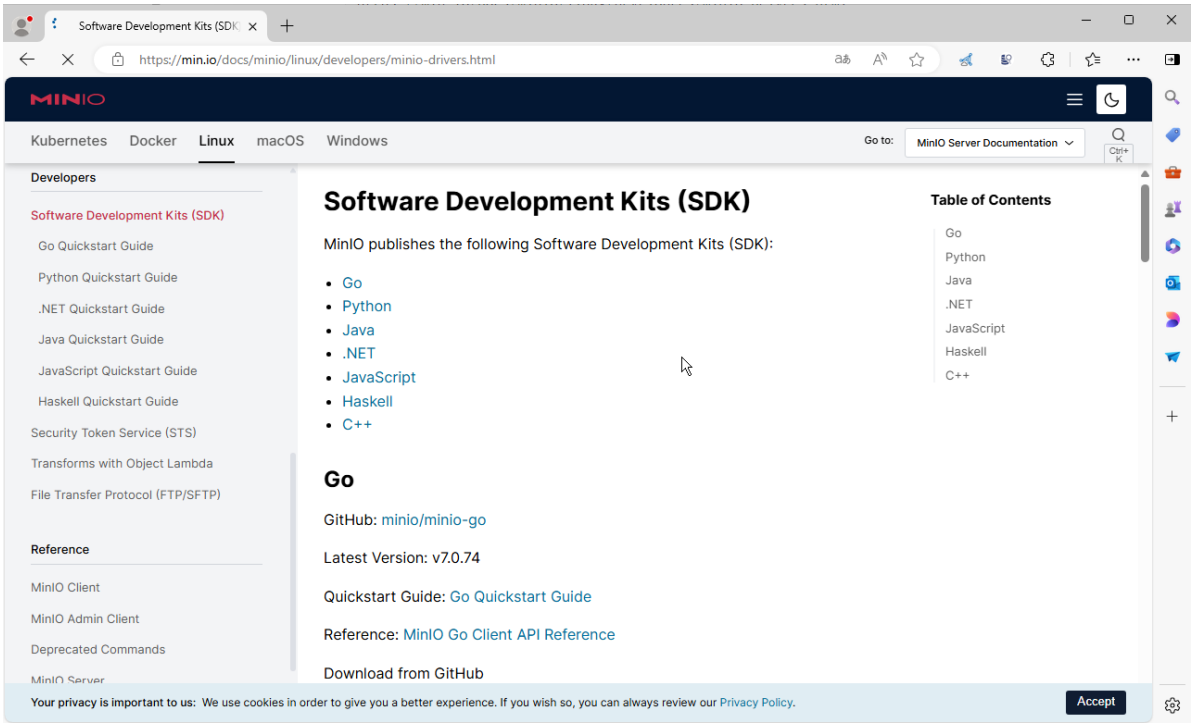
```
#删除目录
[root@ubuntu2204 ~]#mc rm -r --force  minio-cluster/mybucket/netplan/bak
Removed `minio-cluster/mybucket/netplan/bak/00-installer-config.yaml`.
[root@ubuntu2204 ~]#mc tree minio-cluster
minio-cluster
└─ mybucket
   └─ netplan
```

范例: 管理bucket

```
#创建bucket
[root@ubuntu2204 ~]#mc mb minio-cluster/bucket01
Bucket created successfully `minio-cluster/bucket01`.

[root@ubuntu2204 ~]#mc ls minio-cluster
[2023-09-05 15:00:06 CST]     0B bucket01/
[2023-09-05 11:26:05 CST]     0B mybucket/

#上传文件
[root@ubuntu2204 ~]#mc cp /etc/issue minio-cluster/bucket01

[root@ubuntu2204 ~]#mc tree -f  minio-cluster/bucket01
minio-cluster/bucket01
└─ issue

#不能直接删除不为空的bucket
[root@ubuntu2204 ~]#mc rb minio-cluster/bucket01
mc: <ERROR> `minio-cluster/bucket01` is not empty. Retry this command with '--
force' flag if you want to remove `minio-cluster/bucket01` and all its contents

#强制删除不为空的bucket
[root@ubuntu2204 ~]#mc rb --force  minio-cluster/bucket01
Removed `minio-cluster/bucket01` successfully.

#确认删除
[root@ubuntu2204 ~]#mc ls minio-cluster
[2023-09-05 11:26:05 CST]     0B mybucket/
```

3.3 编程语言API访问

```
https://min.io/docs/minio/linux/developers/minio-drivers.html
```



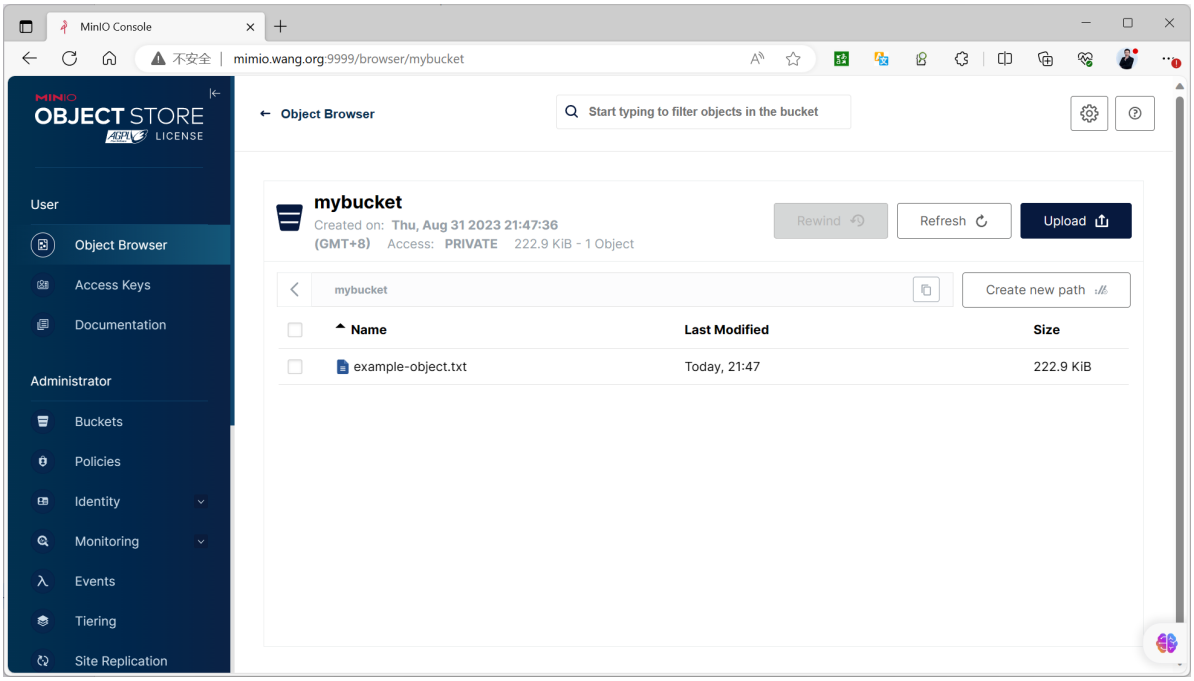
3.3.1 Python 访问

```
https://min.io/docs/minio/linux/developers/minio-drivers.html#python-sdk
https://min.io/docs/minio/linux/developers/python/minio-py.html
https://github.com/minio/minio-py/tree/master/examples
```

范例：

```
[root@ubuntu2204 ~]#apt update && apt -y install python3-pip
[root@ubuntu2204 ~]#python3 -m pip config set global.index-url
https://mirrors.aliyun.com/pypi/simple/

[root@ubuntu2404 ~]#pip3 install minio --break-system-packages
[root@ubuntu2204 ~]#pip3 install minio

#使用以下代码作为示例来访问 MinIO 服务器
[root@ubuntu2204 ~]#cat access_minio.py
#!/usr/bin/python3
import os
from minio import Minio
from minio.error import S3Error
#from minio.error import (ResponseError, BucketAlreadyOwnedByYou,
BucketAlreadyExists)

# 初始化MinIO客户端
minio_client = Minio(
    endpoint='localhost:9000',
    access_key="kH3qu9EQdPY6L7Y4bvA4",
    secret_key="jC7ravHdJG5Gkre5T9sNrhXECWJF5suNIYkBrxu5",
    secure=False  # 如果使用SSL，请将此项设置为True
)

bucket_name = "mybucket"
object_name = "example-object.txt"
local_file_path = "/var/log/syslog"
```

```
try:
    # 检查存储桶是否存在，如果不存在则创建
    if not minio_client.bucket_exists(bucket_name):
        minio_client.make_bucket(bucket_name)
        print(f"Bucket '{bucket_name}' created successfully.")
    else:
        print(f"Bucket '{bucket_name}' already exists.")

    # 上传文件到存储桶
    minio_client.fput_object(bucket_name, object_name, local_file_path)
    print(f"Successfully uploaded '{local_file_path}' to '{object_name}'.")
except ResponseError as e:
    print(f"An error occurred: {e}")

# 下载存储桶中的对象
downloaded_file_path = "example-object.txt"
try:
    minio_client.fget_object(bucket_name, object_name, downloaded_file_path)
    print(f"Successfully downloaded '{object_name}' to
'{downloaded_file_path}'.")
except ResponseError as e:
    print(f"An error occurred while downloading '{object_name}': {e}")

#运行程序
[root@ubuntu2204 ~]#python3 access_minio.py
Bucket 'mybucket' created successfully.
Successfully uploaded '/var/log/syslog' to 'example-object.txt'.
Successfully downloaded 'example-object.txt' to 'example-object.txt'.

#查看存储的数据文件
[root@ubuntu2204 ~]#tree /data/minio/
/data/minio/
└── mybucket
    └── example-object.txt
        ├── 16add02d-b997-49fc-b03e-637856b3c1be
        │   └── part.1
        └── xl.meta

3 directories, 2 files
```



3.3.2 Golang 访问

```
https://min.io/docs/minio/linux/developers/go/minio-go.html
```

在运行代码之前，确保已经安装了 Go，并且已经启动了一个 MinIO 服务器并获取accessKey和secretKey

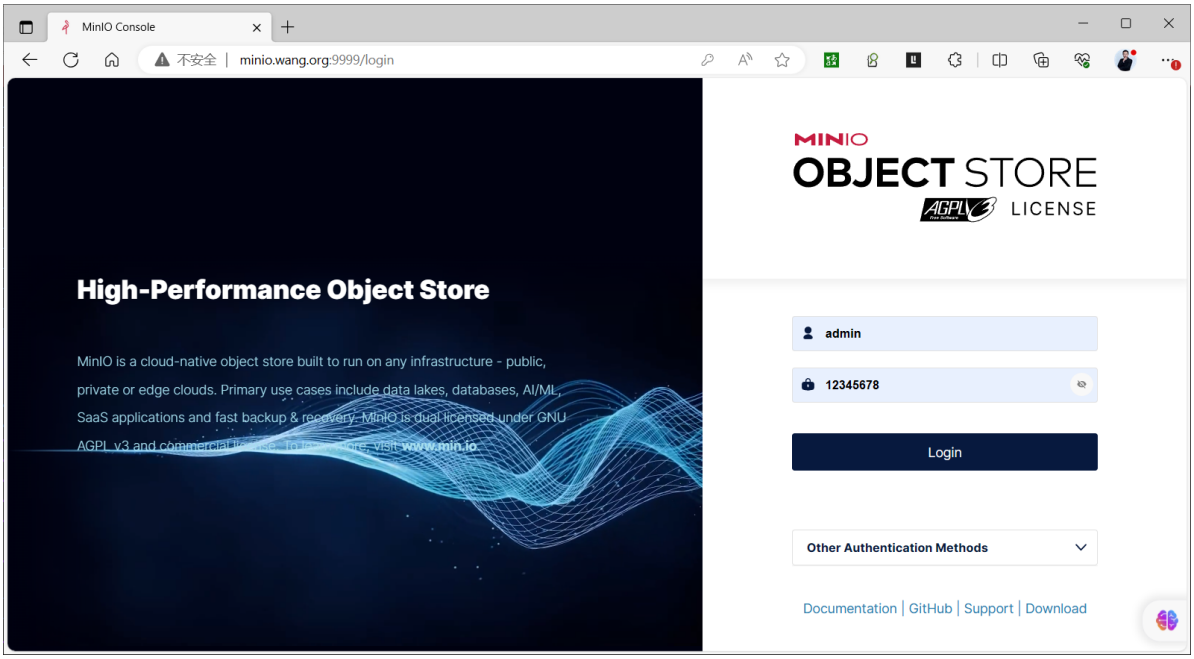
注意：要求golang-v1.20以上版本

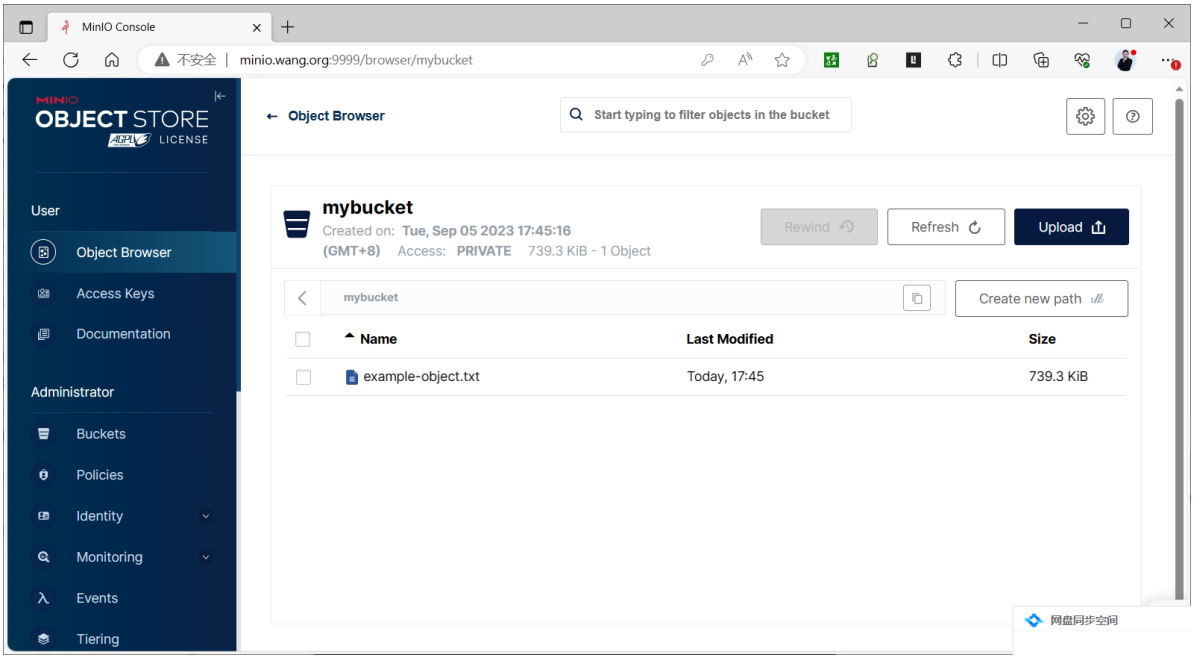
```
#示例代码
[root@ubuntu2204 minio-demo]#ls
main.go
[root@ubuntu2204 minio-demo]#cat main.go
package main

import (
    "context"
    "fmt"
    "log"
    "github.com/minio/minio-go/v7"
    "github.com/minio/minio-go/v7/pkg/credentials"
)

func main() {
    endpoint := "localhost:9000"
    accessKey := "xCLmTTXFrBGRYCY2eqh7"                    //指定事先生成的
accessKey,也可以使用MINIO_ROOT_USER
    secretKey := "SmfdeMiZwRWIfoO4iKwkaGe0scBxDmbQvNWCr1"  //指定事先生成的
secretKey,也可以使用MINIO_ROOT_PASSWORD
    useSSL := false // Change to true if you're using SSL

    // Initialize a new MinIO client object.
    minioClient, err := minio.New(endpoint, &minio.Options{
        Creds:  credentials.NewStaticV4(accessKey, secretKey, ""),
        Secure: useSSL,
    })
    if err != nil {
```

```
        log.Fatalln(err)
    }

    bucketName := "mybucket"
    objectName := "example-object.txt"
    localFilePath := "/var/log/syslog"

    ctx := context.Background()

    // Create a new bucket if it doesn't exist.
    err = minioClient.MakeBucket(ctx, bucketName, minio.MakeBucketOptions{})
    if err != nil {
        exists, errBucketExists := minioClient.BucketExists(ctx, bucketName)
        if errBucketExists == nil && exists {
            fmt.Printf("Bucket '%s' already exists.\n", bucketName)
        } else {
            log.Fatalln(err)
        }
    } else {
        fmt.Printf("Created bucket '%s'.\n", bucketName)
    }

    // Upload a file to the bucket.
    _, err = minioClient.FPutObject(ctx, bucketName, objectName, localFilePath,
minio.PutObjectOptions{})
    if err != nil {
        log.Fatalln(err)
    }
    fmt.Printf("Successfully uploaded '%s' to '%s'.\n", localFilePath,
objectName)

    // Download the uploaded object.
    downloadFilePath := "downloaded/file.txt"
    err = minioClient.FGetObject(ctx, bucketName, objectName, downloadFilePath,
minio.GetObjectOptions{})
    if err != nil {
        log.Fatalln(err)
    }
    fmt.Printf("Successfully downloaded '%s' to '%s'.\n", objectName,
downloadFilePath)
}

#安装golang，注意：要求go-v1.20以上版本
[root@ubuntu2204 minio-demo]#bash install_go.sh

#配置代理
[root@ubuntu2204 minio-demo]#export GOPROXY=https://goproxy.cn

#初始化项和,指定默认的程序名为minio-demo
[root@ubuntu2204 minio-demo]#go mod init minio-demo
go: creating new go.mod: module minio-demo
go: to add module requirements and sums:
    go mod tidy

#生成初始化文件
[root@ubuntu2204 minio-demo]#ls
go.mod  main.go
```

```
[root@ubuntu2204 minio-demo]#cat go.mod
module minio-demo

go 1.18

#下载相关依赖
[root@ubuntu2204 minio-demo]#go get github.com/minio/minio-go/v7
go: added github.com/dustin/go-humanize v1.0.1
go: added github.com/google/uuid v1.3.0
go: added github.com/json-iterator/go v1.1.12
go: added github.com/klauspost/compress v1.16.7
go: added github.com/klauspost/cpuid/v2 v2.2.5
go: added github.com/minio/md5-simd v1.1.2
go: added github.com/minio/minio-go/v7 v7.0.63
go: added github.com/minio/sha256-simd v1.0.1
go: added github.com/modern-go/concurrent v0.0.0-20180306012644-bacd9c7ef1dd
go: added github.com/modern-go/reflect2 v1.0.2
go: added github.com/rs/xid v1.5.0
go: added github.com/sirupsen/logrus v1.9.3
go: added golang.org/x/crypto v0.12.0
go: added golang.org/x/net v0.14.0
go: added golang.org/x/sys v0.11.0
go: added golang.org/x/text v0.12.0
go: added gopkg.in/ini.v1 v1.67.0

[root@ubuntu2204 minio-demo]#ls
go.mod  go.sum  main.go

[root@ubuntu2204 minio-demo]#cat go.mod
module minio-demo

go 1.18

require (
    github.com/dustin/go-humanize v1.0.1 // indirect
    github.com/google/uuid v1.3.0 // indirect
    github.com/json-iterator/go v1.1.12 // indirect
    github.com/klauspost/compress v1.16.7 // indirect
    github.com/klauspost/cpuid/v2 v2.2.5 // indirect
    github.com/minio/md5-simd v1.1.2 // indirect
    github.com/minio/minio-go/v7 v7.0.63 // indirect
    github.com/minio/sha256-simd v1.0.1 // indirect
    github.com/modern-go/concurrent v0.0.0-20180306012644-bacd9c7ef1dd //
indirect
    github.com/modern-go/reflect2 v1.0.2 // indirect
    github.com/rs/xid v1.5.0 // indirect
    github.com/sirupsen/logrus v1.9.3 // indirect
    golang.org/x/crypto v0.12.0 // indirect
    golang.org/x/net v0.14.0 // indirect
    golang.org/x/sys v0.11.0 // indirect
    golang.org/x/text v0.12.0 // indirect
    gopkg.in/ini.v1 v1.67.0 // indirect
)
[root@ubuntu2204 minio-demo]#cat go.sum
github.com/davecgh/go-spew v1.1.0/go.mod
h1:J7Y8YcW2NihsgmVo/mv3lAwl/skON4iLHjSsI+c5H38=
github.com/davecgh/go-spew v1.1.1/go.mod
h1:J7Y8YcW2NihsgmVo/mv3lAwl/skON4iLHjSsI+c5H38=
```

```
github.com/dustin/go-humanize v1.0.1
h1:GzkhY7T5VNhEkwHOPVJgjz+fX1rhBrR7pRT3mDkpeCY=
github.com/dustin/go-humanize v1.0.1/go.mod
h1:Mu1zIs6XwVuF/gI1OepvI0qD18qycQx+mFykh5fBlto=
......

#编译默认生成项目名称的二进制程序
[root@ubuntu2204 minio-demo]#CGO_ENABLED=0 go build

#或者编译生成指定的二进制程序名
[root@ubuntu2204 minio-demo]#CGO_ENABLED=0 go build -o minio-demo
[root@ubuntu2204 minio-demo]#ls
go.mod  go.sum  main.go  minio-demo

#运行
[root@ubuntu2204 minio-demo]#./minio-demo
Created bucket 'mybucket'.
Successfully uploaded '/var/log/syslog' to 'example-object.txt'.
Successfully downloaded 'example-object.txt' to 'downloaded/file.txt'.

[root@ubuntu2204 minio-demo]#tree
.
├── downloaded
│   └── file.txt
├── go.mod
├── go.sum
├── main.go
└── minio-demo

1 directory, 5 files
[root@ubuntu2204 minio-demo]#diff downloaded/file.txt /var/log/syslog
#登录验证文件上传成功
```





3.3.3 JAVA 访问

```
https://min.io/docs/minio/linux/developers/java/minio-java.html
```

范例

```
import io.minio.BucketExistsArgs;
import io.minio.MakeBucketArgs;
import io.minio.MinioClient;
import io.minio.UploadObjectArgs;
import io.minio.errors.MinioException;
import java.io.IOException;
import java.security.InvalidKeyException;
import java.security.NoSuchAlgorithmException;

public class FileUploader {
  public static void main(String[] args)
      throws IOException, NoSuchAlgorithmException, InvalidKeyException {
    try {
      // Create a minioClient with the MinIO server playground, its access key
and secret key.
      MinioClient minioClient =
          MinioClient.builder()
              .endpoint("https://play.min.io")
              .credentials("Q3AM3UQ867SPQQA43P2F",
"zuf+tfteSlswRu7BJ86wekitnifILbZam1KYY3TG")
              .build();

      // Make 'asiatrip' bucket if not exist.
      boolean found =

 minioClient.bucketExists(BucketExistsArgs.builder().bucket("asiatrip").build())
;
      if (!found) {
        // Make a new bucket called 'asiatrip'.

 minioClient.makeBucket(MakeBucketArgs.builder().bucket("asiatrip").build());
```

```
      } else {
        System.out.println("Bucket 'asiatrip' already exists.");
      }

      // Upload '/home/user/Photos/asiaphotos.zip' as object name 'asiaphotos-
2015.zip' to bucket
      // 'asiatrip'.
      minioClient.uploadObject(
          UploadObjectArgs.builder()
              .bucket("asiatrip")
              .object("asiaphotos-2015.zip")
              .filename("/home/user/Photos/asiaphotos.zip")
              .build());
      System.out.println(
          "'/home/user/Photos/asiaphotos.zip' is successfully uploaded as "
              + "object 'asiaphotos-2015.zip' to bucket 'asiatrip'.");
    } catch (MinioException e) {
      System.out.println("Error occurred: " + e);
      System.out.println("HTTP trace: " + e.httpTrace());
    }
  }
}
```

4 MINIO 故障恢复

minio集群有纠删码机制，即使在集群数据盘挂掉一半的情况下，集群中数据也是安全的。

但是如果集群想要正常读写,就需要有N/2+1的节点数才可以正常读写

如果现有minio集群有节点出现故障，就需要更换节点

注意事项

- 如果更换节点旧节点数据量较大，在节点更换时可以正常使用请先备份原有节点数据到新节点，避免同步的数据过多导致网络带宽被占用
- 如果数据量小，可以不进行备份数据，直接进行更换，节点启动完毕会自动同步数据
- 如果节点挂掉时集群还在读写数据，会导致集群挂掉的节点与其他minio节点数据不同，这里在恢复节点后需修复数据（自动修复，无需人为干预）
- 最好部署minio集群时使用hosts文件做地址解析，避免更换节点时修改minio配置文件参数
- 更换节点时需要停止minio集群客户端的读写
- 更换的新节点所有配置信息要和旧节点保持一致，包括minio版本，配置文件，hosts解析文件，数据目录位置以及大小

4.1 节点服务故障重启后自动恢复

如果在写入数据时,节点服务故障,当节点服务启动后,会自动同步数据

范例：节点服务故障重启后自动恢复

```
#正在写入数据时,将某个节点服务停止
[root@minio2 ~]#systemctl stop minio

[root@ubuntu2204 ~]#mc admin  info minio-cluster
●  minio1.wang.org:9000
   Uptime: 3 minutes
   Version: 2023-10-16T04:13:43Z
   Network: 2/3 OK
```

```
   Drives: 4/4 OK
   Pool: 1

•  minio2.wang.org:9000
   Uptime: offline
   Drives: 0/4 OK

•  minio3.wang.org:9000
   Uptime: 14 minutes
   Version: 2023-10-16T04:13:43Z
   Network: 2/3 OK
   Drives: 4/4 OK
   Pool: 1

Pools:
   1st, Erasure sets: 1, Drives per erasure set: 12

3.7 GiB Used, 1 Bucket, 1 Object
1 node offline, 8 drives online, 4 drives offline

#数据写入仍然进行,完成后,可以看到不同节点的数据空间不同
[root@minio1 ~]#df -h /data/
Filesystem                    Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-minio   20G  8.1G   11G  44% /data

[root@minio2 ~]#df -h /data
Filesystem                    Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-minio   20G  2.5G   17G  14% /data

[root@minio3 ~]#df -h /data
Filesystem                    Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-minio   20G   11G  8.1G  57% /data

#模拟磁盘损坏
[root@minio2 ~]#rm -rf  /data/minio*
[root@minio2 ~]#mkdir /data/minio{1..4}
[root@minio2 ~]#chown -R minio.minio  /data/minio{1..4}

#恢复故障节点的服务
[root@minio2 ~]#systemctl start minio

#多次执行空间查看,可以看到数据在同步中
[root@minio2 ~]#df -h /data/
Filesystem                    Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-minio   20G  3.2G   16G  18% /data

[root@minio2 ~]#df -h /data/
Filesystem                    Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-minio   20G  3.4G   16G  19% /data

[root@minio2 ~]#df -h /data/
Filesystem                    Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-minio   20G  3.9G   15G  21% /data

[root@minio2 ~]#df -h /data/
Filesystem                    Size  Used Avail Use% Mounted on
/dev/mapper/ubuntu--vg-minio   20G  8.9G  9.9G  48% /data
```

```
#集群状态恢复
[root@ubuntu2204 ~]#mc admin  info minio-cluster
●  minio1.wang.org:9000
   Uptime: 11 minutes
   Version: 2023-10-16T04:13:43Z
   Network: 3/3 OK
   Drives: 4/4 OK
   Pool: 1

●  minio2.wang.org:9000
   Uptime: 5 minutes
   Version: 2023-10-16T04:13:43Z
   Network: 3/3 OK
   Drives: 4/4 OK
   Pool: 1

●  minio3.wang.org:9000
   Uptime: 22 minutes
   Version: 2023-10-16T04:13:43Z
   Network: 3/3 OK
   Drives: 4/4 OK
   Pool: 1

Pools:
   1st, Erasure sets: 1, Drives per erasure set: 12

13 GiB Used, 1 Bucket, 2 Objects
12 drives online, 0 drives offline
```

4.2 节点故障重新安装系统恢复故障

范例: 3节点集群中一个节点彻底故障并重新安装进行恢复

```
#发现一台节点出故障
[root@ubuntu2204 ~]#mc admin  info minio-cluster
●  minio1.wang.org:9000
   Uptime: 3 hours
   Version: 2023-10-16T04:13:43Z
   Network: 2/3 OK
   Drives: 4/4 OK
   Pool: 1

●  minio2.wang.org:9000  #故障节点
   Uptime: offline
   Drives: 0/4 OK

●  minio3.wang.org:9000
   Uptime: 3 hours
   Version: 2023-10-16T04:13:43Z
   Network: 2/3 OK
   Drives: 4/4 OK
   Pool: 1

Pools:
   1st, Erasure sets: 1, Drives per erasure set: 12

2.6 MiB Used, 1 Bucket, 4 Objects
```

```
1 node offline, 8 drives online, 4 drives offline


#在所有节点上修改/etc/hosts文件中用新节点的IP替代故障节点的IP
[root@minio1 ~]#vim /etc/hosts
10.0.0.101 minio1.wang.org
10.0.0.104 minio2.wang.org  #原主机名保留,更新节点的IP
10.0.0.103 minio3.wang.org


[root@minio1 ~]#for i  in {2..3};do scp /etc/hosts minio$i.wang.org:/etc/;done

#修改反向代理配置,替换故障节点的地址为新节点
过程略

#安装一台新的节点,参考2.3.2.1小节:范例: 二进制安装MinIO 实现3节点4磁盘的分布式集群部署
过程略

#在所有节点上重启服务
[root@minio1 ~]#systemctl restart minio.service

#验证节点恢复
[root@ubuntu2204 ~]#mc admin  info minio-cluster
●  minio1.wang.org:9000
   Uptime: 9 seconds
   Version: 2023-10-16T04:13:43Z
   Network: 3/3 OK
   Drives: 4/4 OK
   Pool: 1

●  minio2.wang.org:9000
   Uptime: 9 seconds
   Version: 2023-10-16T04:13:43Z
   Network: 3/3 OK
   Drives: 0/4 OK
   Pool: 1

●  minio3.wang.org:9000
   Uptime: 9 seconds
   Version: 2023-10-16T04:13:43Z
   Network: 3/3 OK
   Drives: 4/4 OK
   Pool: 1

Pools:
   1st, Erasure sets: 1, Drives per erasure set: 12

13 GiB Used, 1 Bucket, 2 Objects
12 drives online, 0 drives offline

#在新节点上发现数据恢复
[root@ubuntu2204-107 ~]#tree /data/
/data/
├── lost+found
├── minio1
|   └── mybucket
|       ├── example-object1.txt
|       |   └── xl.meta
```

```
|       ├── example-object2.txt
|       |   └── xl.meta
|       └── example-object3.txt
|           └── xl.meta
├── minio2
|   └── mybucket
|       ├── example-object1.txt
|       |   └── xl.meta
|       ├── example-object2.txt
|       |   └── xl.meta
|       └── example-object3.txt
|           └── xl.meta
├── minio3
|   └── mybucket
|       ├── example-object1.txt
|       |   └── xl.meta
|       ├── example-object2.txt
|       |   └── xl.meta
|       └── example-object3.txt
|           └── xl.meta
└── minio4
    └── mybucket
        ├── example-object1.txt
        |   └── xl.meta
        ├── example-object2.txt
        |   └── xl.meta
        └── example-object3.txt
            └── xl.meta

21 directories, 12 files
```

5 MINIO 扩容和缩容

5.1.扩容

通常当MinIO容量使用到70%时,建议考虑进行扩容。

扩容有两种方案

- 通过LVM逻辑卷扩容,前提安装时事先使用LVM
- 通过一个相同规格的集群扩容

5.1.1 基于 LVM 扩容

使用lvm逻辑卷进行扩容minio集群，通过在原有节点增加硬盘来进行扩容minio集群

如果通过lvm逻辑卷扩容，部署原有集群时,minio数据目录需要使用lvm逻辑卷

lvm逻辑卷单个的大小不要超过2T，文件系统过大会导致minio集群IO降低

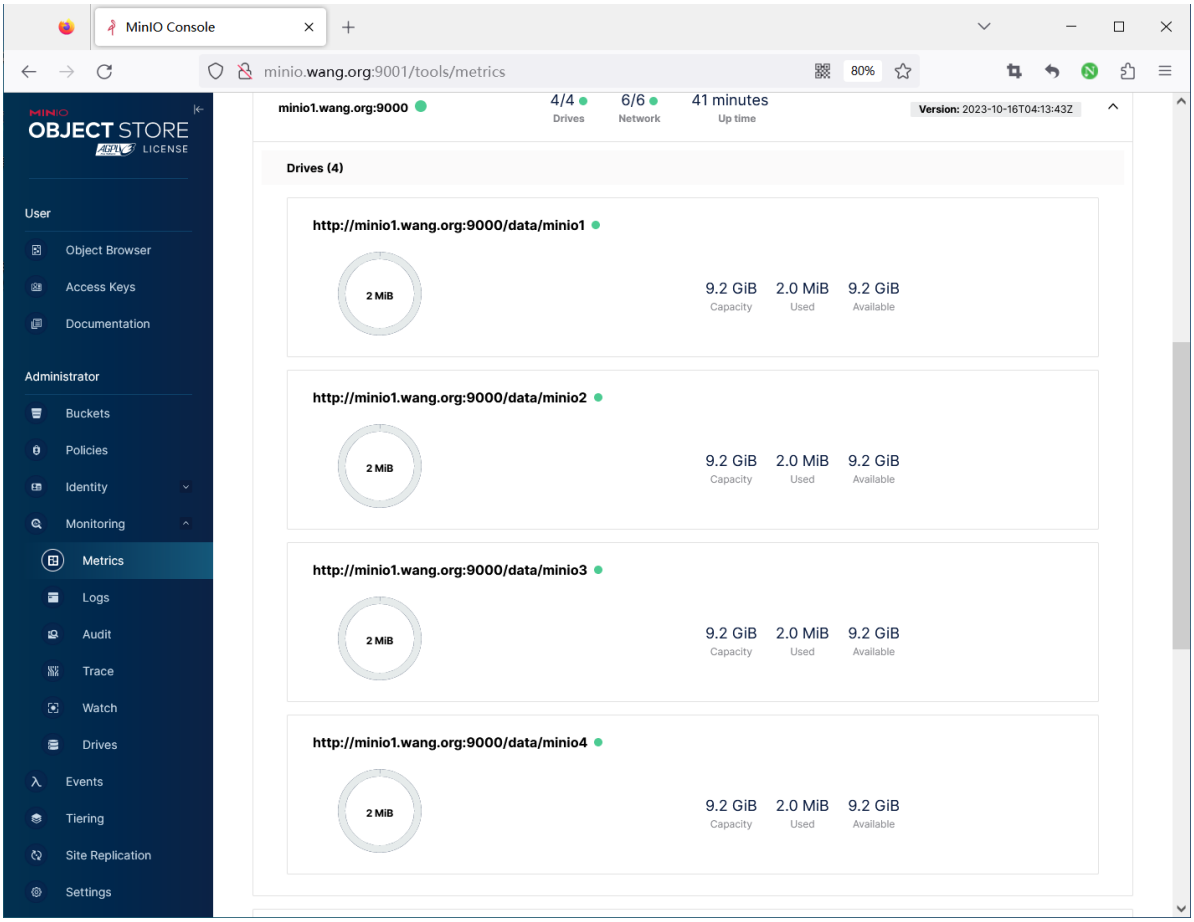
例如：部署原有MinIO集群时使用4块1T的磁盘使用逻辑卷的方式部署四个数据目录，之后扩容时再增加4块1T新的磁盘
把四个数据目录扩容到2T，相应的minio集群容量也会扩容1倍

范例: 添加新磁盘扩容

```
#添加新磁盘到pv
#pvcreate /dev/sdc

#把pv添加到卷组
#vgextend minio /dev/sdc

#扩容逻辑卷
lvresize -r -l +100%FREE /dev/ubuntu-vg/minio
```

范例: 基于2.3.2.1 环境LVM扩容

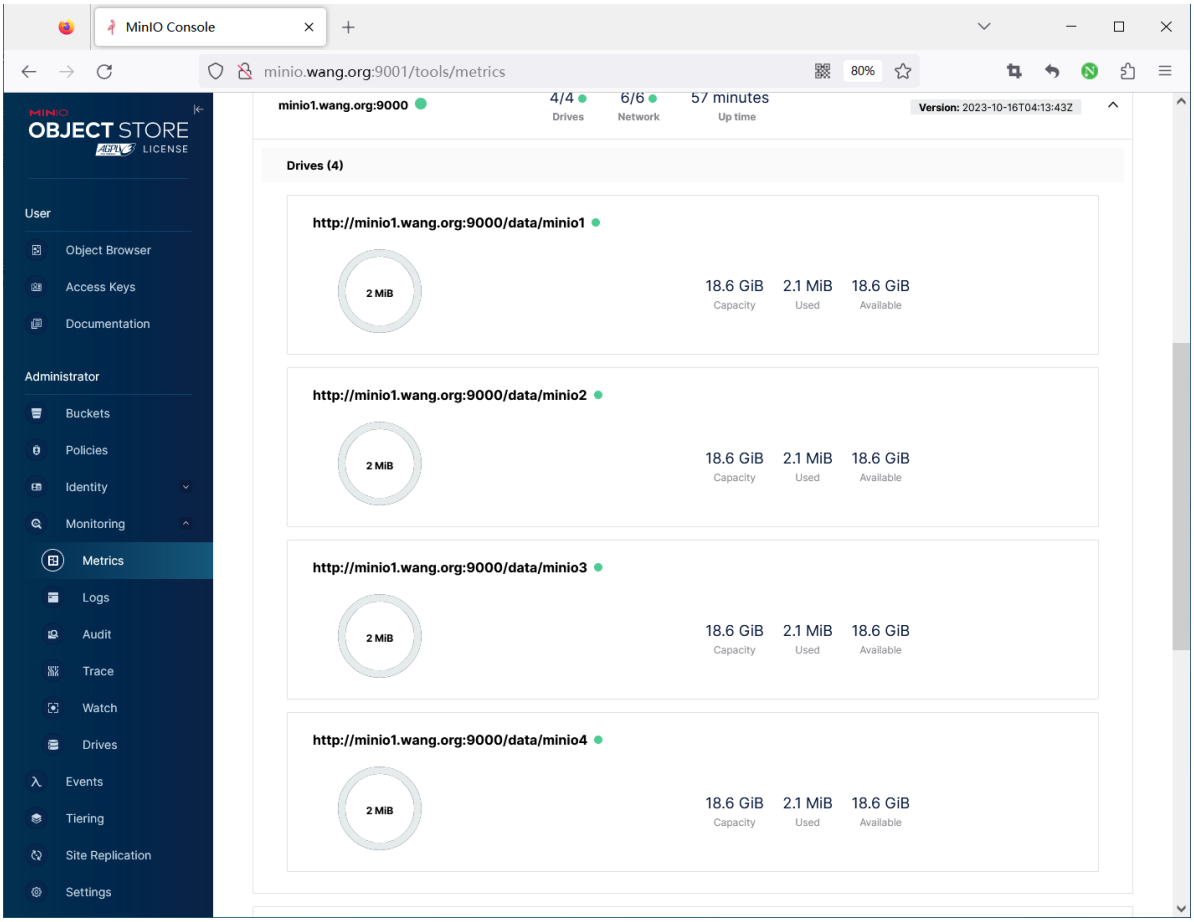
扩容前查看如下



在所有节点上执行下面操作扩容

```
[root@minioX ~]#lvextend -r -L +10G /dev/ubuntu-vg/minio
```

扩容后,可查看到如下效果



5.1.2 通过增加相同规格的集群节点扩容

扩容流程

- 扩容需要准备一个跟原有minio服务器相同规格的集群，进行服务器初始化以及安装minio
- 需要所有minio服务器的配置文件进行修改，并重启所有节点的服务
- 最后负载均衡服务中添加新节点服务器

注意

- 集群扩容时，需要新准备一个和原有集群相同或整数倍的节点和磁盘资源。

  比如原集群为8节点16个数据盘,则可以扩展为16节点32数据盘,或者24个节点48块磁盘
- 扩容节点的数据盘大小配置要跟原有集群一致
- 新加的所有节点必须是没旧MinIO数据新系统，如果有需要清除干净

范例：扩展现有的分布式集群

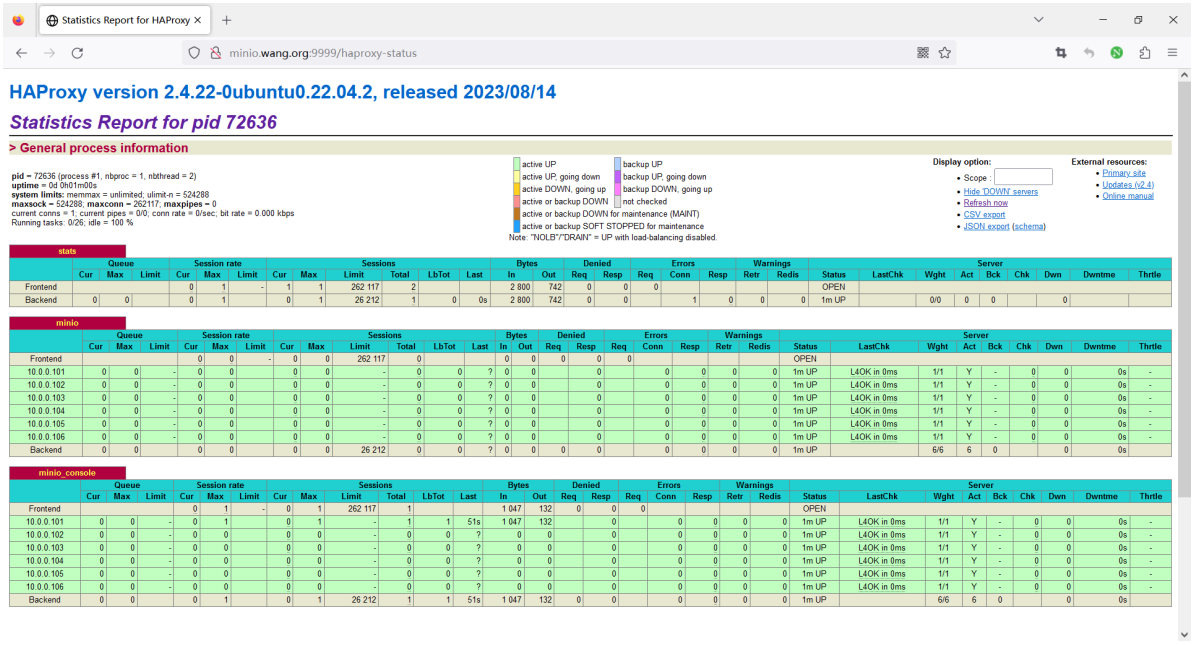
```
#例如通过区的方式启动MinIO集群，共1024块磁盘，命令行如下：
export MINIO_ROOT_USER=admin
export MINIO_ROOT_PASSWORD=12345678
minio server http://host{1...32}/export{1...32}

#MinIO支持通过命令，指定新的集群来扩展现有集群（纠删码模式），扩展后磁盘为2048块，命令行如下：
export MINIO_ROOT_USER=admin
export MINIO_ROOT_PASSWORD=12345678
minio server http://host{1...32]/export{1...32] http://host{33...64
}/export{1...32]

#新的对象上传请求会自动分配到最少使用的集群节点上。通过以上扩展策略，您就可以按需扩展您的集群。重新配置后重启集群，会立即在集群中生效，并对现有集群无影响。如上命令中，我们可以把原来的集群看做一个区，新增集群看做另一个区，新对象按每个区域中的可用空间比例放置在区域中。在每个区域内，基于确定性哈希算法确定位置。

#说明：您添加的每个区域必须具有与原始区域相同的磁盘数量（纠删码集）大小，以便维持相同的数据冗余SLA。例如，第一个区有8个磁盘，您可以将集群扩展为16个、32个或1024个磁盘的区域，您只需确保部署的SLA是原始区域的倍数即可。
```

范例: 基于 2.3.2.1 基于LVM实现二进制部署MinIO 实现3节点的分布式高可用集群的环境扩容

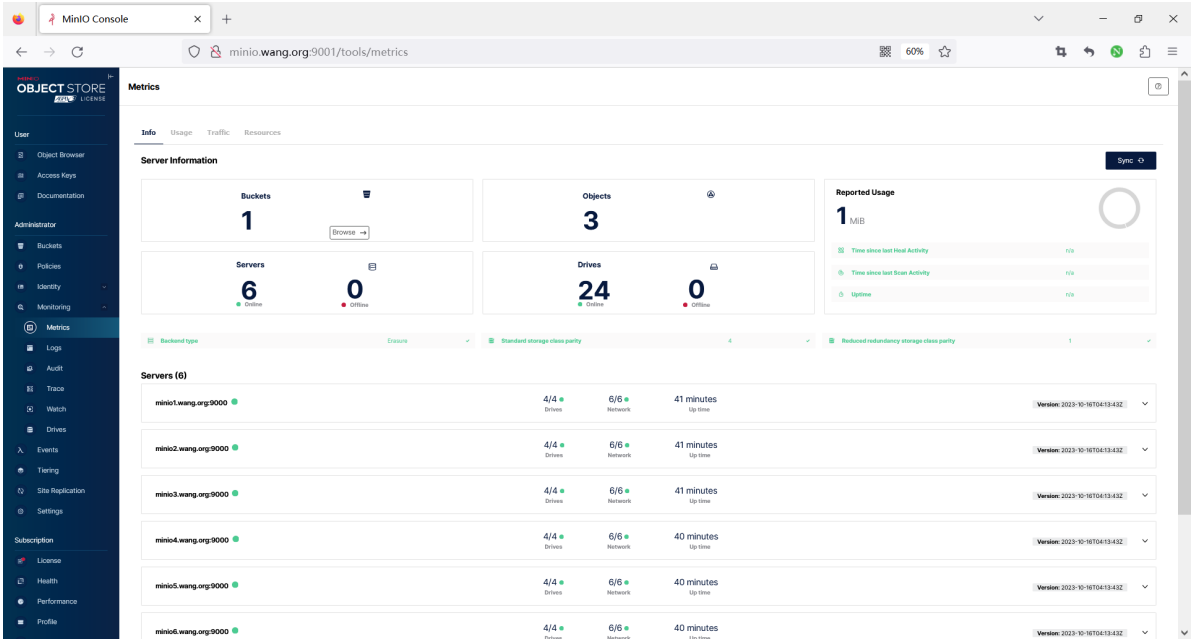
```
#在所有节点修改hosts文件
[root@ubuntu2204 ~]#vim  /etc/hosts
10.0.0.101 minio1.wang.org
10.0.0.102 minio2.wang.org
10.0.0.103 minio3.wang.org
#添加下面三行
10.0.0.104 minio4.wang.org
10.0.0.105 minio5.wang.org
10.0.0.106 minio6.wang.org

#在新添加的节点上执行
[root@minio4 ~]#lvcreate -n minio -l 100%free  ubuntu-vg
#[root@minio4 ~]#lvcreate -n minio -L 10G ubuntu-vg
[root@minio4 ~]#mkfs.ext4 /dev/ubuntu-vg/minio
[root@minio4 ~]#mkdir /data/
[root@minio4 ~]#echo /dev/ubuntu-vg/minio /data/ ext4 defaults 0 0 >> /etc/fstab
[root@minio4 ~]#mount -a
[root@minio4 ~]#mkdir -p /data/minio{1..4}

#在新添加的节点上准备用户和权限
[root@minio4 ~]#useradd -r -s /sbin/nologin minio
[root@minio4 ~]#chown -R minio: /data/minio{1..4}

#在新添加的节点上准备程序
[root@minio4 ~]#scp minio1.wang.org:/usr/local/bin/minio /usr/local/bin/minio

#在所有节点上修改配置文件
[root@minio1 ~]#vim /etc/default/minio
MINIO_ROOT_USER=admin
MINIO_ROOT_PASSWORD=12345678
#只修改下面行
MINIO_VOLUMES='http://minio{1...3}.wang.org:9000/data/minio{1...4}
http://minio{4...6}.wang.org:9000/data/minio{1...4}'
MINIO_OPTS='--console-address :9001'
```

```
MINIO_PROMETHEUS_AUTH_TYPE="public"

#复制配置到新的节点上
[root@minio4 ~]#scp minio1.wang.org:/etc/default/minio /etc/default/minio

#复制service配置到新的节点上
[root@minio4 ~]#scp minio1.wang.org:/lib/systemd/system/minio.service
/lib/systemd/system/minio.service
[root@minio4 ~]#cat /lib/systemd/system/minio.service
[Unit]
Description=MinIO
Documentation=https://docs.min.io
Wants=network-noline.target
After=network-noline.target

[Service]
WorkingDirectory=/usr/local
User=minio
Group=minio
EnvironmentFile=-/etc/default/minio
ExecStartPre=/bin/bash -c "if [ -z \"${MINIO_VOLUMES}\" ]; then echo \"Variable
MINIO_VOLUMES not set in /etc/default/minio\"; exit 1; fi"
ExecStart=/usr/local/bin/minio server $MINIO_OPTS $MINIO_VOLUMES
Restart=always
[Install]
WantedBy=multi-user.target

#在所有节点上重启服务
[root@minio4 ~]#systemctl daemon-reload
[root@minio4 ~]#systemctl enable --now minio.service
[root@minio4 ~]#systemctl restart minio.service

#查看日志
[root@minio1 ~]#tail -f /var/log/syslog
Oct 19 12:02:09 minio1 minio[75077]: Waiting for all MinIO sub-systems to be
initialize...
Oct 19 12:02:09 minio1 minio[75077]: Automatically configured API requests per
node based on available memory on the system: 15
Oct 19 12:02:09 minio1 minio[75077]: All MinIO sub-systems initialized
successfully in 14.008095ms
......

#修改haproxy代理配置
[root@ubuntu2204 ~]#vim /etc/haproxy/haproxy.cfg
.....
listen minio
    bind 10.0.0.100:9000
    mode http
    log global
    server 10.0.0.101 10.0.0.101:9000 check inter 3000 fall 2 rise 5
    server 10.0.0.102 10.0.0.102:9000 check inter 3000 fall 2 rise 5
    server 10.0.0.103 10.0.0.103:9000 check inter 3000 fall 2 rise 5
    server 10.0.0.104 10.0.0.104:9000 check inter 3000 fall 2 rise 5
    server 10.0.0.105 10.0.0.105:9000 check inter 3000 fall 2 rise 5
    server 10.0.0.106 10.0.0.106:9000 check inter 3000 fall 2 rise 5

listen minio_console
    bind 10.0.0.100:9001
```

```
    mode http
    log global
    server 10.0.0.101 10.0.0.101:9001 check inter 3000 fall 2 rise 5
    server 10.0.0.102 10.0.0.102:9001 check inter 3000 fall 2 rise 5
    server 10.0.0.103 10.0.0.103:9001 check inter 3000 fall 2 rise 5
    server 10.0.0.104 10.0.0.104:9001 check inter 3000 fall 2 rise 5
    server 10.0.0.105 10.0.0.105:9001 check inter 3000 fall 2 rise 5
    server 10.0.0.106 10.0.0.106:9001 check inter 3000 fall 2 rise 5

[root@ubuntu2204 ~]#systemctl reload haproxy

#登录Haproxy,查看状态
```



#登录控制台,可看到扩容成功



5.2 缩容

如果不是LVM，缩容相当于重新部署集群

缩容前所有节点的数据备份,再在配置中删除上面的添加的节点,只保留部分节点,重启服务后,再恢复数据

注意：缩容需要重建数据和Access Key 信息

```
#查看当前状态
[root@ubuntu2204 ~]#mc admin  info minio-cluster
●  minio1.wang.org:9000
   Uptime: 3 minutes
   Version: 2023-10-16T04:13:43Z
   Network: 6/6 OK
   Drives: 4/4 OK
   Pool: 1

●  minio2.wang.org:9000
   Uptime: 30 seconds
   Version: 2023-10-16T04:13:43Z
   Network: 6/6 OK
   Drives: 4/4 OK
   Pool: 1

●  minio3.wang.org:9000
   Uptime: 3 minutes
   Version: 2023-10-16T04:13:43Z
   Network: 6/6 OK
   Drives: 4/4 OK
   Pool: 1

●  minio4.wang.org:9000
   Uptime: 3 minutes
   Version: 2023-10-16T04:13:43Z
   Network: 6/6 OK
   Drives: 4/4 OK
   Pool: 2

●  minio5.wang.org:9000
   Uptime: 3 minutes
   Version: 2023-10-16T04:13:43Z
   Network: 6/6 OK
   Drives: 4/4 OK
   Pool: 2

●  minio6.wang.org:9000
   Uptime: 3 minutes
   Version: 2023-10-16T04:13:43Z
   Network: 6/6 OK
   Drives: 4/4 OK
   Pool: 2

Pools:
   1st, Erasure sets: 1, Drives per erasure set: 12
   2nd, Erasure sets: 1, Drives per erasure set: 12

3.9 MiB Used, 1 Bucket, 6 Objects
24 drives online, 0 drives offline

#备份数据
略

#在所有节点修改配置
```

```
[root@minio1 ~]#vim /etc/default/minio
MINIO_ROOT_USER=admin
MINIO_ROOT_PASSWORD=12345678
#MINIO_VOLUMES='http://minio{1...3}.wang.org:9000/data/minio{1...4}
http://minio{4...6}.wang.org:9000/data/minio{1...4}'
MINIO_VOLUMES='http://minio{1...3}.wang.org:9000/data/minio{1...4}' #只保留部分节点
MINIO_OPTS='--console-address :9001'
MINIO_PROMETHEUS_AUTH_TYPE="public"

#关闭删除的节点服务,在集群剩余的所有节点上重启服务
[root@minio1 ~]#systemctl restart minio

#可看集群状态缩容成功
[root@ubuntu2204 ~]#mc admin  info minio-cluster
●  minio1.wang.org:9000
   Uptime: 4 minutes
   Version: 2023-10-16T04:13:43Z
   Network: 3/3 OK
   Drives: 4/4 OK
   Pool: 1

●  minio2.wang.org:9000
   Uptime: 4 minutes
   Version: 2023-10-16T04:13:43Z
   Network: 3/3 OK
   Drives: 4/4 OK
   Pool: 1

●  minio3.wang.org:9000
   Uptime: 4 minutes
   Version: 2023-10-16T04:13:43Z
   Network: 3/3 OK
   Drives: 4/4 OK
   Pool: 1

Pools:
   1st, Erasure sets: 1, Drives per erasure set: 12

0 B Used, 1 Bucket, 0 Objects
12 drives online, 0 drives offline

#还原数据
略
```

6 MINIO 备份还原

6.1 备份还原说明

6.1.1 rclone 工具介绍

Rclone，即"rsync for cloud storage"，号称云存储的瑞士军刀

Rclone是一款专业的用于管理和同步云储存数据的开源命令行工具。通过该工具，用户不仅可以在各类型云盘之间拷贝、同步数据，还能支持资源占用低，速度快，稳定性好等一系列优点。

rclone 由Golang编写，旨在提供在不同平台的文件系统和多种类型的对象存储产品之间的数据同步功能。

rclone支持超过40种不同的云存储服务，如 Amazon S3、Google Drive、Dropbox、Microsoft OneDrive、Google Cloud Storage等，它提供了多种文件传输方式，包括复制、同步、移动和删除文件，并支持文件加密和压缩。

此外，rclone还支持分块上传和下载，允许传输过程中断后恢复，以及文件的校验和合并。它的主要优势在于灵活性和可扩展性，可以用于备份、文件同步、数据迁移等多种场景，并在Windows、macOS、Linux、FreeBSD、NetBSD等平台上均可运行。

Rclone 具有强大的云等同于 unix 命令 rsync、cp、mv、mount、ls、ncdu、tree、rm 和 cat。Rclone 熟悉的语法包括 shell 管道支持和--dry-run保护。它在命令行、脚本或通过其API 使用。

Rclone多种文件传输协议，支持SFTP，HTTP，WebDAV，FTP和DLNA。Rclone是一个成熟的开源软件，最初受rsync的启发并采用Golang编写。其文档和社区也都非常好，提供广泛和友好的使用用例。

rclone的配置简单，可以通过命令行或配置文件进行设置，它提供了丰富的命令行功能，使得操作云存储非常方便。此外，rclone还提供mount操作，允许将远端对象存储挂载到本地文件系统进行访问。

使用rclone工具进行数据同步，把minio集群bucket的对象数据同步到备份服务器

Rclone 配置

配置可以直接添加配置文件的方式或者通过进入交互式配置会话命令一步步的完成配置。

默认配置完成的后配置文件都保存在：`~/.config/rclone/rclone.conf` 目录下。

```
[tencent-cos] # 自定义的配置名称
type = s3  # 存储类型，参考官方文档所有支持的类型
provider = TencentCOS  # 提供商，参考官方文档或者全部
env_auth = false   # 不通过环境变量配置认证
access_key_id = AKxxxxxxxx  # 腾讯云后台生成的密钥key
secret_access_key = Secretxxxxxxx # 腾讯云后台生成的密钥secret
endpoint = cos.ap-chengdu.myqcloud.com # 腾讯云cos所在的地区，看你所在存储桶的公网地址
```

Rclone 语法

```
# 本地同步到网盘
rclone [功能选项] <本地路径> <配置名称:路径> [参数] [参数]

# 网盘同步到本地
rclone [功能选项] <配置名称:路径> <本地路径> [参数] [参数]

# 网盘同步到网盘
rclone [功能选项] <配置名称:路径> <配置名称:路径> [参数] [参数]

# [参数]为可选项

#示例：
#同步本地/data/file的文件夹内容到tencent-cos存储下的/backup文件夹中,并且排除/root/excludes.txt中指定的文件内容
rclone sync /data/file tencent-cos:/backup --exclude-from '/root/backup_file.txt'

# 两个网盘文件同步
rclone copy 配置网盘名称1:网盘路径 配置网盘名称2:网盘路径
```

Rclone 命令列表

使用 `rclone --help` 可查看所有命令，这里只列出常用的命令

命令	说明
rclone copy	复制
rclone move	移动，如果要在移动后删除空源目录，加上 --delete-empty-src-dirs 参数,会删除源文件类似于MV命令
rclone mount	挂载
rclone sync	同步：将源目录同步到目标目录，只更改目标目录
rclone size	查看网盘文件占用大小
rclone delete	删除路径下的文件内容
rclone mkdir	创建目录
rclone rmdir	删除目录
rclone rmdirs	删除指定环境下的空目录。如果加上 --leave-root 参数，则不会删除根目录
rclone check	检查源和目的地址数据是否匹配
rclone ls	列出指定路径下的所有的文件以及文件大小和路径
rclone lsl	比上面多一个显示上传时间
rclone lsd	列出指定路径下的目录
rclone lsf	列出指定路径下的目录和文件

范例: 安装和配置 rclone

```
#准备rclone的配置文件
[root@ubuntu2404 ~]#apt list rclone
Listing... Done
rclone/noble-security,noble-updates,noble-updates,noble-security 1.60.1+dfsg-
3ubuntu0.24.04.2 amd64
N: There is 1 additional version. Please use the '-a' switch to see it
[root@ubuntu2404 ~]#apt update && apt -y install rclone
[root@ubuntu2404 ~]#rclone version
rclone v1.60.1-DEV
- os/version: ubuntu 24.04 (64 bit)
- os/kernel: 6.8.0-48-generic (x86_64)
- os/type: linux
```

```
- os/arch: amd64
- go/version: go1.22.2
- go/linking: dynamic
- go/tags: none


[root@ubuntu2204 ~]#apt update && apt -y install rclone

#实现自动补全
#方法1
[root@ubuntu2404 ~]#echo 'source <(rclone completion bash)' >> ~/.bashrc;exit
#方法2
[root@ubuntu2404 ~]#rclone  completion bash > /etc/bash_completion.d/rclone;exit

#创建配置
#方式1：通过提示自动生成配置
[root@ubuntu2404 ~]#rclone config
Current remotes:

Name                 Type
====                 ====
minio                s3
minio-cluster        s3

e) Edit existing remote
n) New remote
d) Delete remote
r) Rename remote
c) Copy remote
s) Set configuration password
q) Quit config
e/n/d/r/c/s/q>n


#方法2：手动准备配置文件
[root@ubuntu2204 ~]#mkdir  -p .config/rclone
[root@ubuntu2204 ~]#cat >  .config/rclone/rclone.conf
[minio]
type = s3
provider = Minio
evn_auth = false
#access_key_id = admin
#secret_access_key = 12345678
access_key_id = eesdfHZp7abOSUZCIFWq
secret_access_key = szncc I8iITaQMp9ZzJYmv7HOw8ompCAPF9jvFtpE
region = beijing-1
endpoint = http://minio.wang.org:9000

#验证配置
[root@ubuntu2204 ~]#rclone lsd minio:
          -1 2023-10-17 18:10:16        -1 mybucket
```

6.1.2 备份还原过程

注意事项

- 备份服务器要求存储空间至少满足minio集群存储空间的一半

  比如：minio集群存储空间一共48T,则备份服务器需要24T的空间来备份数据
- 第一次进行全量同步，之后进行增量同步
- 可以使用md5校验来确定是否同步数据，根据自身需求选择，md5校验会cpu消耗较高，但是确定对象文件是否改变要比其他方式更准确
- 建议在业务负载较低时执行备份
- 如果数据量大时不建议进行备份，因为本身minio集群已经实现高可用

具体备份还原流程

```
#方式1：通过提示自动生成配置
[root@ubuntu2404 ~]#rclone config
Current remotes:

Name                 Type
====                 ====
minio                s3
minio-cluster        s3

e) Edit existing remote
n) New remote
d) Delete remote
r) Rename remote
c) Copy remote
s) Set configuration password
q) Quit config
e/n/d/r/c/s/q>n


#方法2：手动准备配置文件
[root@minio01 ~]# vim ~/.config/rclone/rclone.conf
[minio] #minio集群名称
type = s3
provider = Minio
evn_auth = false
access_key_id = admin
secret_access_key = 12345678
region = beijing-1
endpoint = http://minio.wang.org:9000

#验证配置命令
rclone lsd <minio集群名称>:

#备份命令
#全量备份
rclone sync <minio集群名称>:/<bucket名称> <备份目录> -vP

#增量备份,使用md5判断文件是否需要重新同步
rclone sync <minio集群名称>:/<bucket名称> <备份目录>  -vP --checksum

#备份选项
```

```
--transfers=N - 并行文件数，默认为4，根据内存，以及带宽调整
--buffer-size=SIZE 加速 sync命令，使用内存缓存大小
--bwlimit UP:DOWN 上传下载限速b|k|M|G,在业务需要时可根据需求设置
--checksum 通过md5判断文件是否有需要同步，消耗cpu比较高
--update --use-server-modtime 通过mtime判断文件是否需要同步,Skip files that are newer
on the destination

#推荐同步备份数据命令
rclone sync <minio集群名称>:/<bucket名称> <备份目录> --transfers=8 --update -v -P

#推荐恢复备份数据命令
rclone sync <备份目录> <minio集群名称>:/<bucket名称> --transfers=8 --update -v --
logfile=rclone.log
```

6.2 备份还原案例

6.2.1 数据备份

范例: 备份

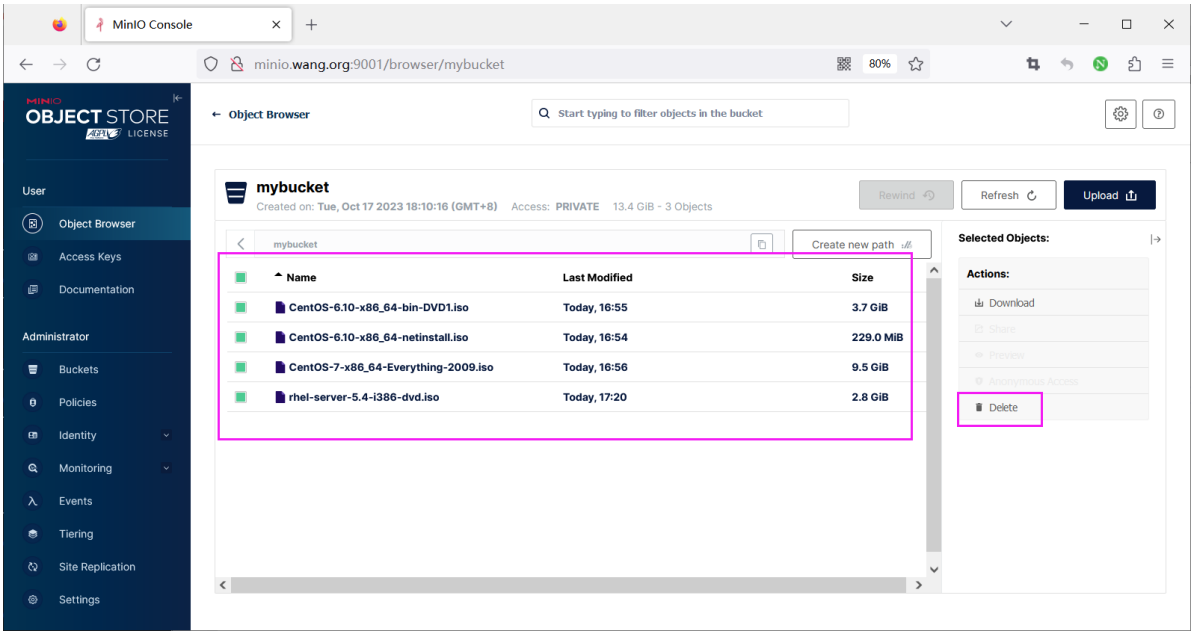
```
#创建连接
[root@ubuntu2204 ~]#mc config host add minio-cluster http://minio.wang.org:9000
admin 12345678
[root@ubuntu2204 ~]#mc config host ls
gcs
  URL       : https://storage.googleapis.com
  AccessKey : YOUR-ACCESS-KEY-HERE
  SecretKey : YOUR-SECRET-KEY-HERE
  API       : S3v2
  Path      : dns

local
  URL       : http://10.0.0.100:9000
  AccessKey : admin
  SecretKey : 12345678
  API       : s3v4
  Path      : auto

minio-cluster
  URL       : http://minio.wang.org:9000
  AccessKey : admin
  SecretKey : 12345678
  API       : s3v4
  Path      : auto

play
  URL       : https://play.min.io
  AccessKey : Q3AM3UQ867SPQQA43P2F
  SecretKey : zuf+tfteSlswRu7BJ86wekitnifILbZam1KYY3TG
  API       : S3v4
  Path      : auto

s3
  URL       : https://s3.amazonaws.com
  AccessKey : YOUR-ACCESS-KEY-HERE
  SecretKey : YOUR-SECRET-KEY-HERE
```

```
  API       : S3v4
  Path      : dns

#查看备份的bucket的大小
[root@ubuntu2204 ~]#mc ls minio-cluster
[2023-10-17 18:10:16 CST]     0B mybucket/

[root@ubuntu2204 ~]#mc du  minio-cluster
13GiB   3 objects
[root@ubuntu2204 ~]#mc du  minio-cluster/mybucket
13GiB   3 objects   mybucket

#准备rclone的配置文件
[root@ubuntu2204 ~]#apt update && apt -y install rclone
[root@ubuntu2204 ~]#mkdir  -p .config/rclone
[root@ubuntu2204 ~]#cat >  .config/rclone/rclone.conf
[minio]
type = s3
provider = Minio
evn_auth = false
access_key_id = admin
secret_access_key = 12345678
region = beijing-1
endpoint = http://minio.wang.org:9000

#验证配置
[root@ubuntu2204 ~]#rclone lsd minio:
          -1 2023-10-17 18:10:16        -1 mybucket

#第一次执行完全备份
#说明:备份目录/data/minio-bak/会自动创建,无需事先创建
[root@ubuntu2204 ~]#rclone sync minio:/mybucket /data/minio-bak/ -vP
2023-10-19 17:17:08 INFO  : CentOS-6.10-x86_64-netinstall.iso: Copied (new)
2023-10-19 17:18:18 INFO  : CentOS-6.10-x86_64-bin-DVD1.iso: Multi-thread Copied
(new)
2023-10-19 17:18:42 INFO  : CentOS-7-x86_64-Everything-2009.iso: Multi-thread
Copied (new)
Transferred:       13.441G / 13.441 GBytes, 100%, 130.502 MBytes/s, ETA 0s
Transferred:            3 / 3, 100%
Elapsed time:      1m45.5s
2023/10/19 17:18:42 INFO  :
Transferred:       13.441G / 13.441 GBytes, 100%, 130.502 MBytes/s, ETA 0s
Transferred:            3 / 3, 100%
Elapsed time:      1m45.5s

#查看备份后的文件
[root@ubuntu2204 ~]#ls /data/minio-bak/
CentOS-6.10-x86_64-bin-DVD1.iso    CentOS-7-x86_64-Everything-2009.iso
CentOS-6.10-x86_64-netinstall.iso

#在MinIO新上传一个文件后,执行增量备份
[root@ubuntu2204 ~]#mc cp rhel-server-5.4-i386-dvd.iso minio-server/mybucket

#执行增量备份
[root@ubuntu2204 ~]#rclone sync minio:/mybucket /data/minio-bak/ -vP --checksum
2023-10-19 17:21:23 INFO  : rhel-server-5.4-i386-dvd.iso: Multi-thread Copied
(new)
```

```
Transferred:        2.799G / 2.799 GBytes, 100%, 64.035 MBytes/s, ETA 0s  #只备份变化文件
Checks:                 3 / 3, 100%
Transferred:            1 / 1, 100%
Elapsed time:         44.8s
2023/10/19 17:21:38 INFO  :
Transferred:        2.799G / 2.799 GBytes, 100%, 64.035 MBytes/s, ETA 0s
Checks:                 3 / 3, 100%
Transferred:            1 / 1, 100%
Elapsed time:         44.8s

#查看备份后的文件
[root@ubuntu2204 ~]#ls /data/minio-bak/
CentOS-6.10-x86_64-bin-DVD1.iso    CentOS-7-x86_64-Everything-2009.iso
CentOS-6.10-x86_64-netinstall.iso  rhel-server-5.4-i386-dvd.iso
```

6.2.2 数据还原

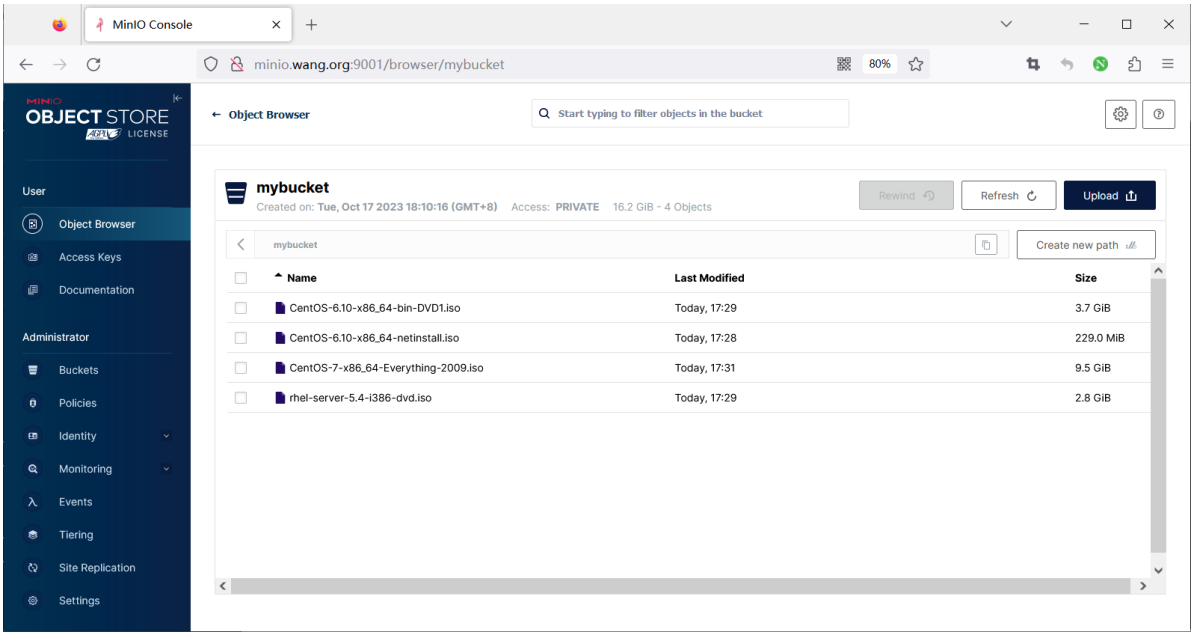
```
#在控制台删除MinIO文件
```




```
#查看数据被清空
[root@ubuntu2204 ~]#mc du  minio-cluster/mybucket
0B  0 objects   mybucket

#还原数据
[root@ubuntu2204 ~]#rclone sync /data/minio-bak/ minio:/mybucket --transfers=8 --update -v -P
2023-10-19 17:28:47 INFO  : CentOS-6.10-x86_64-netinstall.iso: Copied (new)
2023-10-19 17:29:47 INFO  : rhel-server-5.4-i386-dvd.iso: Copied (new)
2023-10-19 17:30:02 INFO  : CentOS-6.10-x86_64-bin-DVD1.iso: Copied (new)
2023-10-19 17:31:31 INFO  : CentOS-7-x86_64-Everything-2009.iso: Copied (new)
Transferred:       16.241G / 16.241 GBytes, 100%, 98.526 MBytes/s, ETA 0s
Transferred:            4 / 4, 100%
Elapsed time:      2m48.8s
2023/10/19 17:31:31 INFO  :
Transferred:       16.241G / 16.241 GBytes, 100%, 98.526 MBytes/s, ETA 0s
Transferred:            4 / 4, 100%
Elapsed time:      2m48.8s
```

```
#验证数据还原
[root@ubuntu2204 ~]#mc du  minio-cluster/mybucket
16GiB   4 objects   mybucket
```

登录控制台,验证数据还原



7 MINIO 监控

7.1. MINIO 监控说明

```
https://min.io/docs/minio/linux/operations/monitoring/collect-minio-metrics-using-prometheus.html?ref=docs-redirect
```

minio监控内置支持Prometheus,推荐使用Prometheus+grafana进行监控

```
#minio集群grafana模板
https://grafana.com/grafana/dashboards/13502-minio-dashboard/

scrape_configs:
#监控MinIO集群
- job_name: minio-job
  #bearer_token: <secret>
  metrics_path: /minio/v2/metrics/cluster
  scheme: http
  static_configs:
  - targets: ['minio.wang.org:9000']

#minio节点主机grafana模板:8919,1860,11074,13978模板
https://grafana.com/grafana/dashboards/13978-node-exporter-quickstart-and-dashboard/?pg=dashboards&plcmt=featured-sub1
```

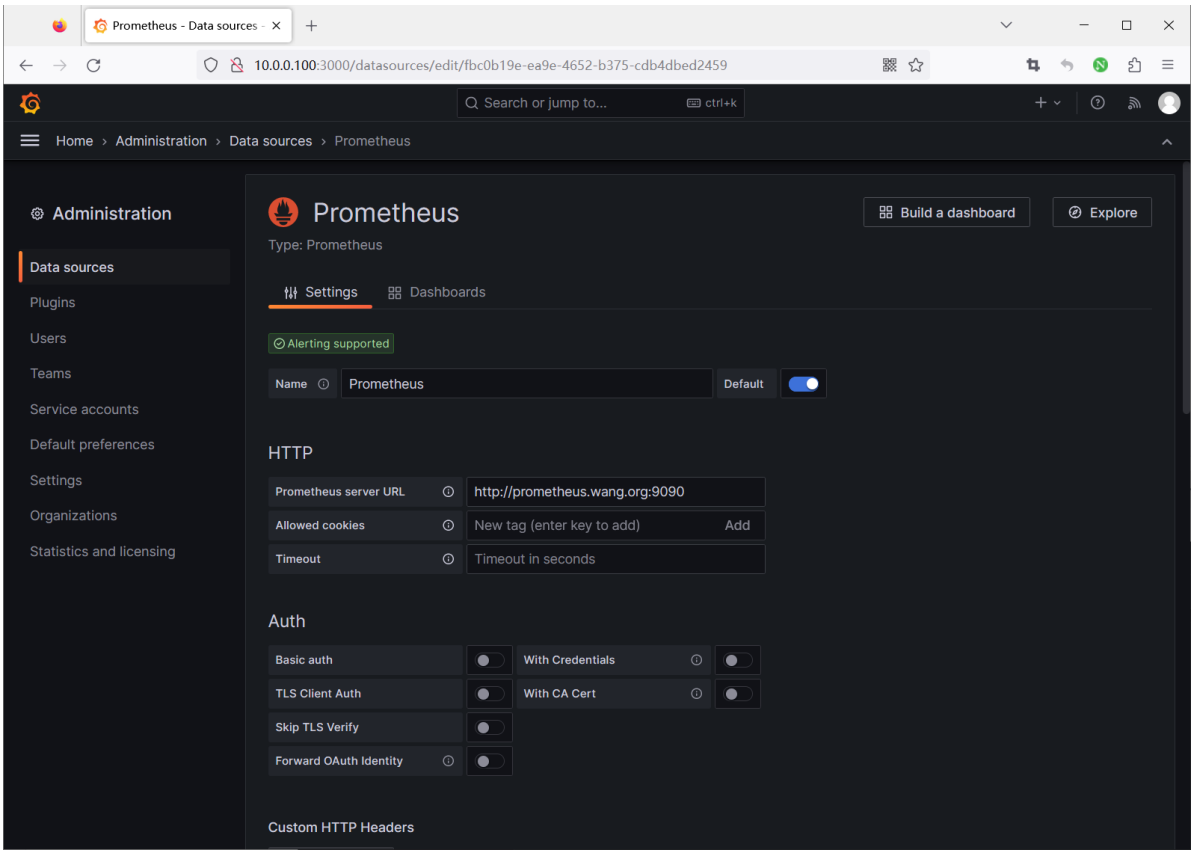
7.2 MINIO 通过 Prometheus 和 Grafana 监控案例

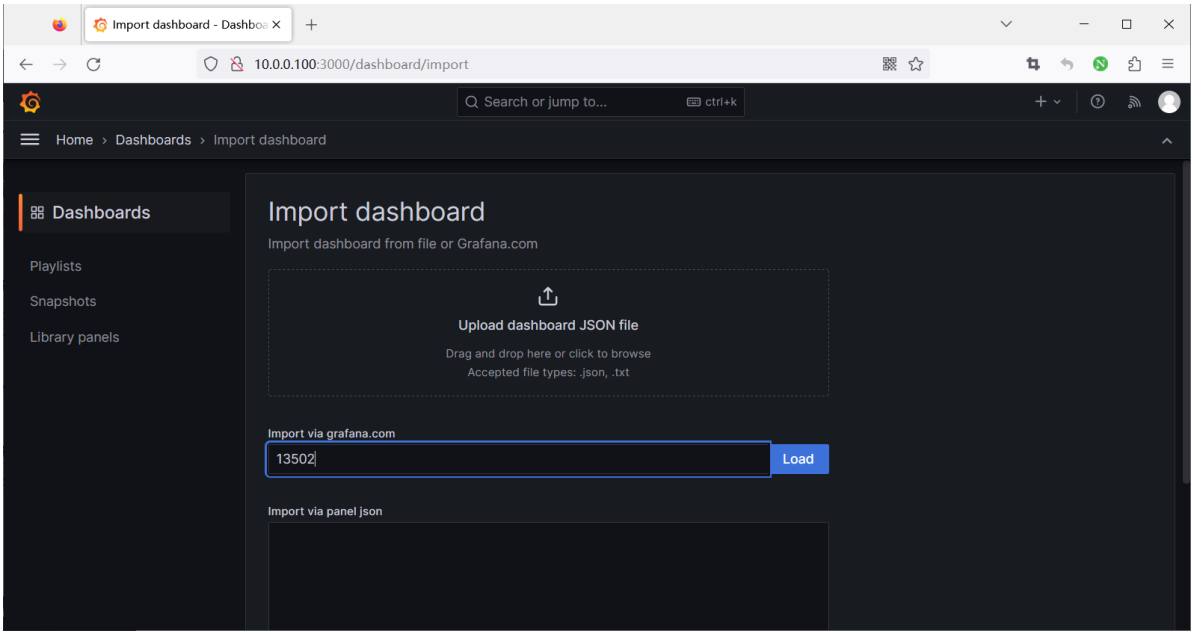
范例:

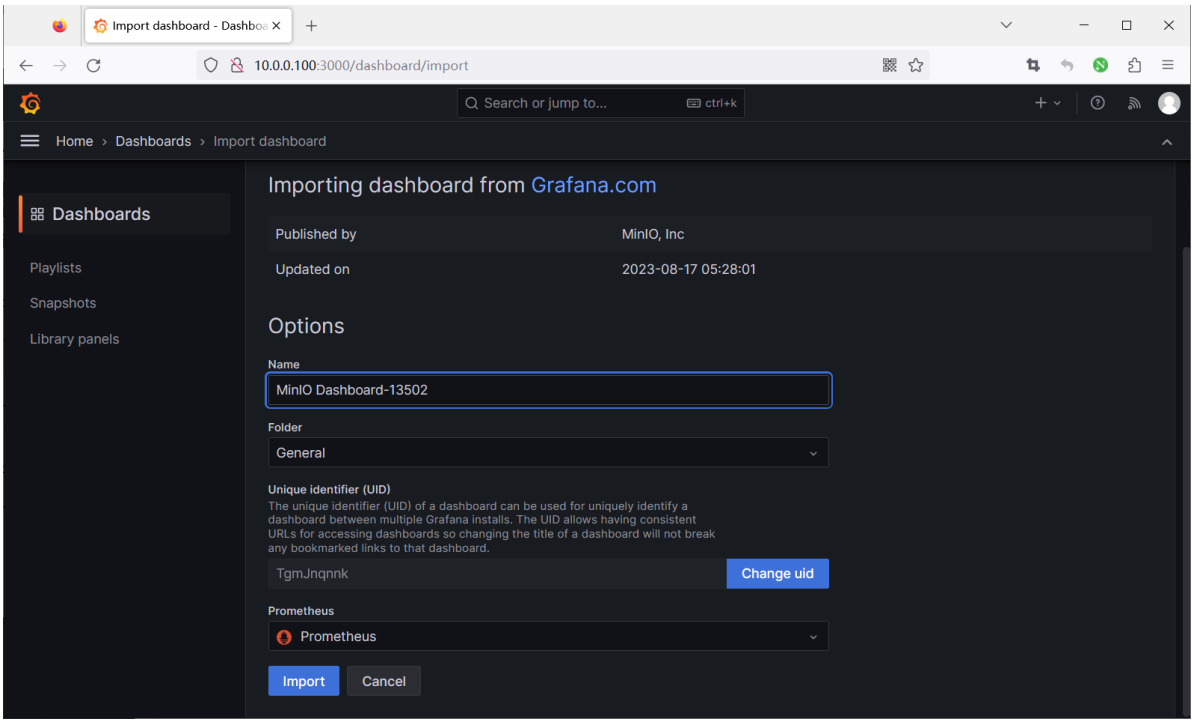
```
[root@ubuntu2204 ~]#curl -s  http:minio.wang.org:9000/minio/v2/metrics/cluster
|head
# HELP minio_audit_failed_messages Total number of messages that failed to send
since start
# TYPE minio_audit_failed_messages counter
minio_audit_failed_messages{server="minio1.wang.org:9000",target_id="sys_console
_0"} 0
minio_audit_failed_messages{server="minio2.wang.org:9000",target_id="sys_console
_0"} 1
minio_audit_failed_messages{server="minio3.wang.org:9000",target_id="sys_console
_0"} 0
minio_audit_failed_messages{server="minio4.wang.org:9000",target_id="sys_console
_0"} 0
minio_audit_failed_messages{server="minio5.wang.org:9000",target_id="sys_console
_0"} 1
minio_audit_failed_messages{server="minio6.wang.org:9000",target_id="sys_console
_0"} 0
# HELP minio_audit_target_queue_length Number of unsent messages in queue for
target
# TYPE minio_audit_target_queue_length gauge


#利用MC工具添加MinIO连接
[root@ubuntu2204 ~]#mc config host add minio-cluster http://minio.wang.org:9000
admin 12345678
Added `minio-cluster` successfully.

[root@ubuntu2204 ~]#mc admin  info minio-cluster
●  minio1.wang.org:9000
   Uptime: 2 hours
   Version: 2023-10-16T04:13:43Z
   Network: 6/6 OK
   Drives: 4/4 OK
   Pool: 1

●  minio2.wang.org:9000
   Uptime: 2 hours
   Version: 2023-10-16T04:13:43Z
   Network: 6/6 OK
   Drives: 4/4 OK
   Pool: 1

●  minio3.wang.org:9000
   Uptime: 2 hours
   Version: 2023-10-16T04:13:43Z
   Network: 6/6 OK
   Drives: 4/4 OK
   Pool: 1

●  minio4.wang.org:9000
   Uptime: 2 hours
   Version: 2023-10-16T04:13:43Z
   Network: 6/6 OK
   Drives: 4/4 OK
   Pool: 2

●  minio5.wang.org:9000
   Uptime: 2 hours
```

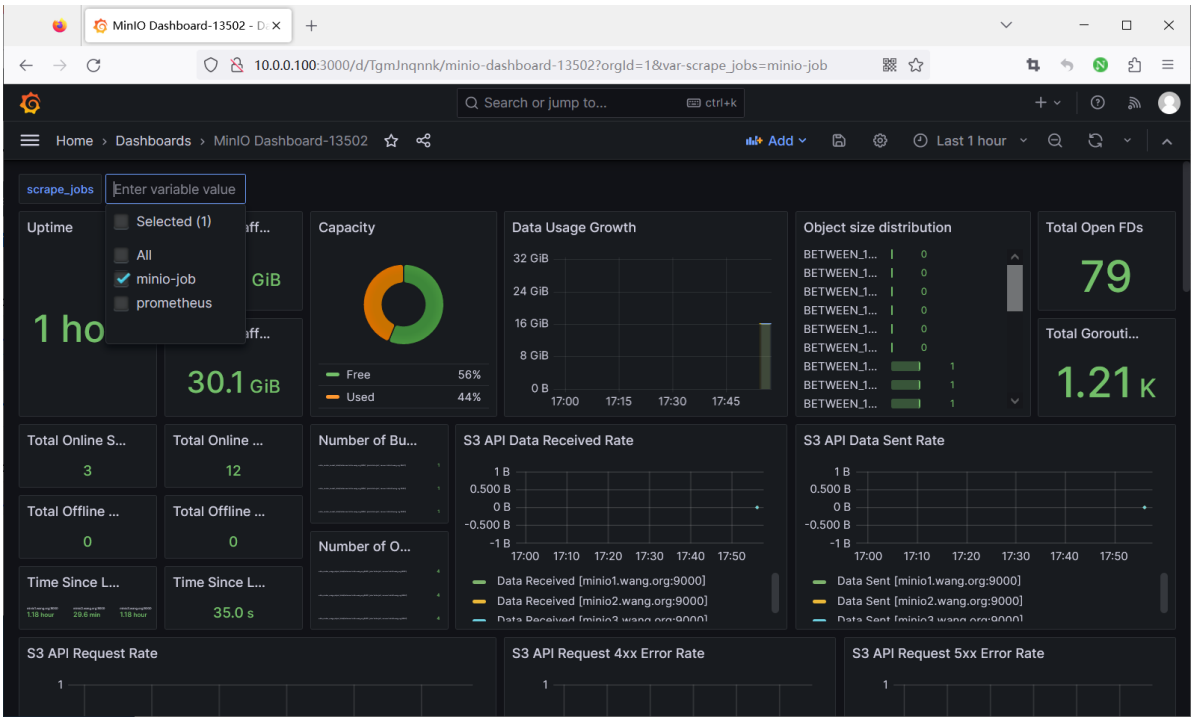
```
   Version: 2023-10-16T04:13:43Z
   Network: 6/6 OK
   Drives: 4/4 OK
   Pool: 2

●  minio6.wang.org:9000
   Uptime: 2 hours
   Version: 2023-10-16T04:13:43Z
   Network: 6/6 OK
   Drives: 4/4 OK
   Pool: 2

Pools:
   1st, Erasure sets: 1, Drives per erasure set: 12
   2nd, Erasure sets: 1, Drives per erasure set: 12

2.6 MiB Used, 1 Bucket, 4 Objects
24 drives online, 0 drives offline


#生成Promethus配置
[root@ubuntu2204 ~]#mc admin  prometheus  generate  minio-cluster
scrape_configs:
- job_name: minio-job
  bearer_token:
eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJwcm9tZXRoZXVzIiwic3ViIjoiYWRtaW4
iLCJleHAiOjQ4NTEyOTYzODJ9.iKHE1uXVPKYAixlJBAIpnb2btslig6WNZz6KQqSWBv7qUnNfYqnqIU
dq1f6mMAmq_G3pfU_pEJmpPfl1jRrQaw
  metrics_path: /minio/v2/metrics/cluster
  scheme: http
  static_configs:
  - targets: ['minio.wang.org:9000']


#安装prometheus和grafana
过程略

#配置Prometheus
[root@ubuntu2204 ~]#vim /usr/local/prometheus/conf/prometheus.yml
......
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped
from this config.
  - job_name: "prometheus"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:9090"]

#添加下面内容
  - job_name: minio-job
    bearer_token:
eyJhbGciOiJIUzUxMiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJwcm9tZXRoZXVzIiwic3ViIjoiYWRtaW4
iLCJleHAiOjQ4NTEyOTYzODJ9.iKHE1uXVPKYAixlJBAIpnb2btslig6WNZz6KQqSWBv7qUnNfYqnqIU
dq1f6mMAmq_G3pfU_pEJmpPfl1jRrQaw
    metrics_path: /minio/v2/metrics/cluster
```

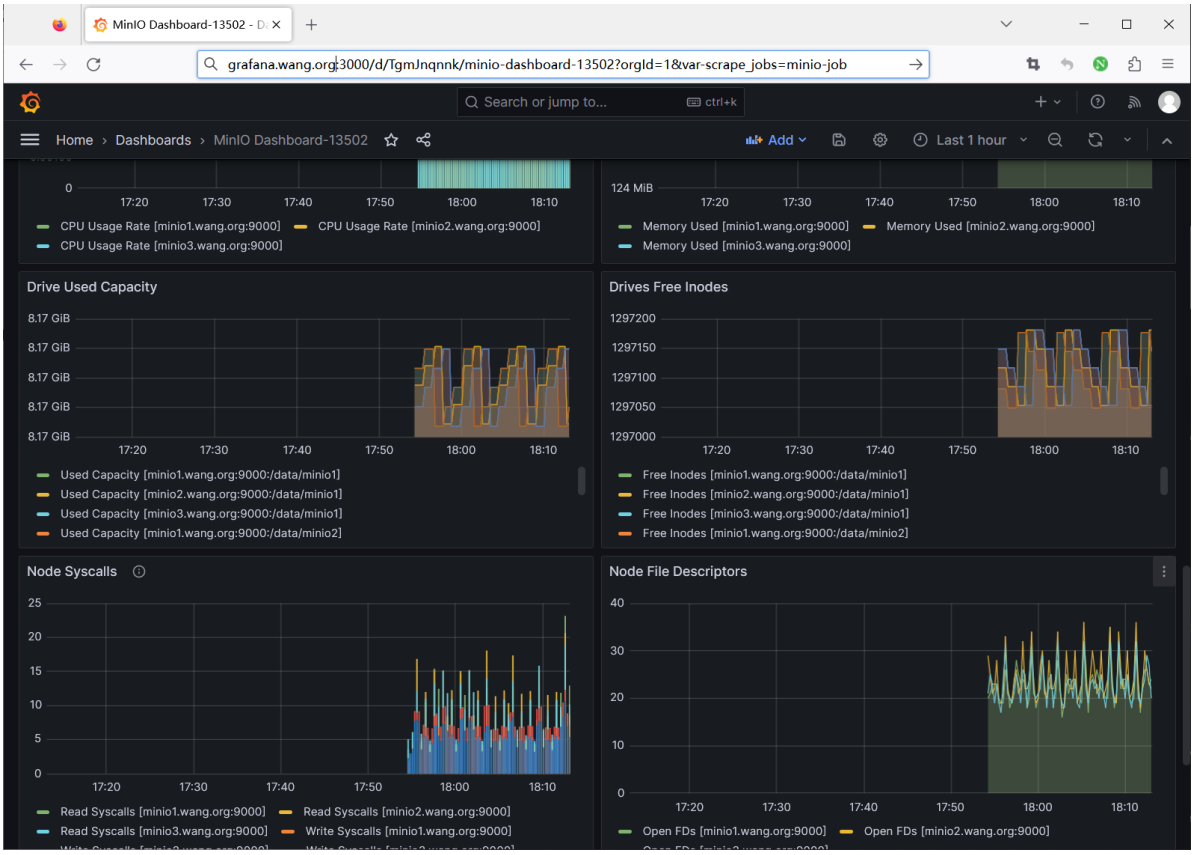
```
    scheme: http
    static_configs:
    - targets: ['minio.wang.org:9000']

[root@ubuntu2204 ~]#systemctl reload prometheus.service

#安装grafana
[root@ubuntu2404 ~]#wget
https://mirrors.tuna.tsinghua.edu.cn/grafana/apt/pool/main/g/grafana/grafana_11.4.0_amd64.deb
[root@ubuntu2204 ~]#wget
https://mirrors.tuna.tsinghua.edu.cn/grafana/apt/pool/main/g/grafana/grafana_11.3.0_amd64.deb
[root@ubuntu2204 ~]#wget
https://mirrors.tuna.tsinghua.edu.cn/grafana/apt/pool/main/g/grafana/grafana_11.1.3_amd64.deb
[root@ubuntu2204 ~]#apt install ./grafana_11.1.3_amd64.deb
[root@ubuntu2204 ~]#systemctl enable --now  grafana-server

#使用默认用户/密码:admin/admin
http://10.0.0.100:3000/


#在Grafana添加Prometheus数据源和导入13502模板
```




Import dashboard - Dashbo
10.0.0.100:3000/dashboard/import
Search or jump to... ctrl+k
Home › Dashboards › Import dashboard
Dashboards
Playlists
Snapshots
Library panels
Import dashboard
Import dashboard from file or Grafana.com
Upload dashboard JSON file
Drag and drop here or click to browse
Accepted file types: .json, .txt
Import via grafana.com
13502
Load
Import via panel json


Import dashboard - Dashbo
10.0.0.100:3000/dashboard/import
Search or jump to... ctrl+k
Home › Dashboards › Import dashboard
Dashboards
Playlists
Snapshots
Library panels
Importing dashboard from Grafana.com
Published by
MinIO, Inc
Updated on
2023-08-17 05:28:01
Options
Name
MinIO Dashboard-13502
Folder
General
Unique identifier (UID)
The unique identifier (UID) of a dashboard can be used for uniquely identify a dashboard between multiple Grafana installs. The UID allows having consistent URLs for accessing dashboards so changing the title of a dashboard will not break any bookmarked links to that dashboard.
TgmJnqnnk
Change uid
Prometheus
Prometheus
Import
Cancel


MinIO Dashboard-13502 - D
10.0.0.100:3000/d/TgmJnqnnk/minio-dashboard-13502?orgId=1&var-scrape_jobs=minio-job
Search or jump to... ctrl+k
Home › Dashboards › MinIO Dashboard-13502
Add
Last 1 hour
scrape_jobs
Enter variable value
Selected (1)
All
minio-job
prometheus
Uptime
Capacity
Free 56%
Used 44%
Data Usage Growth
Object size distribution
Total Open FDs
79
Total Gorouti...
1.21 K
30.1 GiB
Total Online S...
3
Total Online ...
12
Total Offline ...
0
Total Offline ...
0
Time Since L...
Time Since L...
35.0 s
Number of Bu...
Number of O...
S3 API Data Received Rate
S3 API Data Sent Rate
Data Received [minio1.wang.org:9000]
Data Received [minio2.wang.org:9000]
Data Sent [minio1.wang.org:9000]
Data Sent [minio2.wang.org:9000]
S3 API Request Rate
S3 API Request 4xx Error Rate
S3 API Request 5xx Error Rate


MinIO Dashboard-13502 - D
grafana.wang.org:3000/d/TgmJnqnnk/minio-dashboard-13502?orgId=1&var-scrape_jobs=minio-job
Search or jump to... ctrl+k
Home › Dashboards › MinIO Dashboard-13502
Add
Last 1 hour
0
17:20 17:30 17:40 17:50 18:00 18:10
CPU Usage Rate [minio1.wang.org:9000]
CPU Usage Rate [minio2.wang.org:9000]
CPU Usage Rate [minio3.wang.org:9000]
124 MiB
17:20 17:30 17:40 17:50 18:00 18:10
Memory Used [minio1.wang.org:9000]
Memory Used [minio2.wang.org:9000]
Memory Used [minio3.wang.org:9000]
Drive Used Capacity
8.17 GiB
8.17 GiB
8.17 GiB
8.17 GiB
8.17 GiB
17:20 17:30 17:40 17:50 18:00 18:10
Used Capacity [minio1.wang.org:9000:/data/minio1]
Used Capacity [minio2.wang.org:9000:/data/minio1]
Used Capacity [minio3.wang.org:9000:/data/minio1]
Used Capacity [minio1.wang.org:9000:/data/minio2]
Drives Free Inodes
1297200
1297150
1297100
1297050
1297000
17:20 17:30 17:40 17:50 18:00 18:10
Free Inodes [minio1.wang.org:9000:/data/minio1]
Free Inodes [minio2.wang.org:9000:/data/minio1]
Free Inodes [minio3.wang.org:9000:/data/minio1]
Free Inodes [minio1.wang.org:9000:/data/minio2]
Node Syscalls
25
20
15
10
5
0
17:20 17:30 17:40 17:50 18:00 18:10
Read Syscalls [minio1.wang.org:9000]
Read Syscalls [minio2.wang.org:9000]
Read Syscalls [minio3.wang.org:9000]
Write Syscalls [minio1.wang.org:9000]
Node File Descriptors
40
30
20
10
0
17:20 17:30 17:40 17:50 18:00 18:10
Open FDs [minio1.wang.org:9000]
Open FDs [minio2.wang.org:9000]